

Vinícius Augusto Landim Andrade

Implementação de Compilador C-

São José dos Campos - Brasil

Julho de 2026

Vinícius Augusto Landim Andrade

Implementação de Compilador C-

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Julho de 2026

Lista de ilustrações

Figura 1 – Diagrama de blocos da Unidade de Processamento	8
Figura 2 – Caminho de dados do Processador MIPS 32	9
Figura 3 – Diagrama de blocos do Processador MIPS 32	10
Figura 4 – Diagrama de blocos da Fase de Análise	16
Figura 5 – Diagrama de atividades da Fase de Análise	17
Figura 6 – Árvore Sintática Abstrata construída na análise sintática do código gcd.cm	20
Figura 7 – Tabela de símbolos construída para o código sort.cm	23
Figura 8 – Diagrama de blocos da Geração de Código Intermediário	27
Figura 9 – Diagrama de atividades da Geração de Código Intermediário	28
Figura 10 – Diagrama de atividades da geração de quádruplas para <i>Statements</i> . . .	31
Figura 11 – Diagrama de atividades da geração de quádruplas para <i>Expressions</i> . .	32
Figura 12 – Organização da memória de dados	35
Figura 13 – Diagrama de blocos da geração de código <i>assembly</i>	36
Figura 14 – Diagrama de atividades da geração de código <i>assembly</i>	37
Figura 15 – Diagrama de atividades da atualização de registradores em uso	38
Figura 16 – Diagrama de blocos da geração de código executável	45
Figura 17 – Diagrama de atividades da geração de código executável	46
Figura 18 – Diagrama de atividades da tradução da lista de instruções	47

Lista de tabelas

Tabela 1 – Formato de instruções do tipo R	10
Tabela 2 – Formato de instruções do tipo I	10
Tabela 3 – Formato de instruções do tipo J	10
Tabela 4 – Conjunto de instruções	11
Tabela 5 – Instruções de formato R com seus valores de opcode e funct	12
Tabela 6 – Instruções de formato I com seus valores de opcode	12
Tabela 7 – Instruções de formato J com seus valores de opcode	12
Tabela 8 – Símbolos léxicos da linguagem C-	14
Tabela 9 – Definição formal de tokens léxicos	14
Tabela 10 – Gramática formal da linguagem C-	15
Tabela 11 – Descrição das quádruplas	28
Tabela 12 – Mapeamento entre o código-fonte e o código intermediário gerado	34
Tabela 13 – Mapeamento entre o código-fonte fact.cm e o código intermediário gerado	49
Tabela 14 – Mapeamento de fact.cm : Quádrupla, Assembly e Binário	49
Tabela 15 – Mapeamento entre o código-fonte gcd.cm e o código intermediário gerado	53
Tabela 16 – Mapeamento de gcd.cm : Quádrupla, Assembly e Binário	54
Tabela 17 – Mapeamento entre o código-fonte sort.cm e o código intermediário gerado	57
Tabela 18 – Mapeamento de sort.cm : Quádrupla, Assembly e Binário	61

Sumário

1	INTRODUÇÃO	7
2	O PROCESSADOR	8
2.1	Conjunto de Instruções	11
2.2	Organização da memória	12
3	COMPILADOR: FASE DE ANÁLISE	14
3.1	Modelagem	15
3.1.1	Diagrama de Blocos	16
3.1.2	Diagrama de Atividades	17
3.2	Análise Léxica	18
3.3	Análise Sintática	19
3.4	Análise Semântica	21
3.4.1	Tabela de Símbolos	22
3.4.2	Primeira Varredura: Construção da Tabela de Símbolos	23
3.4.3	Segunda Varredura: Verificação de Tipos	25
4	COMPILADOR: FASE DE SÍNTESE	26
4.1	Geração do código intermediário	26
4.1.1	Modelagem	26
4.1.1.1	Diagrama de Blocos	26
4.1.2	Diagrama de Atividades	27
4.1.3	Definição das quádruplas	28
4.1.4	Geração das quádruplas	30
4.1.4.1	Geração de <i>Statements</i>	30
4.1.4.2	Geração de <i>Expressions</i>	32
4.1.5	Exemplo de geração de código intermediário	33
4.2	Gerenciamento de memória	34
4.3	Geração do código <i>Assembly</i>	35
4.3.1	Modelagem	35
4.4	Diagrama de Blocos	35
4.5	Diagrama de Atividades	36
4.5.1	Geração de código <i>Assembly</i> para cada quádrupla	38
4.5.2	Aritmética	38
4.5.3	Memória	39
4.5.3.1	ALLOCVAR	39

4.5.3.2	ALLOCARR	39
4.5.3.3	LOADVAR	40
4.5.3.4	LOADARR	40
4.5.3.5	LOADIMM	40
4.5.3.6	STOREVAR	41
4.5.3.7	STOREARR	41
4.5.4	Controle de Fluxo	41
4.5.5	Funções	42
4.5.5.1	FUNC e ENDFUNC	42
4.5.5.2	PARAM	42
4.5.5.3	CALL	43
4.5.5.4	RETURN	44
4.5.6	Entrada e Saída	44
4.6	Geração do código executável	45
4.6.1	Modelagem	45
4.6.2	Diagrama de Blocos	45
4.6.3	Diagrama de Atividades	46
5	EXEMPLOS	48
5.1	Fact	48
5.2	GCD	51
5.3	Sort	56
6	CONCLUSÃO	68
	REFERÊNCIAS	69
	APÊNDICES	70
	APÊNDICE A – GLOBALS.H	71
	APÊNDICE B – SCANNER.L	72
	APÊNDICE C – PARSER.Y	74
	APÊNDICE D – ANALYZE.H	82
	APÊNDICE E – ANALYZE.C	83
	APÊNDICE F – SYMTAB.H	90

APÊNDICE G – SYMTAB.C	91
APÊNDICE H – CGEN.H	95
APÊNDICE I – CGEN.C	97
APÊNDICE J – ASSEMBLY.H	106
APÊNDICE K – ASSEMBLY.C	107
APÊNDICE L – BINARY.H	117
APÊNDICE M – BINARY.C	118
APÊNDICE N – UTIL.H	122
APÊNDICE O – UTIL.C	123
APÊNDICE P – MAIN.C	128

1 Introdução

Compiladores são ferramentas fundamentais no desenvolvimento de *software*, responsáveis por traduzir programas escritos em linguagens de alto nível para representações executáveis pela máquina. Além dessa tradução, um compilador realiza a verificação estrutural e semântica do código-fonte, detectando inconsistências antes da execução e garantindo que o programa gerado seja uma representação fiel da intenção do programador. Sendo assim, a compreensão dos mecanismos internos de um compilador, desde a análise léxica até a geração de código executável, é essencial para o desenvolvimento de sistemas computacionais eficientes e corretos.

No contexto da disciplina de Laboratório de Sistemas Computacionais: Compiladores, o projeto final consiste na implementação de um compilador completo para a linguagem C-, um subconjunto da linguagem C. Tal compilador deve ser capaz de receber um programa-fonte escrito nessa linguagem e produzir, ao final do processo de compilação, o código binário correspondente, pronto para ser carregado e executado pelo processador alvo.

O processador alvo deste compilador é um processador MIPS 32 monociclo, desenvolvido em trabalhos anteriores. Trata-se de uma arquitetura RISC com separação entre memória de instruções e memória de dados, seguindo o modelo Harvard, com instruções de largura fixa em 32 bits organizadas nos formatos R, I e J. Desse modo, o compilador desenvolvido neste projeto foi projetado especificamente para esse processador, levando em consideração seu conjunto de instruções, sua organização de memória e seus registradores ao longo de todas as etapas de geração de código.

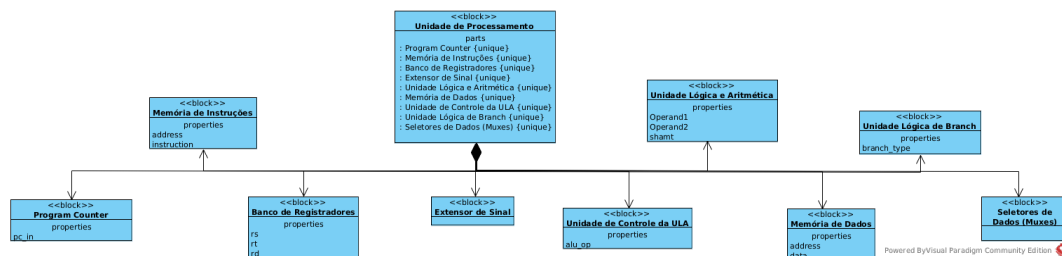
O presente relatório está organizado da seguinte forma. O Capítulo 2 descreve a arquitetura do processador alvo, apresentando seu diagrama de blocos, seu conjunto de instruções e a organização da memória de dados. O Capítulo 3 detalha a fase de análise do compilador, cobrindo as implementações léxica, sintática e semântica. O Capítulo 4 apresenta a fase de síntese, descrevendo a geração de código intermediário, a geração de código *assembly* e a tradução para código executável binário. Por fim, o Capítulo 5 apresenta exemplos de programas compilados, exibindo a correspondência entre o código-fonte, o código intermediário, o *assembly* e o binário gerado.

2 O Processador

O processador alvo utilizado neste trabalho segue a arquitetura RISC (*Reduced Instruction Set Computer*), baseada no modelo MIPS 32 monociclo descrito por [Patterson e Hennesy \(2005\)](#). A arquitetura adota um conjunto de instruções de largura fixa em 32 bits e disponibiliza 32 registradores de propósito geral, cada um com 32 bits de largura.

Em termos estruturais, a arquitetura é composta por um contador de programa (PC), responsável por armazenar o endereço da instrução corrente; uma memória de instruções, que contém o programa em execução; um banco de 32 registradores de 32 bits, com portas independentes de leitura e escrita; e uma unidade lógica e aritmética (ALU), responsável pelas operações aritméticas e lógicas do conjunto de instruções. Uma memória de dados do tipo RAM, fisicamente separada da memória de instruções, é destinada ao armazenamento das informações manipuladas durante a execução, em conformidade com a arquitetura Harvard adotada. A relação entre esses blocos e sua integração no nível de sistema é apresentada na Figura 1.

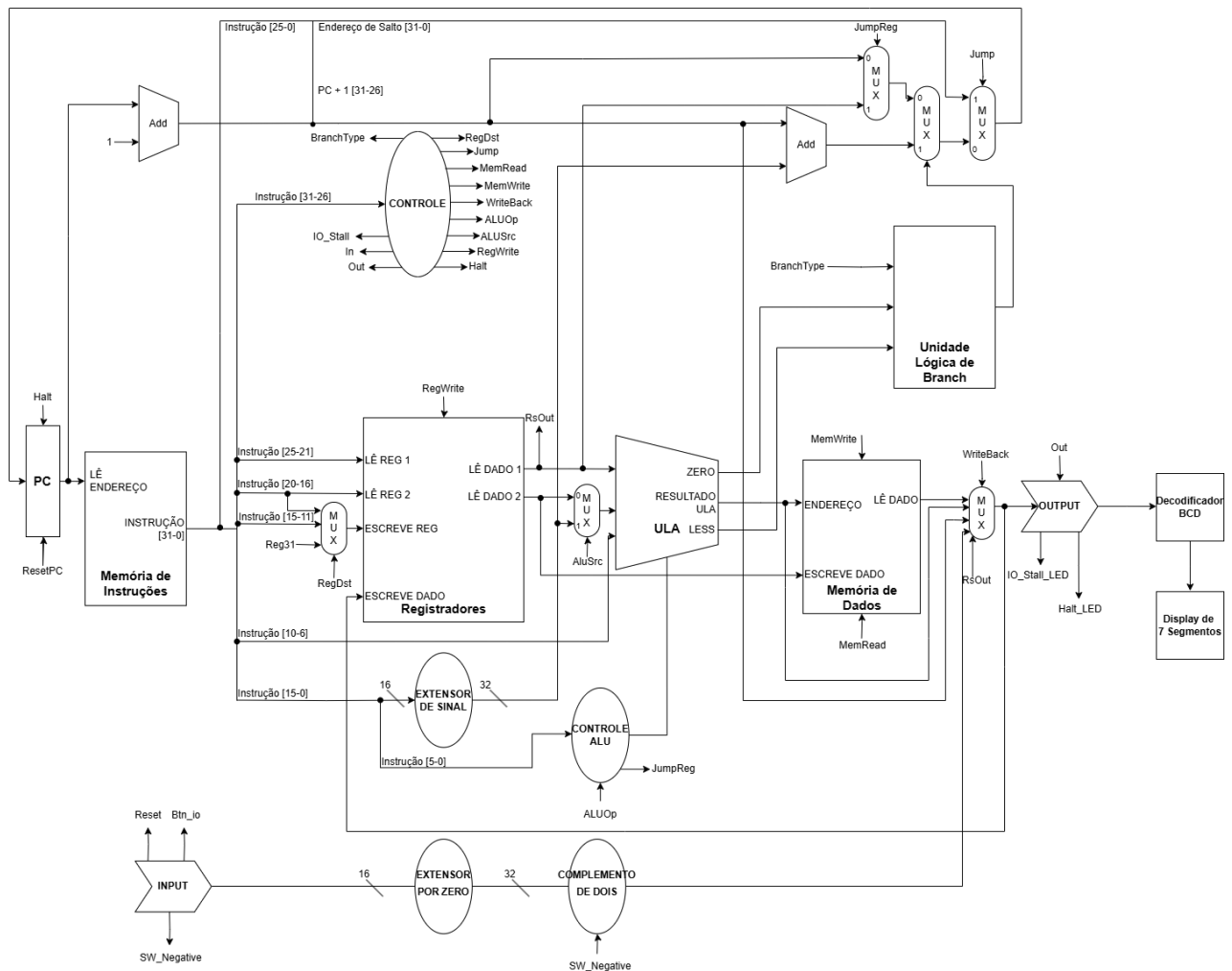
Figura 1 – Diagrama de blocos da Unidade de Processamento



Fonte: O Autor

Complementando o caminho de dados, a arquitetura conta com uma Unidade de Controle (UC) principal, responsável por decodificar o campo de opcode das instruções e gerar os sinais de controle que direcionam o fluxo de dados. Multiplexadores são utilizados em diversos pontos do caminho de dados para selecionar entre fontes alternativas de operandos e de resultados. Uma Unidade de Controle específica da ALU determina a operação a ser executada com base no campo *funct* da instrução e no sinal de controle da UC principal. Um Extensor de Sinal converte valores imediatos e de entrada de 16 para 32 bits, preservando o sinal. Por fim, uma Unidade Lógica de *Branch* avalia as condições de desvio, e somadores dedicados calculam o próximo valor do PC, seja por incremento sequencial, por desvio condicional ou por salto. A organização interna desses blocos, é detalhada na Figura 2, que apresenta o *datapath* do processador.

Figura 2 – Caminho de dados do Processador MIPS 32



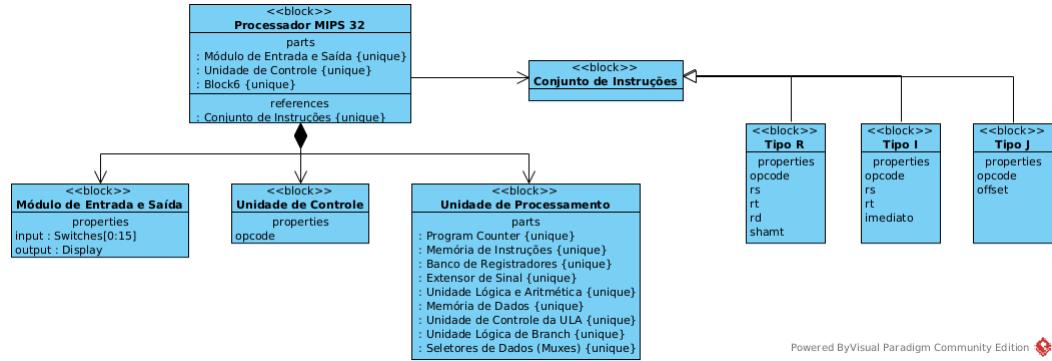
Fonte: O Autor

No nível de topo da arquitetura, ilustrado na Figura 3, o Processador MIPS 32 é composto por três blocos principais, sendo elas a Unidade de Processamento, a Unidade de Controle – já descritas até então – e o Módulo de Entrada e Saída. Este último é responsável por toda a comunicação do processador com o ambiente externo no kit FPGA, isolando o caminho de dados interno dos periféricos físicos da placa.

O Módulo de Entrada e Saída expõe duas interfaces: uma de entrada, conectada aos *switches* da placa, e uma de saída, conectada aos *displays* de 7 segmentos. A interface de entrada converte o estado das chaves físicas em um valor binário de 16 bits, disponibilizado ao restante do processador a partir de um registrador endereçado. A interface de saída realiza o caminho inverso, recebendo o conteúdo de um registrador escolhido e traduzindo-o para o formato de exibição dos *displays*, incluindo o tratamento do sinal para valores negativos. Essa separação de responsabilidades mantém o Módulo de Entrada e Saída como um bloco independente da Unidade de Processamento, comunicando-se com ela apenas

através dos sinais de controle gerados pela Unidade de Controle e dos dados trocados com o Banco de Registradores.

Figura 3 – Diagrama de blocos do Processador MIPS 32



Fonte: O Autor

O formato das instruções segue o mesmo conjunto proposto por [Patterson e Hennesy \(2005\)](#), definido em três formatos principais: tipo R, tipo I e tipo J. Os modos de endereçamento implementados também seguem essa referência, compreendendo endereçamento imediato, por registrador, por base e deslocamento, relativo ao contador de programa (PC) e pseudodireto.

Tabela 1 – Formato de instruções do tipo R

Tamanho (bits)	6	5	5	5	5	6
Campo	opcode	rs	rt	rd	shamt	funct
Bits	31 - 26	25 - 21	20 - 16	15 - 11	10 - 6	5 - 0

Fonte: O Autor

Tabela 2 – Formato de instruções do tipo I

Tamanho (bits)	6	5	5	16
Campo	opcode	rs	rt	imediato
Bits	31 - 26	25 - 21	20 - 16	15 - 0

Fonte: O Autor

Tabela 3 – Formato de instruções do tipo J

Tamanho (bits)	6	26
Campo	opcode	target
Bits	31 - 26	25 - 0

Fonte: O Autor

2.1 Conjunto de Instruções

O conjunto de instruções do processador MIPS 32 é apresentado na Tabela 4. A arquitetura original implementada no processador contempla um conjunto mais amplo de instruções; no entanto, no escopo da linguagem C — apenas o subconjunto listado é utilizado. Para cada instrução, a tabela indica a operação realizada, a descrição de sua funcionalidade e o formato — tipo R, I ou J — ao qual pertence.

Tabela 4 – Conjunto de instruções

Instrução	Operação	Funcionalidade	Tipo
Aritméticas:			
ADD	$rd = rs + rt$	Soma	R
ADDI	$rt = rs + \text{imediato}$	Soma com imediato	I
SUB	$rd = rs - rt$	Subtração	R
MUL	$rd = rs * rt$	Multiplicação	R
DIV	$rd = rs / rt$	Divisão	R
Transferência de Dados:			
LW	$rt = \text{Mem}[rs + \text{offset}]$	Leitura de uma palavra	I
SW	$\text{Mem}[rs + \text{offset}] = rt$	Armazenamento de uma palavra	I
Desvio Condicional:			
BEQ	se $rs == rt$, $PC = PC + \text{offset}$	Desvio se igual	I
BNE	se $rs \neq rt$, $PC = PC + \text{offset}$	Desvio se diferente	I
BLT	se $rs < rt$, $PC = PC + \text{offset}$	Desvio se menor	I
BGT	se $rs > rt$, $PC = PC + \text{offset}$	Desvio se maior	I
BLE	se $rs \leq rt$, $PC = PC + \text{offset}$	Desvio se menor ou igual	I
BGE	se $rs \geq rt$, $PC = PC + \text{offset}$	Desvio se maior	I
Desvio Incondicional:			
J	$PC = \text{endereço}$	Salto de endereço	J
JAL	$\text{reg} = PC$; $PC = \text{endereço}$	Chamada de sub-rotina	J
JR	$PC = \text{Reg}[rs]$	Retorno de sub-rotina	R
Controle:			
HALT	-	Parada	J
Entrada e Saída:			
IN	$rt = \text{imediato (switches)}$	Escrita de um imediato recebido do <i>input</i>	I
OUT	$\text{saída} = \text{Reg}[rs]$	Saída do conteúdo de um registrador	I

Fonte: O Autor

Definido o conjunto de instruções, é necessário conhecer a codificação binária associada a cada uma delas, uma vez que essa informação é utilizada diretamente pelo compilador na etapa de geração de código, responsável pela tradução das instruções em *assembly* para seus equivalentes em binário. As Tabelas 5, 6 e 7 apresentam, respectivamente, os valores de *opcode* — e, no caso do formato R, também de *funct* — para cada instrução dos formatos R, I e J.

Tabela 5 – Instruções de formato R com seus valores de opcode e funct

Opcode	Funct	Instrução
000000	000000	ADD
000000	000001	SUB
000000	000010	MUL
000000	000011	DIV
000000	001000	JR

Fonte: O Autor

Tabela 6 – Instruções de formato I com seus valores de opcode

Opcode	Instrução
000101	LW
000110	SW
001000	BEQ
001001	BNE
001010	BLT
001011	BGT
001100	BLE
001101	BGE
001110	IN
001111	OUT

Fonte: O Autor

Tabela 7 – Instruções de formato J com seus valores de opcode

Opcode	Instrução
010000	J
010001	JAL
010011	HALT

Fonte: O Autor

2.2 Organização da memória

A organização de memória segue o modelo Harvard, com separação física entre memória de instruções e memória de dados, permitindo o acesso simultâneo a ambas dentro de um único ciclo de operação.

A respeito da memória de instruções, ela é do tipo ROM, organizada em 32 colunas, correspondentes aos 32 bits de cada instrução, e 1024 linhas, comportando até 1024 instruções. Vale ressaltar que no processador desenvolvido a memória é *word-addressable*, o que difere do modelo *byte-addressable* de [Patterson e Hennesy \(2005\)](#). Como consequência direta para a geração de código, o incremento do contador de programa ocorre em passos de $PC + 1$, e não de $PC + 4$.

Acerca da memória de dados, ela possui 1024 posições de 32 *bits*, totalizando 4KB de capacidade. Detalhes a respeito da sua segmentação para o desenvolvimento do

compilador serão detalhados na seção de gerenciamento de memória, que será apresentada mais adiante.

3 Compilador: Fase de Análise

A fase de análise de um compilador é responsável por verificar a estrutura e a consistência do programa-fonte de acordo com as regras formais da linguagem. Essa etapa engloba a análise léxica, a análise sintática e a análise semântica. Ao longo desse processo, são construídas a Tabela de Símbolos e a Árvore Sintática Abstrata (AST), estruturas fundamentais para as fases posteriores do compilador.

A respeito do compilador desenvolvido neste projeto, adotam-se as convenções da linguagem C-, um subconjunto da linguagem C descrito por Louden e Silva (2004, p. 493-502).

Primeiramente, as convenções léxicas empregadas consistem no uso dos símbolos apresentados na Tabela 8 e dos *tokens* com regras gramaticais descritos na Tabela 9. Vale ressaltar que a linguagem C- possui algumas restrições. Uma delas é a sensibilidade a letras maiúsculas e minúsculas. Além disso, o espaço em branco, composto por espaços, quebras de linha e tabulações, é descartado pelo analisador léxico, servindo exclusivamente como delimitador entre identificadores (IDs), palavras-chave e números (NUMs). Por fim, comentários são representados pela notação `/*...*/`, podendo ocupar múltiplas linhas e ser inseridos em qualquer posição que admita espaços em branco, sendo proibido o seu aninhamento.

Tabela 8 – Símbolos léxicos da linguagem C-

Categoria	Símbolos
Palavras-chave	<code>else if int return void while</code>
Operadores aritméticos	<code>+ - * /</code>
Operadores relacionais	<code>< <= > >= == !=</code>
Operador de atribuição	<code>=</code>
Delimitadores	<code> ; , () [] { }</code>
Comentário	<code>/* */</code>

Fonte: adaptado de Louden e Silva (2004)

Tabela 9 – Definição formal de tokens léxicos

Token	Definição
ID	letra (letra)*
NUM	dígito (dígito)*
letra	<code>a-z ∪ A-Z</code>
dígito	<code>0-9</code>

Fonte: adaptado de Louden e Silva (2004)

A respeito da sintaxe da linguagem C-, ela é definida pela Forma de Backus-Naur (BNF) apresentada na Tabela 10. Cada uma das regras gramaticais apresentadas possui

um significado semântico, que serão consideradas na sua etapa de análise.

Tabela 10 – Gramática formal da linguagem C–

1. <i>programa</i>	→ <i>declaração-lista</i>
2. <i>declaração-lista</i>	→ <i>declaração-lista</i> <i>declaração</i> <i>declaração</i>
3. <i>declaração</i>	→ <i>var-declaração</i> <i>fun-declaração</i>
4. <i>var-declaração</i>	→ <i>tipo-especificador</i> ID ; <i>tipo-especificador</i> ID [NUM] ;
5. <i>tipo-especificador</i>	→ int void
6. <i>fun-declaração</i>	→ <i>tipo-especificador</i> ID (<i>params</i>) <i>composto-decl</i>
7. <i>params</i>	→ <i>param-lista</i> void
8. <i>param-lista</i>	→ <i>param-lista</i> , <i>param</i> <i>param</i>
9. <i>param</i>	→ <i>tipo-especificador</i> ID <i>tipo-especificador</i> ID []
10. <i>composto-decl</i>	→ { <i>local-declarações</i> <i>statement-lista</i> }
11. <i>local-declarações</i>	→ <i>local-declarações</i> <i>var-declaração</i> ϵ
12. <i>statement-lista</i>	→ <i>statement-lista</i> <i>statement</i> ϵ
13. <i>statement</i>	→ <i>expressão-decl</i> <i>composto-decl</i> <i>seleção-decl</i> <i>iteração-decl</i> <i>retorno-decl</i>
14. <i>expressão-decl</i>	→ <i>expressão</i> ; ;
15. <i>seleção-decl</i>	→ if (<i>expressão</i>) <i>statement</i> if (<i>expressão</i>) <i>statement</i> else <i>statement</i>
16. <i>iteração-decl</i>	→ while (<i>expressão</i>) <i>statement</i>
17. <i>retorno-decl</i>	→ return ; return <i>expressão</i> ;
18. <i>expressão</i>	→ <i>var</i> = <i>expressão</i> <i>simples-expressão</i>
19. <i>var</i>	→ ID ID [<i>expressão</i>]
20. <i>simples-expressão</i>	→ <i>soma-expressão</i> <i>relacional</i> <i>soma-expressão</i> <i>soma-expressão</i>
21. <i>relacional</i>	→ <= < > >= == !=
22. <i>soma-expressão</i>	→ <i>soma-expressão</i> <i>soma termo</i> <i>termo</i>
23. <i>soma</i>	→ + -
24. <i>termo</i>	→ <i>termo</i> <i>mult fator</i> <i>fator</i>
25. <i>mult</i>	→ * /
26. <i>fator</i>	→ (<i>expressão</i>) <i>var</i> <i>ativação</i> NUM
27. <i>ativação</i>	→ ID (<i>args</i>)
28. <i>args</i>	→ <i>arg-lista</i> ϵ
29. <i>arg-lista</i>	→ <i>arg-lista</i> , <i>expressão</i> <i>expressão</i>

Fonte: adaptado de Louden e Silva (2004)

Por fim, antes de entrar nos detalhes de desenvolvimento, cabe ressaltar que a implementação dos módulos da fase de análise foi baseada no código de referência apresentado por Louden e Silva (2004, p. 503–522), tendo sido adaptada para atender às especificidades do compilador desenvolvido neste projeto.

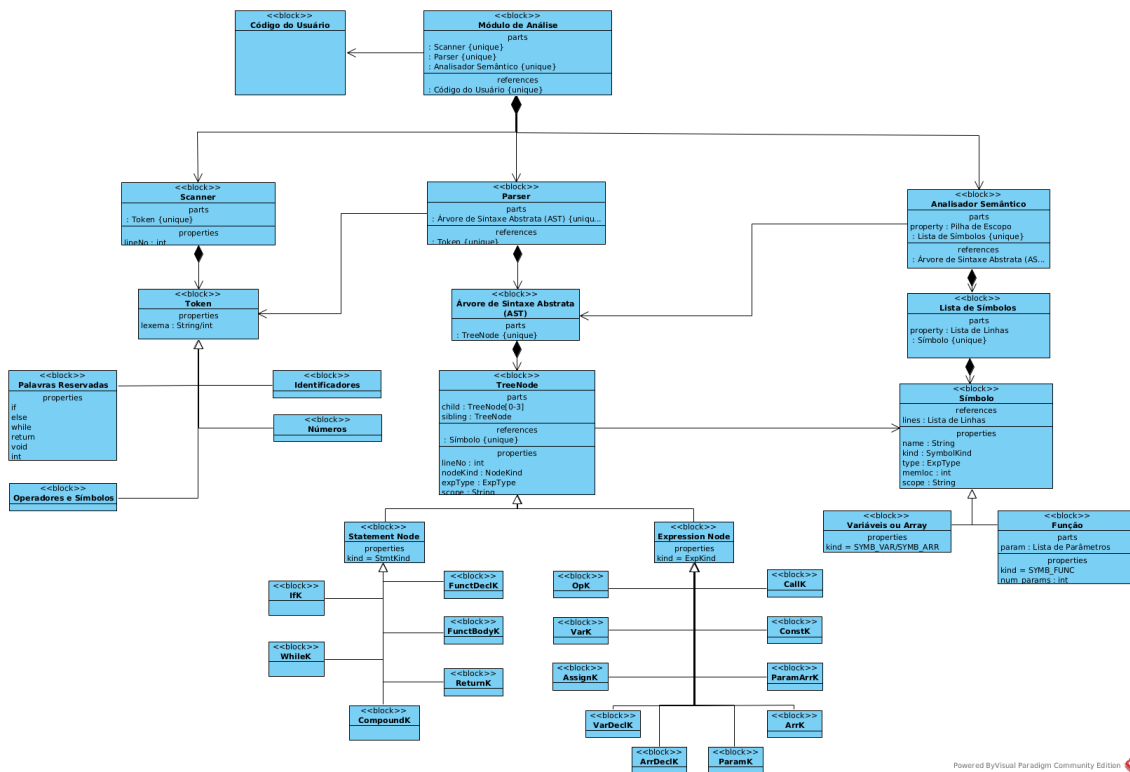
3.1 Modelagem

Com as convenções léxicas da linguagem C– definidas, a fase de análise foi projetada com base em diagramas de blocos e de atividades, utilizados para representar o fluxo e a organização dos processos envolvidos.

3.1.1 Diagrama de Blocos

A partir do diagrama de blocos, apresentado na Figura 4, observa-se que a Fase de Análise compreende a análise léxica, executada pelo *scanner*, a análise sintática, feita pelo *parser*, e a análise semântica.

Figura 4 – Diagrama de blocos da Fase de Análise



Fonte: O Autor

O *scanner* é responsável pela leitura do programa-fonte como um fluxo de caracteres, agrupando-os de modo a formar um lexema (LOUDEN; SILVA, 2004, p. 31). Assim, dos lexemas são produzidos os *tokens*, que categorizam cada cadeia de caracteres identificada durante a varredura. Os *tokens* serão os produtos da análise léxica e serão usadas na etapa seguinte.

A análise sintática, por sua vez, tem como objetivo determinar a estrutura do programa (LOUDEN; SILVA, 2004, p. 95). Para isso, o *parser* solicita *tokens* ao *scanner* a fim de verificar a conformidade do código-fonte com as regras gramaticais da Tabela 10, relação esta representada pela associação de referência na Figura 4. O resultado desse processo dita a aceitação ou rejeição do programa, culminando na construção da Árvore Sintática Abstrata (AST).

A AST é formada por nós, inseridos na estrutura no momento em que ocorre o casamento de uma regra de produção. Cada nó possui atributos que serão posteriormente preenchidos pela análise semântica. Além disso, os nós dividem-se em *Statements*, quando

representam instruções que executam ações, e *Expressions*, quando denotam operações que produzem um valor.

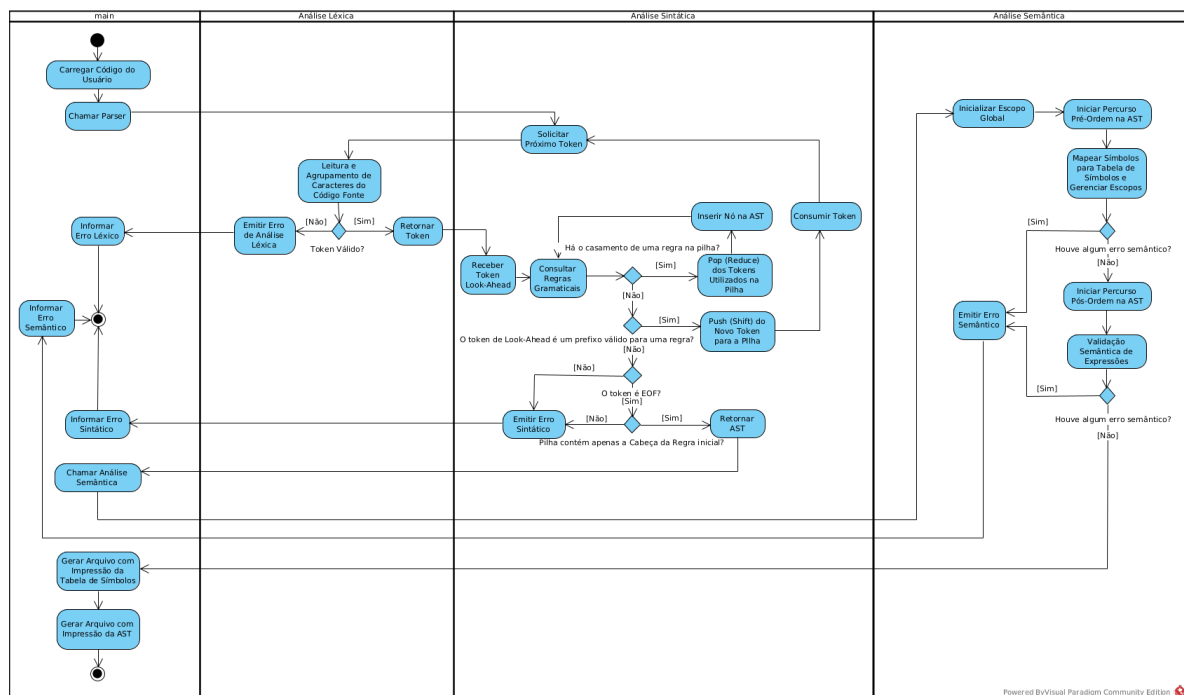
Por fim, a análise semântica é responsável por computar e verificar informações adicionais sobre a estrutura sintática já validada (LOUDEN; SILVA, 2004, p. 259). Assim, essa fase garante que a sintaxe validada anteriormente possua coerência lógica para a execução.

É nessa etapa que ocorre o gerenciamento e o mapeamento de escopos e símbolos. Para isso, emprega-se uma pilha de escopos, essencial tanto para as validações semânticas quanto para a inserção dos símbolos no escopo atual. Junto disso, utiliza-se uma lista para rastrear e organizar as informações de cada símbolo, associando-as aos respectivos escopos. As entradas dessa lista podem representar variáveis, vetores ou funções, armazenando os dados que formam a tabela de símbolos e que serão utilizados de forma recorrente na fase de síntese.

3.1.2 Diagrama de Atividades

Estabelecidos os elementos estruturais da fase de análise, o diagrama de atividades da Figura 5 ilustra o fluxo sequencial de execução desse módulo.

Figura 5 – Diagrama de atividades da Fase de Análise



Fonte: O Autor

O processo tem início na raia **main**, responsável por carregar o código-fonte para análise. Na sequência, o *parser* é invocado para realizar a análise sintática e validar a

estrutura do programa. Inicia-se, então, um ciclo no qual o *scanner* extrai e fornece *tokens* à análise sintática, que consulta as regras gramaticais. Havendo o casamento de uma regra, os *tokens* armazenados são consumidos e inseridos na AST. Caso contrário, o *token* é empilhado, e o ciclo de solicitação e análise prossegue.

Tanto na análise léxica quanto na sintática, a detecção de erros provoca a interrupção antecipada da compilação. Na ausência de falhas, o ciclo sintático se encerra ao encontrar o *token* de fim de arquivo (EOF), retornando a AST completa. De posse dessa árvore, a raiz `main` aciona a análise semântica.

A primeira ação da análise semântica é inicializar o escopo global na pilha de escopos. Em seguida, a AST é percorrida em pré-ordem, visitando-se primeiro o nó raiz, depois a subárvore à esquerda e, por fim, a subárvore à direita. O foco dessa travessia é o mapeamento de símbolos e escopos a partir dos nós de declaração de variáveis e funções. É nesse momento que a tabela de símbolos é construída, ficando pronta para as validações subsequentes. Também verifica-se a ocorrência de erros semânticos atrelados a essas declarações, o que será detalhado na seção de análise semântica.

Como próxima etapa, realiza-se um percurso em pós-ordem na AST, acessando a subárvore à esquerda, seguida da subárvore à direita e, por último, o nó raiz. Com a tabela de símbolos estruturada, as validações semânticas propriamente ditas são executadas nessa varredura. De maneira geral, avalia-se o uso correto de variáveis, a compatibilidade de tipos e a conformidade das regras de retorno.

Concluída a análise semântica sem o retorno de erros, o fluxo volta à raiz `main`, que gera arquivos de saída contendo a impressão da tabela de símbolos e da AST para o usuário. Desse modo, as estruturas de dados essenciais ficam completas e prontas para uso nas fases de síntese do compilador.

3.2 Análise Léxica

Para a implementação da análise léxica, utilizou-se a ferramenta Flex (*Fast Lexical Analyzer Generator*). A Implementação deste módulo foi feita pelo `scanner.1`, apresentado no Apêndice B.

O código é responsável pela transformação do programa-fonte, fornecido como um fluxo de caracteres, em uma sequência de *tokens* consumidos posteriormente pelo *parser*. Essa conversão é realizada por um autômato finito determinístico gerado automaticamente pela ferramenta Flex a partir das expressões regulares declaradas no arquivo `scanner.1`, dispensando a implementação manual do reconhecedor léxico.

O arquivo define os padrões auxiliares correspondentes aos não-terminais léxicos da Tabela 9, além de um estado exclusivo para o tratamento isolado de comentários

multilinha. Cada palavra-chave, operador e delimitador é associado a uma ação que devolve diretamente a constante de *token* correspondente ao *parser*. Já os identificadores e números exigem uma ação semântica adicional: o lexema reconhecido é copiado para a estrutura de atributos compartilhada com o *parser* antes do retorno do *token*, preservando seu valor para as fases seguintes de análise.

O espaço em branco é descartado, servindo apenas como delimitador entre *tokens*, enquanto quebras de linha incrementam um contador global que registram o número de linhas do código. Os comentários são tratados por meio de uma transição de estado: ao encontrar a abertura do comentário, o analisador ignora todo o conteúdo subsequente até localizar o fechamento correspondente, o que impede, por construção, o seu aninhamento. Por fim, qualquer caractere não reconhecido por nenhuma regra anterior é capturado por um padrão genérico, que reporta um erro léxico e descarta o caractere, permitindo que a varredura continue e múltiplos erros sejam identificados em uma única execução.

3.3 Análise Sintática

A implementação da análise sintática foi realizada por meio da ferramenta Bison (*GNU Bison Parser Generator*), responsável por gerar automaticamente um analisador do tipo *LALR* a partir das regras gramaticais declaradas no arquivo `parser.y`, descritas na Tabela 10.

Primeiramente, o analisador léxico é acionado sob demanda pelo *parser*, que solicita um novo *token* sempre que necessário para a aplicação de uma regra gramatical, caracterizando uma integração sob demanda entre as duas fases.

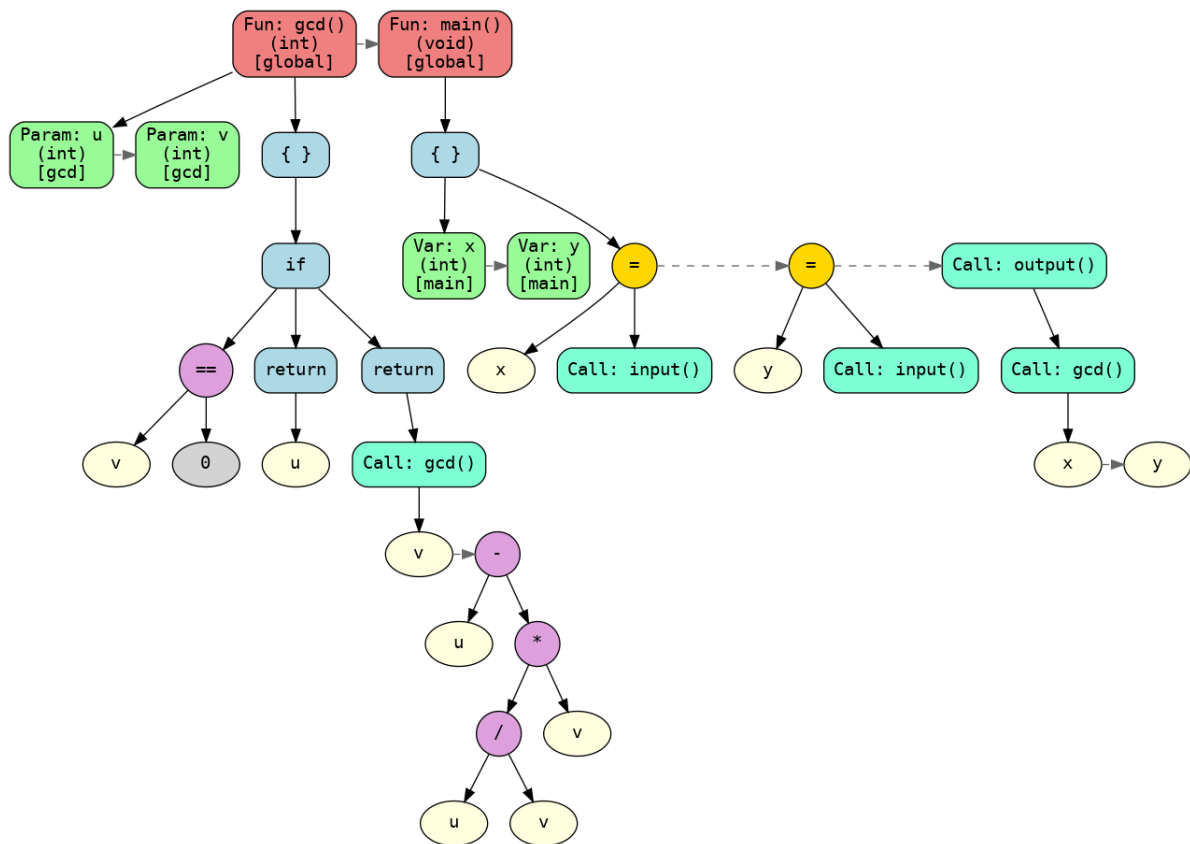
A responsabilidade principal da análise sintática, além da validação estrutural, é a construção da AST. Uma vez que uma regra gramatical é satisfeita, os *tokens* que a compõem são inseridos em um nó da árvore sintática. A inserção desses elementos na AST segue uma convenção que distingue suas relações, classificando-os como nós filhos ou irmãos. Isso ocorre porque um programa contém tanto estruturas hierárquicas, nas quais um elemento depende de outro ou é composto por ele, quanto estruturas sequenciais, em que elementos distintos ocorrem um após o outro em um mesmo nível de organização.

Partindo pela relação hierárquica, ela ocorre sempre quando existe uma dependência estrutural entre os nós, ou seja, quando um nó só pode ser compreendido em função de seus componentes internos. Esse é o caso, por exemplo, dos operandos de uma expressão aritmética, que são filhos do nó que representa a operação, ou da condição e do corpo de uma estrutura de controle como o `if` e o `while`, que são filhos do nó correspondente a essa estrutura. Em todos esses casos, percorrer a árvore descendo por um nó até seus filhos corresponde a decompor uma construção da linguagem em suas partes constituintes, refletindo a hierarquia sintática definida pela gramática.

Já a relação sequencial, ela ocorre quando dois ou mais nós ocupam o mesmo nível lógico do programa e não há dependência estrutural entre eles, apenas uma ordem de execução ou declaração. As instruções dentro de um mesmo bloco de código, as declarações de variáveis dentro de um mesmo escopo, e os argumentos de uma chamada de função são exemplos dessa relação: cada elemento da sequência é independente dos demais, mas todos pertencem ao mesmo contexto e devem ser processados na ordem em que aparecem no programa-fonte. Por essa razão, esses elementos não são representados como filhos uns dos outros, mas como uma cadeia de irmãos encadeados a partir do primeiro elemento da sequência.

Para ilustrar essas relações, observe na Figura 6 a AST construída durante a análise sintática do código `gcd.cm` (Listing 5.2). Note que a estruturação do escopo é formada por nós filhos associados ao nó de agrupamento da função, enquanto a sequência de declarações (`Var`), atribuições (`=`) e instruções — como a chamada da função `output()` — é disposta na forma de nós irmãos. Além disso, nota-se que a operação matemática $u - u/v \cdot v$ é modelada como uma ramificação de nós filhos, refletindo suas relações de precedência e dependência.

Figura 6 – Árvore Sintática Abstrata construída na análise sintática do código `gcd.cm`



Fonte: O Autor

A convenção adotada na construção da AST é fundamental para a fase de análise

semântica, etapa na qual o percurso da árvore se baseia diretamente nessa padronização. Além disso, também é essencial compreender os dados que cada nó armazena – ou pode vir a armazenar –, visto que essas informações são igualmente imprescindíveis para a próxima etapa.

Cada nó da AST armazena, além das referências aos seus filhos e ao irmão seguinte, um conjunto de informações que o caracteriza e o distingue dos demais. A primeira é a sua classificação em *Statement* ou *Expression*, conforme mencionado anteriormente, que determina se o nó representa uma instrução executada por seu efeito ou uma expressão avaliada por seu valor. Associado a essa classificação, cada nó possui um identificador mais específico de sua natureza sintática – por exemplo, se uma *Expression* representa uma constante, uma variável, uma operação ou uma chamada de função, ou se um *Statement* representa uma estrutura condicional, um laço de repetição ou um retorno.

Além disso, cada nó armazena um atributo que guarda a informação específica daquela construção. O que esse atributo representa depende do tipo do nó: em uma constante, é o seu valor numérico; em um identificador, é o seu nome; em uma operação aritmética ou relacional, é o operador utilizado. Também, cada nó também registra a linha do código-fonte em que a construção foi reconhecida, informação usada mais adiante para indicar com precisão a localização de erros semânticos.

Por último, dois campos do nó permanecem vazios ao final da análise sintática e são preenchidos apenas durante a análise semântica: o tipo da expressão ou declaração, usado nas verificações de compatibilidade de tipos (`int` ou `void`), e a referência ao símbolo correspondente na tabela de símbolos, que vincula o nó à variável, vetor ou função que ele representa ou utiliza. É justamente o preenchimento dessas duas informações, somado à verificação das regras de tipo e escopo, que constitui o propósito central da fase semântica.

Em contrapartida ao processo de construção da árvore em caso de sucesso, qualquer sequência de *tokens* que não corresponda às regras gramaticais acionará a rotina de tratamento de erros sintáticos. Esse mecanismo é responsável por reportar o *token* inesperado e a linha de sua ocorrência, interrompendo imediatamente a compilação. Dessa forma, o *parser* cumpre seu papel de estruturar a AST para códigos válidos e impedir que programas malformados avancem para a análise semântica.

3.4 Análise Semântica

A análise semântica é responsável por validar a coerência lógica do programa após a construção da AST, etapa em que são verificadas regras que a gramática livre de contexto, por si só, não é capaz de expressar. A implementação desse módulo, presente no arquivo `analyze.c`, organiza-se em duas varreduras sucessivas e independentes sobre a AST, cada uma com um percurso e um propósito distintos: a primeira constrói a tabela

de símbolos percorrendo a árvore em pré-ordem, e a segunda verifica a compatibilidade de tipos percorrendo a árvore em pós-ordem.

3.4.1 Tabela de Símbolos

Antes de detalhar as duas varreduras, é necessário compreender a estrutura que ambas utilizam como base: a tabela de símbolos, implementada no arquivo `symtab.c`. Diferentemente de uma tabela única e global, a tabela de símbolos do compilador é organizada como uma pilha de escopos, em que cada escopo mantém sua própria lista de símbolos declarados.

Cada nó de escopo armazena um nome identificador, uma referência ao escopo pai e uma lista encadeada dos símbolos declarados diretamente nele. Quando um novo escopo é aberto, como ao entrar em uma função ou em um bloco de código delimitado por chaves, ele é empilhado sobre o escopo atual; quando o escopo é encerrado, ele é desempilhado, e o escopo atual retorna a ser o escopo pai. Esse mecanismo garante que identificadores declarados em um bloco interno não sejam visíveis fora dele, ao mesmo tempo em que permite que blocos internos acessem identificadores declarados em escopos externos.

Cada símbolo armazenado na tabela, por sua vez, mantém seu nome, o escopo ao qual pertence, seu tipo, e sua classificação como variável, vetor ou função. Também é armazenada sua localização de memória, calculada no momento da declaração e utilizada nas fases posteriores de geração de código. Essa localização corresponde ao deslocamento do símbolo dentro de seu escopo, atribuído sequencialmente conforme as declarações são encontradas: variáveis simples ocupam uma posição, enquanto vetores ocupam o tamanho declarado somado a um, considerando também o espaço para o armazenamento do ponteiro do vetor. Assim, cada novo símbolo recebe um deslocamento que leva em conta o espaço acumulado por todos os símbolos anteriores do mesmo escopo. Além disso, é mantida uma lista de todas as linhas do código-fonte em que o símbolo foi utilizado, empregada na impressão final da tabela de símbolos como instrumento de depuração. No caso específico de símbolos que representam funções, é mantida adicionalmente uma lista dos tipos de seus parâmetros, na ordem em que foram declarados, permitindo a verificação posterior da quantidade e da compatibilidade dos argumentos em cada chamada. Um exemplo de impressão da tabela de símbolos pode ser observado na Figura 7.

Figura 7 – Tabela de símbolos construída para o código `sort.cm`

Nome do Símbolo	Tipo	Classificacao	Escopo	Loc	Linhas
main	void	Funcao	global	-	33
sort	void	Funcao	global	-	20 41
minloc	int	Funcao	global	-	5 25
vet	int	Vetor	global	0	3 38 41 44
output	void	Funcao	global	-	0 44
input	int	Funcao	global	-	0 38
k	int	Variavel	minloc	5	6 7 13 17
x	int	Variavel	minloc	4	6 8 11 12
i	int	Variavel	minloc	3	6 9 10 11 12 13 15
high	int	Variavel	minloc	2	5 10
low	int	Variavel	minloc	1	5 7 8 9
a	int	Vetor	minloc	0	5 8 11 12
k	int	Variavel	sort	4	21 25 26 27
i	int	Variavel	sort	3	21 22 23 25 27 28 29
high	int	Variavel	sort	2	20 23 25
low	int	Variavel	sort	1	20 22
a	int	Vetor	sort	0	20 25 26 27 28
t	int	Variavel	sort:block1	5	24 26 28
i	int	Variavel	main	0	35 36 37 38 39 42 43 44 45

Fonte: O Autor

A respeito de seu uso, a tabela é utilizada para a busca por um símbolo. Isso pode ocorrer de três formas, cada uma atendendo a uma necessidade distinta de validação: a busca hierárquica, que parte do escopo atual e percorre os escopos pais até encontrar o símbolo ou esgotar a hierarquia, sendo utilizada para validar o uso de identificadores já declarados em qualquer escopo acessível; a busca restrita ao escopo atual, utilizada para detectar declarações duplicadas dentro de um mesmo escopo; e a busca restrita ao escopo global, utilizada exclusivamente para a validação de chamadas de função, já que a linguagem C não permite a declaração de funções dentro de outras funções.

3.4.2 Primeira Varredura: Construção da Tabela de Símbolos

A primeira varredura, implementada pela função `buildSymtab`, percorre a AST em pré-ordem, visitando primeiro o nó raiz, depois recursivamente cada um de seus filhos, e por fim o nó irmão seguinte. Essa ordem de percurso é necessária porque a inserção de um símbolo na tabela deve ocorrer antes da análise de qualquer construção que venha a utilizá-lo, e porque a abertura e o fechamento de escopos devem respeitar a ordem em que blocos e funções aparecem no código-fonte.

Antes de iniciar o percurso, a função insere no escopo global os símbolos correspondentes às funções de entrada e saída da linguagem, `input` e `output`, que não possuem declaração explícita no código-fonte do programa analisado.

Durante o percurso, cada nó é submetido a um tratamento que depende de sua classificação. Ao encontrar uma declaração de função, o compilador verifica se já existe uma função de mesmo nome declarada no escopo atual; em caso positivo, reporta o erro de função já declarada. Caso contrário, a função é inserida na tabela de símbolos, um

novo escopo é aberto para o seu corpo, e, se o nome da função for `main`, registra-se sua presença para verificação posterior. Cada bloco de código delimitado por chaves, por sua vez, também abre um novo escopo, nomeado de forma a refletir sua posição dentro da função que o contém.

A declaração de uma variável simples ou de um vetor é submetida a três verificações: se o tipo declarado é `void`, é reportado o erro de variável do tipo `void`; se já existe um símbolo de mesmo nome no escopo atual, é reportado o erro de variável já declarada no escopo; caso nenhuma das duas condições se aplique, o símbolo é inserido na tabela, com sua localização de memória avançada de acordo com o tamanho ocupado — uma posição para variáveis simples, e uma quantidade de posições igual ao tamanho declarado para vetores.

De forma semelhante, os parâmetros de uma função são submetidos a verificações próprias: um parâmetro do tipo `void` é reportado como erro de parâmetro do tipo `void` — exceto no caso específico do `void` isolado utilizado para declarar uma função sem parâmetros, que não corresponde a um parâmetro nomeado e não gera erro —, e um parâmetro de nome já existente na função é reportado como erro de parâmetro já declarado nesta função. Em ambos os casos de erro, um símbolo de contingência é inserido na tabela para permitir que a análise prossiga sem interrupção, e o parâmetro é igualmente registrado na lista de parâmetros da função corrente, de modo a preservar a contagem correta de parâmetros mesmo diante de um erro.

O uso de uma variável ou de um vetor já declarado é validado por meio da busca hierárquica na tabela de símbolos: caso o identificador não seja encontrado em nenhum escopo acessível, é reportado o erro de variável não declarada; no caso específico do vetor, há também a verificação de que o símbolo encontrado de fato corresponda a um vetor, sendo reportado o erro quando essa condição não se confirma. De maneira análoga, a chamada de uma função é validada por meio da busca restrita ao escopo global: caso a função não seja encontrada, é reportado o erro de função não declarada; caso o identificador exista mas não corresponda a uma função, é reportado o erro de que o identificador não é uma função.

Adicionalmente, ao longo de toda a primeira varredura, é mantida a verificação de que nenhuma nova declaração ocorra após a função `main`, restrição decorrente da ausência de protótipos de função na linguagem C: qualquer declaração de variável, vetor ou função encontrada no escopo global após a declaração da `main` é reportada como erro de declaração inválida após a função `main`. Ao final da varredura, caso nenhuma função de nome `main` tenha sido encontrada em todo o programa, é reportado o erro de ausência da função `main`.

3.4.3 Segunda Varredura: Verificação de Tipos

A segunda varredura, implementada pela função `typeCheck`, percorre a AST em pós-ordem, visitando recursivamente cada filho do nó antes do próprio nó, e o nó irmão seguinte apenas ao final. Essa inversão em relação à primeira varredura é necessária porque a verificação de tipo de uma construção depende do tipo (`int` ou `void`) já resolvido de seus operandos. Por exemplo, o tipo de uma operação aritmética só pode ser conferido depois que o tipo de ambos os operandos já tiver sido determinado, o que exige que os filhos sejam visitados antes do próprio nó.

Operações aritméticas e relacionais têm seus dois operandos verificados quanto à compatibilidade de tipo, sendo reportado o erro de operandos incompatíveis quando qualquer um deles não for do tipo inteiro. Para que a verificação prossiga nos níveis superiores da árvore para detectar outros erros, o resultado da operação é fixado como inteiro, independentemente da ocorrência do erro. O índice de um vetor é submetido à mesma verificação, sendo ele exclusivamente um inteiro. Já em uma atribuição, ambos os lados são verificados quanto à compatibilidade, sendo referenciado o erro de tipos incompatíveis quando não corresponderem.

A respeito da chamada de uma função, ela é submetida a validação de seus parâmetros. Primeiramente, os tipos dos argumentos fornecidos na chamada são comparados, em ordem, com os tipos dos parâmetros registrados na declaração da função, sendo declarado o erro de tipo incompatível no argumento para cada divergência encontrada. Ao final da comparação, caso a quantidade de argumentos fornecidos não corresponda à quantidade de parâmetros esperados, é apontado o erro de total de argumentos incorreto para a função.

As estruturas de controle de fluxo também são verificadas. Nelas, tanto a condição de uma estrutura `if` quanto a de uma estrutura `while` devem ser do tipo inteiro. Por fim, a instrução de retorno é validada de acordo com o tipo de retorno declarado para a função em questão. Caso a função seja do tipo `void`, qualquer valor de retorno presente é reportado como erro; caso a função seja do tipo inteiro, um retorno vazio não deve ser aceito. Além disso, para funções do tipo `int`, ao final da verificação de seu corpo, é conferido se ao menos uma instrução de retorno foi encontrada ao longo de seu bloco, sendo apontado o erro de ausência de retorno para uma função inteira para caso nenhum tenha ocorrido.

4 Compilador: Fase de Síntese

A fase de síntese é a tarefa final de um compilador, com o objetivo de gerar um código executável para uma máquina-alvo. A sequência de instruções obtidas devem ser uma representação fiel da semântica do código-fonte. Para isso, nessa etapa é necessário, além das características da linguagem-fonte, de informações a respeito da arquitetura alvo, como registradores, modos de endereçamento e o gerenciamento da memória (LOUDEN; SILVA, 2004, p.401).

Com esse objetivo, a fase de síntese é dividida em três etapas: A geração de código intermediário, a geração de código *assembly* e a tradução para código executável.

4.1 Geração do código intermediário

A geração de código intermediário é a primeira etapa da fase de síntese. Ela é responsável por organizar uma estrutura inicial do código executável a partir de um código abstrato (LOUDEN; SILVA, 2004, p.401). Para isso, o módulo de geração de código intermediário percorre as subárvores linearizando a AST a partir da geração das estruturas intermediárias.

A estrutura intermediária utilizada neste compilador é a quádrupla, que organiza uma instrução abstrata em 4 campos: um para a instrução e três para os operandos. O projeto e implementação desse módulo serão discutidos nas seções a seguir.

4.1.1 Modelagem

4.1.1.1 Diagrama de Blocos

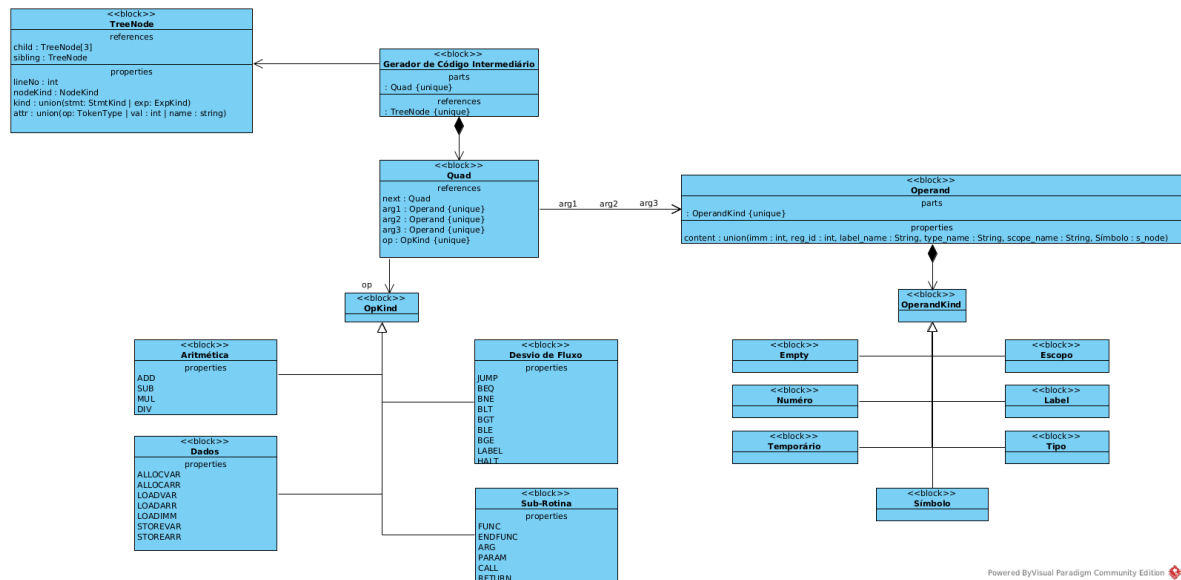
Como um panorama geral, o diagrama de blocos exibido na Figura 8 ilustra a organização do gerador de código intermediário. Conforme mencionado, essa etapa é realizada a partir do percurso da AST gerada na fase de análise, dependência que é representada pela associação de referência entre o bloco principal e o bloco correspondente ao nó da árvore.

O produto desse módulo são as quádruplas, que serão emitidas e armazenadas numa lista. A respeito de sua composição, elas são formadas pelo campo de operação e por três argumentos (1, 2 e 3). Cada argumento é estruturado sob um tipo de dado denominado *Operand*, que especifica a categoria do operando em questão: vazio, número, registrador temporário, símbolo, escopo, *label* ou tipo (`int` ou `void`). Para cada categoria, um conteúdo específico é armazenado para impressão e uso posterior na tradução para

assembly.

No que diz respeito às operações, elas podem ser generalizadas em aritméticas, de manipulação de dados, de desvio de fluxo e de sub-rotina. O conjunto completo dessas operações pode ser observado na Figura 8 e será detalhado nas seções subsequentes.

Figura 8 – Diagrama de blocos da Geração de Código Intermediário



Fonte: O Autor

4.1.2 Diagrama de Atividades

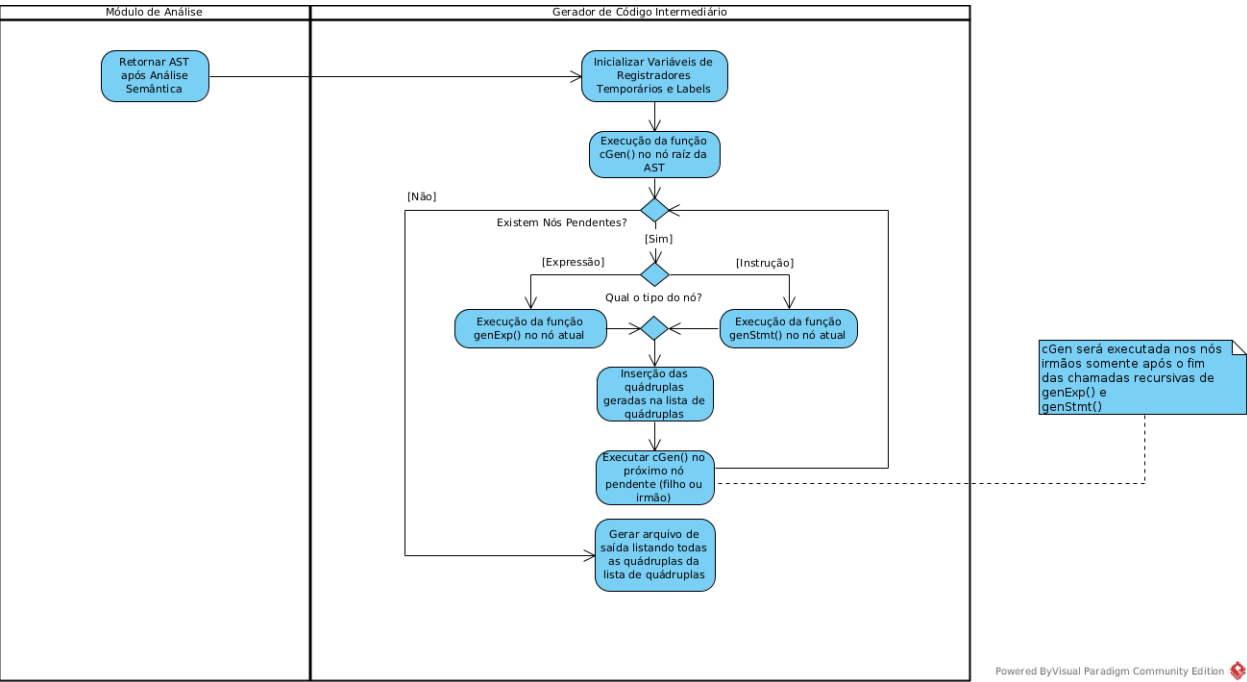
A sequência da geração de código intermediário pode ser representada pelo diagrama de atividades da Figura 9. O fluxo desse módulo tem início na raiz do Gerador de Código Intermediário, imediatamente após o recebimento da AST produzida pelo Módulo de Análise, com a inicialização das variáveis responsáveis pela numeração dos registradores temporários e dos rótulos que serão definidos ao longo dessa etapa.

Em seguida, a função **cGen** é invocada sobre o nó raiz da AST para iniciar a varredura da árvore. Essa função tem como responsabilidade verificar o tipo de um nó e delegar o seu tratamento a uma rotina especializada correspondente. Para nós do tipo *Statement* a função chamada será **genStmt** e para nós do tipo *Expression* será invocado **genExp**. Após esse tratamento, **cGen** avança automaticamente para o nó irmão do nó atual, invocando-se recursivamente sobre ele. Essa chamada recursiva sobre irmãos é o mecanismo pelo qual **cGen** percorre sequências de nós de mesmo nível – como as instruções de um bloco ou as declarações de um escopo. O percurso dos nós filhos, por sua vez, não ocorre em **cGen** diretamente, mas sim dentro de **genStmt** e **genExp**, que invocam **cGen** recursivamente sobre os filhos de cada nó quando necessário para a geração do código correspondente. Dessa forma, a varredura completa da AST parte da combinação entre

a descida nos filhos, controlada pelas funções especializadas, e o avanço entre irmãos, controlado por **cGen**.

Concluída a varredura de todos os nós, um arquivo de saída é produzido com a impressão da sequência de quádruplas geradas, ao mesmo tempo em que a lista de quádruplas completa fica disponível para uso pela fase subsequente de geração de código assembly.

Figura 9 – Diagrama de atividades da Geração de Código Intermediário



Fonte: O Autor

4.1.3 Definição das quádruplas

No projeto da geração de código intermediário, foram definidas as quádruplas apresentadas na Tabela 11. Nela, detalham-se a categoria, a operação e os argumentos de cada instrução, juntamente com uma breve descrição de seus significados. Esse conjunto será adotado no restante da fase de síntese, sendo cada operando estruturado para garantir uma representação intermediária coesa e legível, mas, principalmente, útil para a etapa de geração de código *assembly*.

Tabela 11 – Descrição das quádruplas

Categoria	Operação	Arg 1	Arg 2	Arg 3	Descrição
Aritmética	ADD	destino	origem	origem	Soma arg2 e arg3, armazena em arg1

Continua na próxima página...

Tabela 11 – continuação

	SUB	destino	origem	origem	Subtrai arg3 de arg2, armazena em arg1
	MUL	destino	origem	origem	Multiplica arg2 e arg3, armazena em arg1
	DIV	destino	origem	origem	Divide arg2 por arg3, armazena em arg1
Memória	ALLOCVAR	símbolo	escopo	–	Reserva memória para uma variável
	ALLOCARR	símbolo	imediato	escopo	Reserva memória para um vetor de tamanho arg2
	LOADVAR	destino	símbolo	–	Carrega o valor de uma variável em arg1
	LOADARR	destino	símbolo	orig. (índice)	Carrega o valor de uma posição do vetor em arg1
	LOADIMM	destino	imediato	–	Carrega o valor imediato arg2 em arg1
	STOREVAR	símbolo	origem	–	Armazena arg2 na posição de memória de arg1
	STOREARR	símbolo	origem	orig. (índice)	Armazena arg2 na posição do vetor indicada por arg3
Controle de Fluxo	BEQ	origem	origem	<i>label</i>	Desvia para arg3 se $\text{arg1} = \text{arg2}$
	BNE	origem	origem	<i>label</i>	Desvia para arg3 se $\text{arg1} \neq \text{arg2}$
	BGE	origem	origem	<i>label</i>	Desvia para arg3 se $\text{arg1} \geq \text{arg2}$
	BLE	origem	origem	<i>label</i>	Desvia para arg3 se $\text{arg1} \leq \text{arg2}$
	BGT	origem	origem	<i>label</i>	Desvia para arg3 se $\text{arg1} > \text{arg2}$
	BLT	origem	origem	<i>label</i>	Desvia para arg3 se $\text{arg1} < \text{arg2}$
	JUMP	<i>label</i>	–	–	Desvia incondicionalmente para arg1
	LABEL	<i>label</i>	–	–	Demarca um destino de desvio
	HALT	–	–	–	Interrompe a execução

Continua na próxima página...

Tabela 11 – continuação

Funções	FUNC	tipo	símbolo	–	Demarca o início de uma função
	ARG	tipo	símbolo	escopo	Declara um parâmetro formal
	ARGARR	tipo	símbolo	escopo	Declara um parâmetro formal de vetor
	RETURN	origem	–	–	Retorna o valor de arg1 ao chamador
	ENDFUNC	símbolo	–	–	Demarca o encerramento de uma função
	PARAM	origem	–	–	Empilha arg1 como argumento da próxima chamada
	CALL	destino	símbolo	imediato	Chama arg2 com arg3 argumentos; retorno em arg1
Entrada/ Saída	INPUT	destino	–	–	Lê um valor da entrada e armazena em arg1
	OUTPUT	origem	–	–	Escreve o valor de arg1 na saída

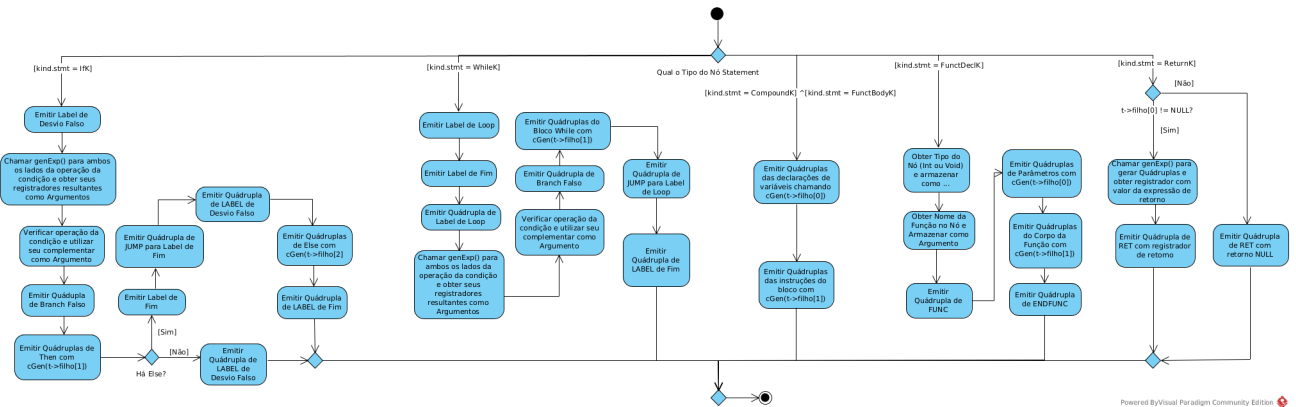
Fonte: O Autor.

4.1.4 Geração das quádruplas

Uma vez estabelecido o conjunto de quádruplas, o módulo gerador emite uma sequência dessas instruções para cada subárvore da AST. Conforme visto anteriormente, esse processo ocorre por meio da tradução conjunta dos nós de *Statements* e *Expressions*, resultando na construção lógica e linear do código intermediário. Sendo assim, a seguir será detalhado o tratamento executado pelas funções de geração de código para cada um desses nós.

4.1.4.1 Geração de *Statements*

O diagrama de atividades ilustrado na Figura 10 apresenta a sequência de funcionamento da função **genStmt** e suas emissões de quádruplas. Note que nessa rotina há o tratamento de quatro categorias de nós: declaração de função (**FunctDeclK**), bloco composto (**CompoundK**), estruturas de controle de fluxo (**IfK** e **WhileK**) e instrução de retorno (**ReturnK**).

Figura 10 – Diagrama de atividades da geração de quádruplas para *Statements*

Fonte: O Autor

Começando pela declaração de uma função, ela delimita seu código com as quádruplas **FUNC** e **ENDFUNC**, ambas carregando o símbolo da função. Entre elas, **cGen** é invocada recursivamente sobre os parâmetros e, em seguida, sobre o corpo da função. Os parâmetros do tipo **ARG** e **ARGARR** são emitidos durante essa varredura, carregando o tipo, o símbolo e o escopo de cada parâmetro declarado. O bloco composto (**CompoundK**) não emite nenhuma quádrupla própria, sendo sua contribuição inteiramente determinada pelas quádruplas emitidas pela varredura de suas declarações locais e de sua lista de instruções.

A respeito da instrução de **if**, ela opera em dois modos conforme a presença ou ausência do bloco **else**. Em ambos, a condição é avaliada primeiro e um desvio condicional é emitido para um rótulo de saída, de modo que, caso a condição seja falsa, a execução salte diretamente para fora do bloco **then**. O corpo do bloco **then** é então emitido na sequência. Quando há bloco **else**, um desvio incondicional é emitido ao final do bloco **then** para um rótulo de encerramento da estrutura toda, impedindo que a execução prossiga para o bloco **else**; o rótulo de saída do **then** é então posicionado, seguido do bloco **else** e, por fim, seguido do rótulo de encerramento. Na ausência do **else**, apenas o rótulo de saída é posicionado após o bloco **then**.

Para o laço de repetição, a primeira atividade é posicionar um rótulo de início imediatamente antes da avaliação da condição, de modo que cada iteração retorne a esse ponto. A condição é avaliada e um desvio condicional é emitido para um rótulo de encerramento. Assim, o corpo do laço é emitido na sequência, seguido de um desvio incondicional de retorno ao rótulo de início, fechando o ciclo. O rótulo de encerramento é então posicionado, demarcando o ponto a partir do qual a execução prossegue após a saída do laço.

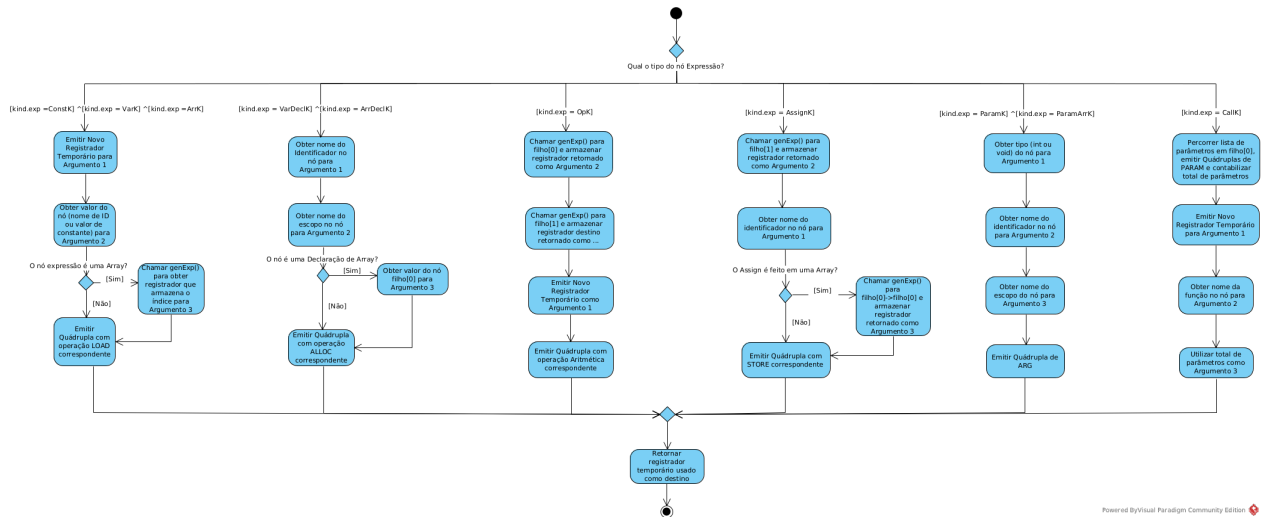
Em ambas as estruturas, a natureza da condição determina qual quádrupla de desvio é emitida. Quando a condição é uma operação relacional explícita, o operador do desvio é o inverso lógico do operador original – por exemplo, um **if (a < b)** emite um **BGE**, garantindo que o desvio para fora do bloco ocorra exatamente quando a condição é falsa. Quando a condição não é uma operação relacional – como o uso direto de uma variável ou o retorno de uma chamada de função –, ela é comparada implicitamente com zero por meio de uma quádrupla **BEQ**, seguindo a convenção da linguagem C de que qualquer valor diferente de zero é interpretado como verdadeiro.

Por fim, acerca da instrução de retorno, ela emite uma quádrupla **RETURN** carregando o registrador que contém o valor a ser devolvido, quando presente, ou vazia quando se trata de um retorno sem valor em função **void**.

4.1.4.2 Geração de *Expressions*

A respeito da função **genExp**, sua sequência de atividades é descrita no diagrama ilustrado na Figura 11. Tal rotina executa o tratamento de sete categorias de nós: declarações de variável e vetor, parâmetros, operações aritméticas e relacionais, constantes, uso de variáveis e vetores, atribuição e chamada de função.

Figura 11 – Diagrama de atividades da geração de quádruplas para *Expressions*



Fonte: O Autor

Os primeiros casos tratados por **genExp** são os de declarações de variável simples e de vetor são. Para uma variável simples, é emitida uma quádrupla **ALLOCVAR** carregando o símbolo declarado e o nome do escopo corrente, informação obtida diretamente do nó da AST. Para um vetor, é emitida uma quádrupla **ALLOCARR** com o mesmo símbolo e escopo, acrescida do tamanho declarado como operando imediato, que é obtido do nó filho que representa essa constante na AST. Em nenhum dos dois casos há retorno de registrador, já que declarações não produzem um valor a ser utilizado por um nó pai.

Os parâmetros simples e de vetor emitem, respectivamente, **ARG** e **ARGARR**, carregando o tipo declarado, o símbolo e o escopo da função à qual pertencem. Vale destacar que esses casos são atingidos quando **cGen** percorre os filhos do nó **FuncDeclK** durante o tratamento de uma declaração de função em **genStmt** — não por uma chamada direta e independente de **genExp**. Isso significa que as quádruplas **ARG** e **ARGARR** são sempre emitidas entre as quádruplas **FUNC** e **ENDFUNC** da função correspondente, preservando a ordem de declaração dos parâmetros.

A respeito das constantes numéricas, essas emitem uma quádrupla **LOADIMM** com o valor literal do nó como operando imediato, armazenando-o em um novo registrador temporário que é então retornado ao nó pai. O uso de uma variável simples segue o mesmo padrão: é emitida uma quádrupla **LOADVAR** com o símbolo correspondente, e o registrador de destino é retornado. O uso de um vetor exige uma etapa adicional, de modo que, antes de emitir **LOADARR**, a subárvore do índice é gerada recursivamente por meio de **genExp**, obtendo-se o registrador que contém o valor calculado do índice; somente então a quádrupla é emitida, com o símbolo do vetor e esse registrador como operandos.

As operações aritméticas e relacionais seguem a mesma estrutura de geração. Primeiro, os dois operandos são gerados recursivamente por meio de chamadas a **genExp**, cada uma retornando o registrador que contém o valor calculado. Com os dois registradores em mãos, é emitida a quádrupla correspondente

ao operador do nó, tendo esses registradores como origem e um novo registrador temporário como destino. Esse registrador de destino é retornado ao nó pai, de modo que o resultado da operação possa ser utilizado como operando em uma expressão maior.

Acerca da atribuição, ela começa pela geração recursiva do lado direito, obtendo o registrador com o valor a ser atribuído. Em seguida, o lado esquerdo é examinado: se for uma variável simples, é emitida **STOREVAR** com o símbolo e o registrador de origem; se for uma posição de vetor, o índice é primeiro gerado recursivamente, e então é emitida **STOREARR** com o símbolo, o registrador de origem e o registrador do índice. Em ambos os casos, o registrador que contém o valor atribuído é retornado como resultado da expressão de atribuição.

Por fim, a chamada de função distingue três situações. Para a função nativa **input**, é emitida diretamente uma quádrupla **INPUT** em um novo registrador de destino, que é imediatamente retornado. Para a função nativa **output**, o único argumento é gerado recursivamente e passado como origem de uma quádrupla **OUTPUT**, sem retorno de registrador.

Para as demais funções definidas pelo programa, a geração percorre os argumentos na ordem em que aparecem como nós irmãos na AST. Para cada argumento, **genExp** é invocada recursivamente, obtendo o registrador com o valor calculado, e uma quádrupla **PARAM** é imediatamente emitida com esse registrador. Esse processo se repete para todos os argumentos, acumulando as quádruplas **PARAM** em sequência antes de qualquer instrução de chamada. Somente após a emissão de todos os **PARAM** é que a quádrupla **CALL** é emitida, carregando o símbolo da função chamada e a contagem total de argumentos como imediato. Assim, um novo registrador temporário é alocado como destino do valor de retorno e retornado ao nó pai. Quando a função é do tipo **void**, esse registrador simplesmente não será referenciado por nenhuma quádrupla subsequente.

4.1.5 Exemplo de geração de código intermediário

Para demonstração do módulo de geração de código intermediário, a Tabela 12 foi desenvolvida, sendo apresentada na primeira coluna um código-fonte e na segunda coluna sua correspondência em quádruplas.

Tabela 12 – Mapeamento entre o código-fonte e o código intermediário gerado

Código C–	Código Intermediário
<code>void main(void) {</code>	(FUNC, void, main, -)
<code> int x; int y;</code>	(ALLOCVAR, x, main, -) (ALLOCVAR, y, main, -)
<code> x = input();</code>	(INPUT, \$t1, -, -) (STOREVAR, x, \$t1, -)
<code> y = input();</code>	(INPUT, \$t2, -, -) (STOREVAR, y, \$t2, -)
<code> if (x>y)</code>	(LOADVAR, \$t3, x, -) (LOADVAR, \$t4, y, -) (BLE, \$t3, \$t4, L1)
<code> output(x);</code>	(LOADVAR, \$t5, x, -) (OUTPUT, -, \$t5, -)
<i>(salto do if para o fim)</i>	(JUMP, L2, -, -)
<code> else</code>	(LABEL, L1, -, -)
<code> output(y);</code>	(LOADVAR, \$t6, y, -) (OUTPUT, -, \$t6, -)
<i>(fim do if/else)</i>	(LABEL, L2, -, -)
<code>}</code>	(ENDFUNC, main, -, -)

Fonte: O Autor.

4.2 Gerenciamento de memória

Uma vez que a representação abstrata foi desenvolvida, as etapas seguintes de tradução irão se basear totalmente nos aspectos de arquitetura da máquina-alvo. Além do conjunto de instruções, apresentado na Tabela 4, outra característica fundamental é a organização da memória.

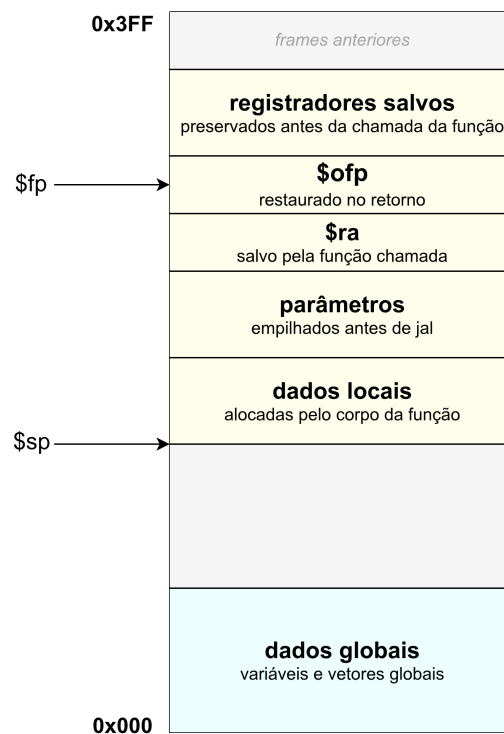
A memória de dados é segmentada em duas regiões com lógicas distintas. A primeira representa os dados globais, que são dispostos a partir da base da memória, isto é, a partir da posição zero, crescendo em direção às posições superiores. A outra, diz respeito aos dados locais, que, por sua vez, são organizados a partir do topo da memória em direção à base, seguindo o modelo clássico de pilha com crescimento descendente.

Para viabilizar o gerenciamento dos dados locais, dois registradores são reservados exclusivamente para armazenar endereços de memória: o *stack pointer* (\$sp) e o *frame pointer* (\$fp). O ponteiro de pilha indica a última posição reservada na pilha, sendo atualizado a cada inserção ou remoção de dados. O ponteiro de quadro, por sua vez, demarca a base do quadro de ativação do escopo corrente, servindo como referência fixa para o acesso aos dados locais daquele escopo.

Cada função em execução ocupa um quadro de ativação próprio, empilhado sobre os demais durante a chamada e desempilhado ao retorno. A organização interna de cada quadro segue uma estrutura fixa, ilustrada na Figura 12, disposta nas seguintes seções, do topo para a base: os registradores em uso pela função chamadora, salvos antes da chamada para preservar seu conteúdo; o antigo valor do ponteiro

de quadro (**\$ofp**), necessário para a restauração do quadro da função chamadora ao retorno; o endereço de retorno (**\$ra**), que indica a instrução a ser executada após o encerramento da função; os parâmetros recebidos pela função; e, por fim, as variáveis locais declaradas no corpo da função. O ponteiro de pilha acompanha cada inserção, sempre apontando para a última posição ocupada, enquanto o ponteiro de quadro permanece fixado na posição do antigo ponteiro de quadro durante toda a execução da função.

Figura 12 – Organização da memória de dados



Fonte: O Autor

4.3 Geração do código *Assembly*

Com o código intermediário gerado no módulo anterior, as características da máquina-alvo conhecidas e a organização da memória planejada, torna-se possível realizar a geração do código *assembly*. Nessa etapa, ocorre a tradução de cada quádrupla em uma sequência de instruções, respeitando os formatos e o conjunto de instruções da arquitetura, conforme apresentado nas Tabelas 1, 2, 3 e 4. Para isso, são considerados os registradores físicos, os deslocamentos de memória e a convenção de chamada de funções, adequando o programa abstrato efetivamente para o *assembly* da arquitetura de destino.

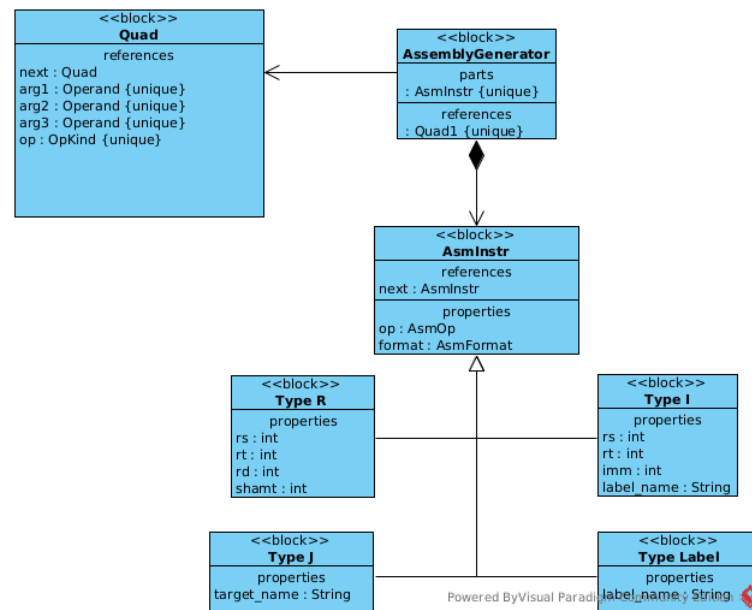
4.3.1 Modelagem

4.4 Diagrama de Blocos

A organização do módulo de geração de código *assembly* é ilustrada no diagrama de blocos da Figura 13. Conforme apresentado, esse módulo opera diretamente sobre a lista de quádruplas produzida pela etapa anterior, percorrendo-a sequencialmente e emitindo, para cada quádrupla, a sequência de instruções *assembly* correspondente.

O produto desse módulo é uma lista encadeada de instruções *assembly*, em que cada nó é representado pela estrutura **AsmInstr**. Essa estrutura armazena a operação a ser executada, o formato da instrução — que pode ser do tipo R, I, J ou rótulo — e uma *union* cujos campos variam conforme esse formato. Para instruções do tipo R, são armazenados os três registradores envolvidos e o campo de deslocamento de bits; para o tipo I, os dois registradores, o imediato e, quando aplicável, o nome do rótulo de desvio; para o tipo J, apenas o nome do alvo do salto. O formato adicional de rótulo, exclusivo à lista de instruções *assembly* e sem correspondência direta nos formatos da arquitetura, é empregado para demarcar posições na sequência de instruções que servirão como destino de desvios condicionais e incondicionais. A organização de cada formato segue diretamente as definições das Tabelas 1, 2 e 3.

Figura 13 – Diagrama de blocos da geração de código *assembly*

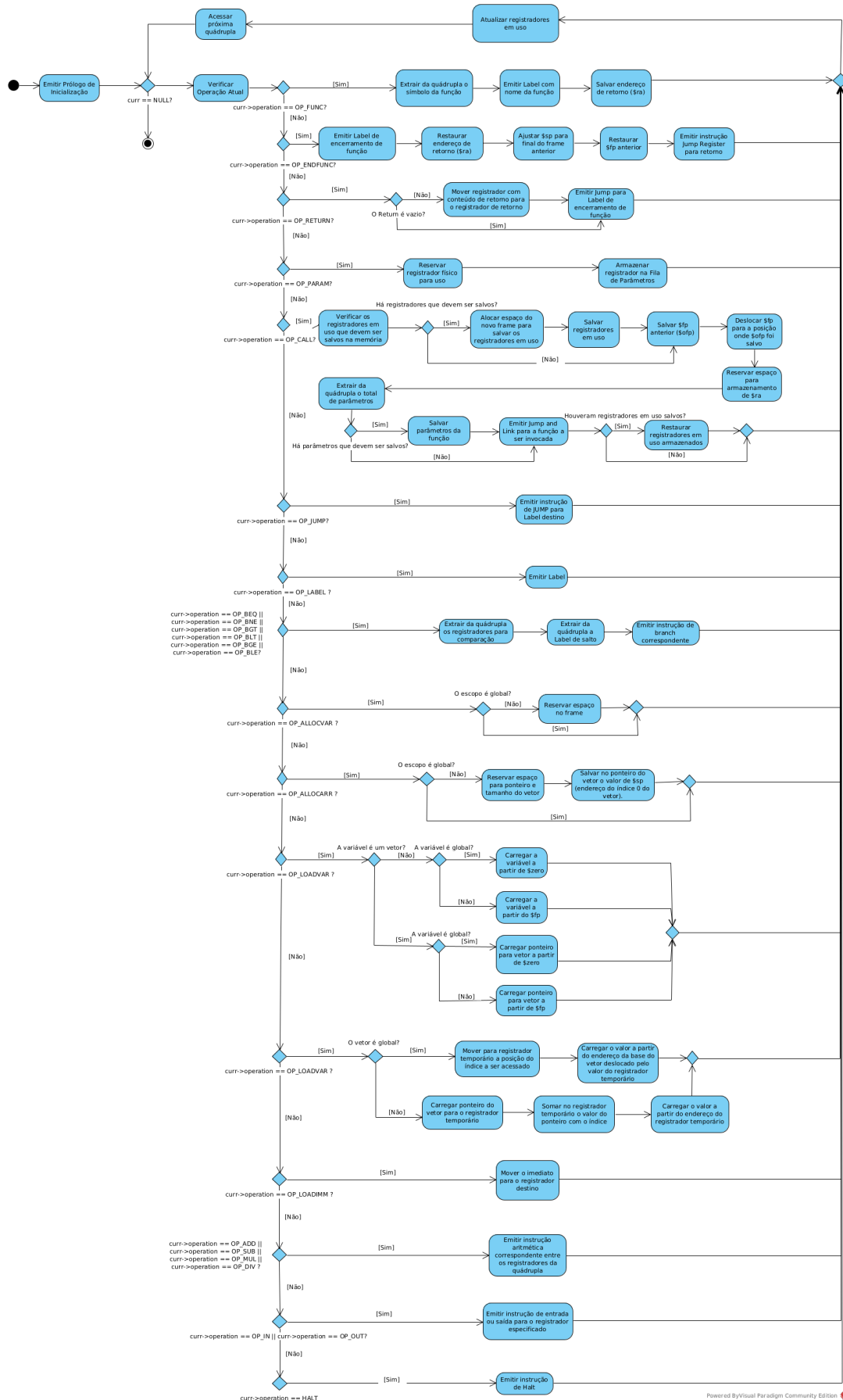


Fonte: O Autor

4.5 Diagrama de Atividades

A sequência de atividades do módulo de geração de código *assembly* é apresentada na Figura 14. Conforme mencionado, esse módulo percorre circularmente a lista de quádruplas, emitindo as instruções correspondentes a cada uma delas.

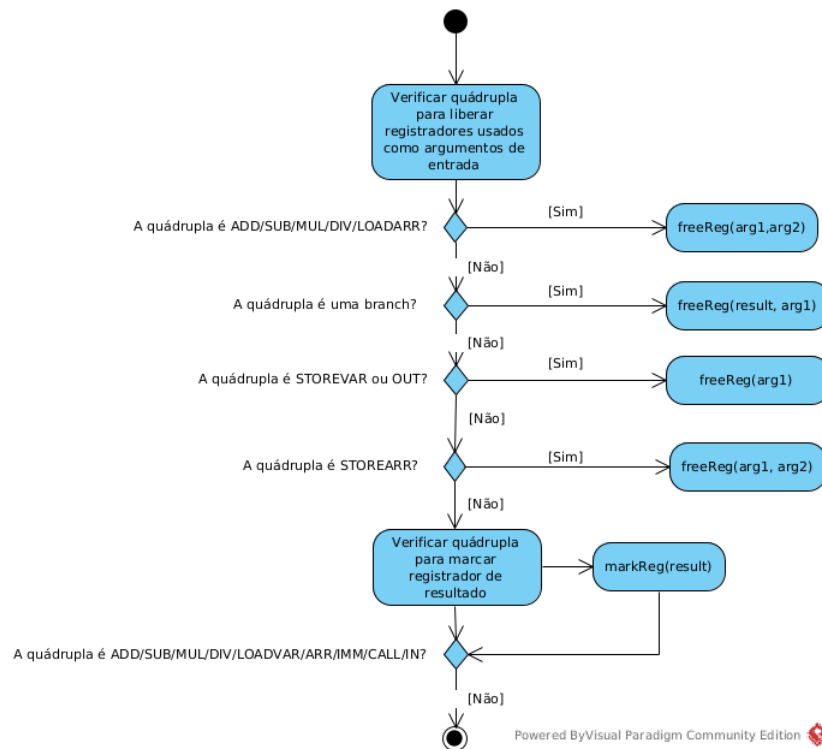
A execução tem início com a emissão do prólogo de inicialização, comum a todos os programas compilados. Nele, o *stack pointer* (**\$sp**) e o *frame pointer* (**\$fp**) são inicializados para o endereço **0x3FF**, correspondente ao topo da memória de dados. Em seguida, o **\$sp** é decrementado em uma posição. Esse deslocamento decorre da organização do quadro de ativação: na entrada de todo escopo, há uma reserva de espaço para o endereço de retorno, de modo que a inserção de parâmetros e variáveis locais ocorre a partir da segunda posição do quadro, já que a posição zero é destinada ao **\$ofp** e a posição um ao **\$ra**. Sem esse deslocamento inicial, as variáveis da função **main** estariam defasadas em uma posição em relação ao esperado pelo restante do módulo. Por fim, um desvio incondicional é emitido para a função **main**, transferindo o controle para o início do programa.

Figura 14 – Diagrama de atividades da geração de código *assembly*

Para cada quádrupla da lista, é emitida a sequência de instruções *assembly* correspondente ao seu operador e ao contexto de seus operandos. O detalhamento dessas emissões é apresentado na seção seguinte.

Após a emissão das instruções de cada quádrupla, é executada a atualização do vetor de controle de registradores em uso, por meio da função `updateRegsInUse`. Essa atualização opera em duas fases sequenciais. Na primeira, são liberados os registradores físicos cujos valores foram consumidos como operandos de entrada da quádrupla atual e que não serão mais necessários — como os operandos de operações aritméticas, os dois registradores comparados em desvios condicionais, o valor armazenado por instruções de escrita em memória, e os registradores de parâmetros e retorno. Na segunda fase, é marcado como em uso o registrador físico de destino da quádrupla, quando ela produz um novo valor — como ocorre nas operações aritméticas, nos carregamentos de memória e constantes, nas chamadas de função e nas leituras de entrada. Esse controle é o que permite ao módulo identificar quais registradores precisam ser preservados antes de cada chamada de função e reutilizar registradores físicos de forma segura ao longo de toda a tradução. O funcionamento detalhado dessa atividade é ilustrado no subdiagrama da Figura 15.

Figura 15 – Diagrama de atividades da atualização de registradores em uso



Fonte: O Autor

Concluída a atualização, o fluxo avança para a próxima quádrupla da lista, repetindo o ciclo até que todas tenham sido processadas e a lista de instruções *assembly* esteja completa.

4.5.1 Geração de código *Assembly* para cada quádrupla

4.5.2 Aritmética

O processo de tradução das operações aritméticas consiste em um mapeamento direto entre as quádruplas intermediárias e as instruções nativas do tipo R da máquina-alvo. Como a fase de código inter-

mediário já garantiu que as expressões complexas fossem desmembradas em operações de no máximo três operandos, o gerador de código *assembly* precisa apenas converter os registradores virtuais intermediários em registradores físicos da CPU.

Para isso, a função de mapeamento garante que os operandos sejam alocados em registradores de propósito geral válidos (neste projeto, limitados a um conjunto de 26 registradores). Uma vez mapeados, a instrução correspondente (add, sub, mul ou div) é emitida, lendo os valores dos registradores físicos de origem e gravando o resultado no registrador físico de destino, sem a necessidade de interações adicionais com a memória.

Algoritmo 1 Tradução de ADD, SUB, MUL, DIV

```

1: $rd ← getPhysicalReg(arg1.reg_id)
2: $rs ← getPhysicalReg(arg2.reg_id)
3: $rt ← getPhysicalReg(arg3.reg_id)
4: emit op $rd, $rs, $rt                                ▷ op ∈ {add, sub, mul, div}

```

4.5.3 Memória

4.5.3.1 ALLOCVAR

A alocação de variáveis simples exige um tratamento distinto fundamentado no escopo do símbolo. Variáveis globais têm seus endereços resolvidos estaticamente pelo compilador a partir da base da memória, dispensando instruções de alocação em tempo de execução. Em contrapartida, as variáveis locais exigem o crescimento dinâmico da pilha, realizado por meio do decremento de posições no apontador de pilha (\$sp).

Algoritmo 2 Tradução de ALLOCVAR

```

1: if sym.scope = global then
2:   nenhuma instrução emitida
3: else
4:   emit addi $sp, $sp, -1
5: end if

```

4.5.3.2 ALLOCARR

Semelhante às variáveis simples, os vetores globais não emitem instruções dinâmicas. Para os vetores locais, no entanto, reserva-se o espaço total dos elementos somado a um *slot* adicional na pilha. Esse *slot* extra atuará como o ponteiro base do vetor dentro do *frame* de ativação, sendo imediatamente inicializado com o endereço do primeiro elemento.

Algoritmo 3 Tradução de ALLOCARR

```

1: if sym.scope = global then
2:   nenhuma instrução emitida
3: else
4:   emit addi $sp, $sp, -(size + 2) ▷ reserva espaço para os elementos e o ponteiro
5:   emit sw $sp, MEM[$fp + frameloc] ▷ armazena $sp como ponteiro de base do
    vetor; frameloc = -(sym.memloc + 2)
6: end if

```

4.5.3.3 LOADVAR

O carregamento de uma variável depende da natureza do símbolo armazenado. Se o símbolo representar um vetor, o que é carregado para o registrador é o seu endereço base. Caso seja uma variável simples, o valor armazenado é carregado diretamente. Em ambos os casos, símbolos globais são acessados por um deslocamento imediato em relação ao registrador **\$zero**, enquanto símbolos locais são acessados via um deslocamento negativo em relação ao ponteiro do *frame* atual (**\$fp**).

Algoritmo 4 Tradução de LOADVAR

```

1: $rd ← getPhysicalReg(arg1.reg_id)
2: if sym.kind = SYMB_ARR then
3:   if sym.scope = global then
4:     emit addi $rd, $zero, sym.loc           ▷ endereço estático do vetor
5:   else
6:     emit lw $rd, MEM[$fp + sym.offset]     ▷ ponteiro do vetor no frame
7:   end if
8: else
9:   if sym.scope = global then
10:    emit lw $rd, MEM[$zero + sym.loc]
11:   else
12:    emit lw $rd, MEM[$fp + sym.offset]
13:   end if
14: end if

```

4.5.3.4 LOADARR

A manipulação de posições indexadas de vetores demanda o cálculo dinâmico do endereço efetivo. O gerador emite instruções para somar o valor do índice ao endereço base do vetor a partir do uso do registrador temporário (**\$tmp**). Se for um vetor local, esse endereço base é primeiro recuperado da pilha; se for global, o índice é somado diretamente ao deslocamento estático utilizando o **\$tmp**.

Algoritmo 5 Tradução de LOADARR

```

1: $rd ← getPhysicalReg(arg1.reg_id)
2: $idx ← getPhysicalReg(arg3.reg_id)         ▷ registrador com o índice
3: if sym.scope = global then
4:   emit add $tmp, $zero, $idx               ▷ $tmp = $idx
5:   emit lw $rd, MEM[$tmp + sym.memloc]
6: else
7:   emit lw $tmp, MEM[$fp + frameloc]       ▷ recupera ponteiro do vetor;
   frameloc = -(sym.memloc + 2)
8:   emit add $tmp, $tmp, $idx               ▷ soma índice ao ponteiro
9:   emit lw $rd, MEM[$tmp + 0]
10: end if

```

4.5.3.5 LOADIMM

O carregamento de valores imediatos (constantes numéricas) é realizado de forma direta, emitindo-se uma instrução de soma do valor literal com o registrador **\$zero** e depositando o resultado no registrador físico de destino.

Algoritmo 6 Tradução de LOADIMM

```

1:  $\$rd \leftarrow \text{getPhysicalReg}(\text{arg1.reg\_id})$ 
2: emit addi  $\$rd, \$zero, imm$ 

```

4.5.3.6 STOREVAR

O armazenamento em uma variável simples escreve o valor presente no registrador de origem diretamente na posição de memória correspondente. Para o escopo global, o acesso parte de $\$zero$; já para o local, o acesso parte de $\$fp$.

Algoritmo 7 Tradução de STOREVAR

```

1:  $\$rs \leftarrow \text{getPhysicalReg}(\text{arg2.reg\_id})$ 
2: if  $\text{sym.scope} = \text{global}$  then
3:   emit sw  $\$rs, MEM[\$zero + \text{sym.loc}]$ 
4: else
5:   emit sw  $\$rs, MEM[\$fp + \text{sym.offset}]$ 
6: end if

```

4.5.3.7 STOREARR

O armazenamento em vetor segue o mesmo cálculo de endereço efetivo de LOADARR.

Algoritmo 8 Tradução de STOREARR

```

1:  $\$rs \leftarrow \text{getPhysicalReg}(\text{arg2.reg\_id})$ 
2:  $\$idx \leftarrow \text{getPhysicalReg}(\text{arg3.reg\_id})$ 
3: if  $\text{sym.scope} = \text{global}$  then
4:   emit add  $\$tmp, \$zero, \$idx$   $\triangleright \$tmp = \$idx$ 
5:   emit sw  $\$rs, MEM[\$tmp + \text{sym.memloc}]$ 
6: else
7:   emit lw  $\$tmp, MEM[\$fp + \text{frameloc}]$   $\triangleright \text{frameloc} = -(\text{sym.memloc} + 2)$ 
8:   emit add  $\$tmp, \$tmp, \$idx$ 
9:   emit sw  $\$rs, MEM[\$tmp + 0]$ 
10: end if

```

4.5.4 Controle de Fluxo

Os desvios condicionais seguem o mesmo padrão, mapeando os dois registradores de origem e emitindo a instrução de desvio correspondente com o rótulo de destino já definido pelo código intermediário. O cálculo do *offset* relativo ao endereço de destino é realizado na fase de tradução binária.

Algoritmo 9 Tradução de BEQ, BNE, BLT, BGT, BLE, BGE

```

1:  $\$rs \leftarrow \text{getPhysicalReg}(\text{arg1.reg\_id})$ 
2:  $\$rt \leftarrow \text{getPhysicalReg}(\text{arg2.reg\_id})$ 
3: emit op  $\$rs, \$rt, label \triangleright \text{op} \in \{\text{beq}, \text{bne}, \text{blt}, \text{bgt}, \text{ble}, \text{bge}\}, label = \text{arg3.label\_name}$ 

```

Para os desvios incondicionais, a instrução **JUMP** é mapeada diretamente para uma instrução do tipo J, alterando o fluxo de execução para o rótulo especificado. A quádrupla **LABEL**, por sua vez, não gera uma instrução executável, limitando-se a demarcar uma posição literal no arquivo *assembly* gerado.

Algoritmo 10 Tradução de JUMP e LABEL

```

1: JUMP: emit j label                                ▷ label = arg1.label_name
2: LABEL: emit label:                                  ▷ demarca posição na lista de instruções

```

A instrução de parada não possui operandos e emite diretamente o comando de interrupção (**halt**), finalizando a execução do programa no processador.

Algoritmo 11 Tradução de HALT

```

1: emit halt

```

4.5.5 Funções

4.5.5.1 FUNC e ENDFUNC

O prólogo e o epílogo das sub-rotinas são fundamentais para a preservação de contexto. O início de uma função (**FUNC**) emite seu rótulo e, caso não seja a função **main**, salva o endereço de retorno (**\$ra**) na pilha para prevenir falhas em chamadas aninhadas. O encerramento (**ENDFUNC**) da função **main** emite a instrução de parada; para as demais, o epílogo posiciona um rótulo de saída e reverte o *frame*, restaurando o **\$ra**, o apontador do *frame* anterior (**\$ofp**), reajustando a pilha e executando o salto de retorno ao chamador.

Algoritmo 12 Tradução de FUNC

```

1: emit sym.name:                                       ▷ rótulo da função
2: if sym.name ≠ main then
3:   emit sw $ra, MEM[$fp + (-1)]  ▷ salva endereço de retorno no slot 1 do frame
4: end if

```

Algoritmo 13 Tradução de ENDFUNC

```

1: if sym.name = main then
2:   emit halt
3: else
4:   emit sym.name_end:                                ▷ rótulo de encerramento, destino dos RETURN
5:   emit lw $ra, MEM[$fp + (-1)]                      ▷ restaura endereço de retorno
6:   emit addi $sp, $fp, 1                             ▷ reajusta o stack pointer
7:   emit lw $fp, MEM[$fp + 0]  ▷ restaura $ofp usando $fp ainda não atualizado
8:   emit jr $ra                                       ▷ retorna ao chamador
9: end if

```

4.5.5.2 PARAM

Durante o preparo de uma invocação, a quádrupla **PARAM** não emite instruções na saída. O gerador apenas acumula os registradores físicos correspondentes aos argumentos em uma lista interna temporária, aguardando o momento exato da chamada para empilhá-los todos juntos na ordem correta.

Algoritmo 14 Tradução de PARAM

-
- ```

1: $\$rs \leftarrow \text{getPhysicalReg}(\text{arg1.reg_id})$
2: $\text{pendingParams} += \rs ▷ acumula para empilhamento em CALL

```
- 

**4.5.5.3 CALL**

A tradução de **CALL** concentra a maior sequência de instruções do módulo, sendo responsável por quatro etapas em sequência: a preservação do contexto da função chamadora, a construção do quadro de ativação da função chamada, o empilhamento dos argumentos e, ao retorno, a restauração do estado anterior.

A primeira etapa preserva os registradores físicos que estão atualmente em uso pela função chamadora, identificados pelo vetor de controle **regInUse**. Como a função chamada pode utilizar livremente qualquer registrador físico durante sua execução, os valores que o chamador ainda precisa após o retorno precisam ser salvos antes da transferência de controle. Para isso, o **\$sp** é decrementado em um único passo pelo total de registradores a salvar, e cada um deles é armazenado sequencialmente nas posições correspondentes da pilha.

A segunda etapa constrói o quadro de ativação da função chamada. Primeiro, o valor atual do **\$fp**, que representa a base do quadro da função chamadora, é empilhado e passa a ser o **\$ofp**, necessário para restaurar o quadro ao retorno. Em seguida, o **\$fp** é atualizado para apontar para essa posição recém-empilhada, estabelecendo a nova base do quadro. Por fim, o **\$sp** é decrementado mais uma posição para reservar o slot do endereço de retorno (**\$ra**), que será preenchido pela própria instrução **jal** no momento do salto.

A terceira etapa empilha os argumentos da chamada. Os registradores acumulados pelas quádruplas **PARAM** anteriores, armazenados em **pendingParams**, são empilhados sequencialmente na ordem em que foram registrados. Após o empilhamento, a instrução **jal** é emitida, transferindo o controle para a função chamada e salvando automaticamente o endereço de retorno em **\$ra**.

Ao retorno da função chamada, caso ela não seja do tipo **void**, o valor de retorno disponível em **\$ret** é copiado para o registrador físico de destino da quádrupla **CALL**. Por fim, os registradores que haviam sido preservados no início são restaurados a partir das posições em que foram salvos, e o **\$sp** é reajustado em um único passo para desfazer o espaço reservado para eles, devolvendo a pilha ao estado que estava antes da chamada.

**Algoritmo 15** Tradução de CALL

---

```

1: if countSavedRegs > 0 then
2: emit addi $sp, $sp, -countSavedRegs ▷ reserva espaço para todos os
 registradores em um passo
3: for cada $r em regInUse do
4: emit sw $r, MEM[$sp + offset] ▷ salva cada registrador em uso
5: end for
6: end if
7: emit addi $sp, $sp, -1
8: emit sw $fp, MEM[$sp + 0] ▷ empilha o $ofp
9: emit add $fp, $sp, $zero ▷ $fp aponta para o $ofp recém-empilhado
10: emit addi $sp, $sp, -1 ▷ reserva slot para $ra
11: for cada $p em pendingParams do
12: emit addi $sp, $sp, -1
13: emit sw $p, MEM[$sp + 0] ▷ empilha cada argumento
14: end for
15: emit jal sym.name
16: if sym.type ≠ void then
17: $rd ← getPhysicalReg(arg1.reg_id)
18: emit add $rd, $ret, $zero ▷ copia valor de retorno para registrador de destino
19: end if
20: if countSavedRegs > 0 then
21: for cada $r em regInUse (ordem crescente) do
22: emit lw $r, MEM[$sp + offset]
23: end for
24: emit addi $sp, $sp, +countSavedRegs ▷ restaura o stack pointer em um único
 passo
25: end if

```

---

## 4.5.5.4 RETURN

A instrução de retorno avalia se há um operando a ser devolvido. Caso exista, o valor é copiado para o registrador físico de retorno padronizado da arquitetura **\$ret**. Na sequência, executa-se um desvio incondicional para o rótulo de encerramento da função corrente, direcionando a responsabilidade de limpeza do *frame* ao bloco de código gerado pelo **ENDFUNC**.

**Algoritmo 16** Tradução de RETURN

---

```

1: if arg1 ≠ vazio then
2: $rs ← getPhysicalReg(arg1.reg_id)
3: emit add $ret, $rs, $zero ▷ move valor de retorno para $ret
4: end if
5: emit j currentFunc_end ▷ desvia para o rótulo de encerramento da função corrente

```

---

## 4.5.6 Entrada e Saída

O suporte às operações básicas de entrada e saída é simples e direto, sendo mapeado para instruções dedicadas de *hardware* do processador alvo. A leitura de dados (**INPUT**) aciona a instrução **in**,

que carrega a entrada do sistema no registrador físico de destino. Analogamente, a escrita (**OUTPUT**) emite a instrução **out**, transferindo o valor do registrador de origem correspondente em uma única operação.

**Algoritmo 17** Tradução de INPUT e OUTPUT

```
1: INPUT:
2: $rd ← getPhysicalReg(arg1.reg_id)
3: emit in $rd
4:
5: OUTPUT:
6: $rs ← getPhysicalReg(arg2.reg_id)
7: emit out $rs
```

## 4.6 Geração do código executável

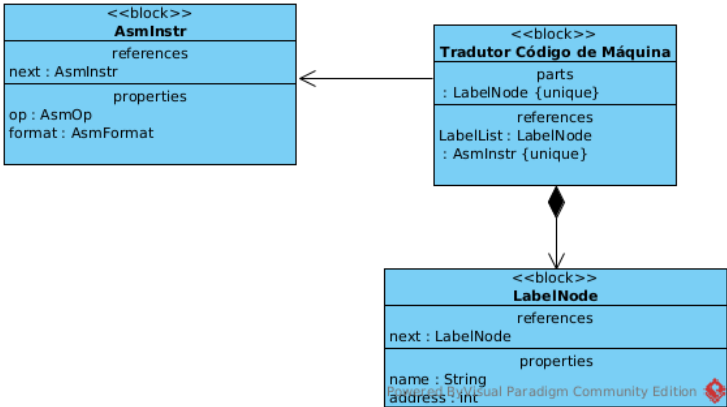
A geração do código executável é a etapa final da fase de síntese. A partir da lista de instruções *assembly* produzida pela etapa anterior, esse módulo é responsável por traduzir cada instrução para sua representação binária de 32 bits, seguindo o formato de instrução da arquitetura alvo. O resultado é gravado como uma sequência de cadeias binárias, uma por instrução, prontas para serem carregadas pela memória de instruções do processador.

### 4.6.1 Modelagem

### 4.6.2 Diagrama de Blocos

A organização estrutural desse módulo é ilustrada no diagrama de blocos da Figura 16. O bloco principal, o tradutor binário, depende da lista de instruções *assembly* produzida pelo módulo anterior e compõe internamente uma tabela de rótulos, responsável por associar cada nome de rótulo ao seu endereço numérico na memória de instruções. Essa tabela é consultada durante a codificação de instruções que referenciam rótulos como destino — desvios condicionais, saltos incondicionais e chamadas de função —, sendo o único dado interno que o módulo precisa manter além da própria lista de entrada.

Figura 16 – Diagrama de blocos da geração de código executável



Fonte: O Autor

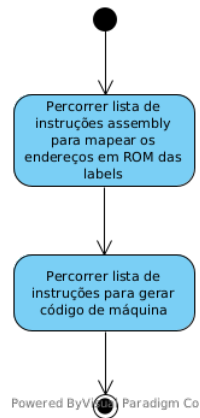
### 4.6.3 Diagrama de Atividades

O diagrama de atividades da Figura 17 apresenta o fluxo de execução do módulo de geração de código executável. O processo é organizado em duas passadas sucessivas sobre a lista de instruções *assembly*.

A primeira passada percorre a lista integralmente sem emitir código, com o único propósito de mapear cada rótulo ao seu endereço numérico na memória de instruções. Apenas instruções executáveis avançam o contador de endereços — rótulos são registrados na tabela mas não ocupam posição na ROM. Essa passada prévia é indispensável porque desvios e chamadas referenciam rótulos que podem estar posicionados adiante na lista, e seus endereços só podem ser calculados corretamente após o mapeamento completo.

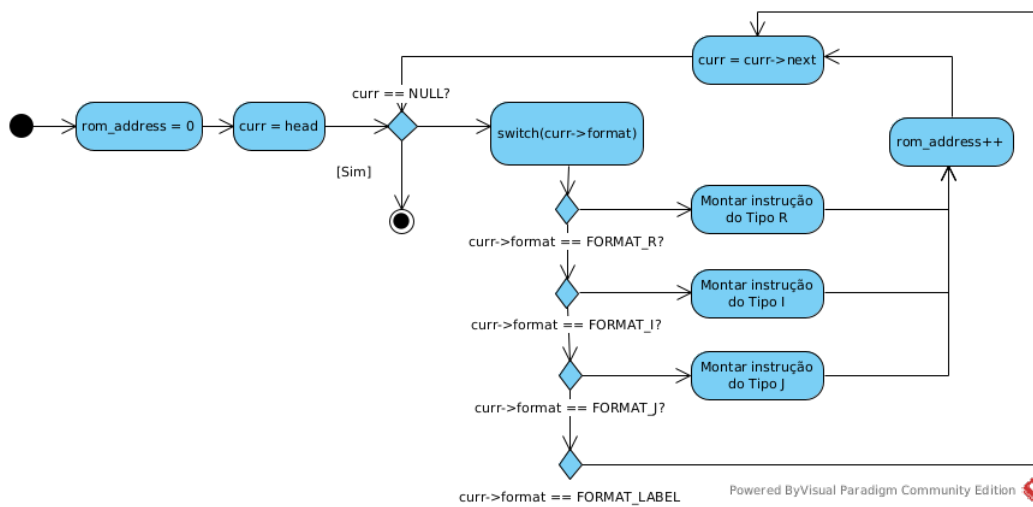
A segunda passada, detalhada na Figura 18, percorre novamente a lista e codifica cada instrução. Rótulos são ignorados, pois já cumpriram seu papel na primeira passada. Para cada instrução executável, o formato determina como a palavra de 32 bits é montada: instruções do tipo R têm seus campos de registradores e código de função posicionados por deslocamento de bits; instruções do tipo I têm seu campo de dezesseis bits preenchido com o imediato, o deslocamento de memória ou o *offset* de desvio relativo, calculado pela diferença entre o endereço do rótulo de destino — consultado na tabela preenchida na primeira passada — e o endereço da instrução subsequente; instruções do tipo J têm os vinte e seis bits de endereço preenchidos com o endereço absoluto do rótulo de destino. A palavra resultante é então impressa bit a bit, do mais significativo ao menos significativo, e o contador de endereços é incrementado antes de avançar para a próxima instrução.

Figura 17 – Diagrama de atividades da geração de código executável



Fonte: O Autor

Figura 18 – Diagrama de atividades da tradução da lista de instruções



Fonte: O Autor

## 5 Exemplos

Nessa seção serão exibidos os exemplos de códigos compilados, apresentando seu código-fonte em C- e sua equivalência para código intermediário e a tradução das quádruplas tanto para código *assembly* quanto para código executável. Dentre os exemplos, há o programa `gcd.cm`, que calcula o máximo divisor comum entre duas entradas pelo algoritmo de Euclides; o código `fact.cm`, que computa o fatorial de uma entrada; e o `sort.cm`, que ordena por seleção um vetor global de dez elementos lidos da entrada.

### 5.1 Fact

O programa `fact.cm`, apresentado no Listing 5.1, calcula o fatorial de um número inteiro lido da entrada, implementando o algoritmo recursivo clássico: para  $n \leq 0$ , retorna 1 como caso base; caso contrário, retorna  $n \times \text{fact}(n - 1)$ .

```

1 int fact(int n) {
2 if (n <= 0) {
3 return 1;
4 }
5 return n * fact(n - 1);
6 }
7
8 void main(void) {
9 int n;
10 n = input();
11 output(fact(n));
12 }
```

Listing 5.1 – Código-fonte de `fact.cm`

Apesar de sua estrutura simples, esse exemplo exercita aspectos relevantes do compilador. A estrutura condicional `if` sem bloco `else` é traduzida com um único rótulo de saída, demonstrando o caso mais simples de controle de fluxo. A chamada recursiva exige a construção completa do quadro de ativação a cada invocação, incluindo o salvamento e a restauração do endereço de retorno. O aspecto mais significativo, porém, está na multiplicação `n * fact(n-1)`: o valor de `n` precisa ser preservado durante a chamada recursiva, pois será utilizado como operando da multiplicação somente após o retorno. Para isso, o registrador que contém `n` é identificado como em uso pelo vetor de controle `regInUse` no momento da tradução de `CALL`, o que aciona o mecanismo de salvamento de contexto, de modo que o registrador é empilhado antes da chamada e restaurado após o retorno, garantindo que seu valor esteja disponível para a multiplicação subsequente.

A equivalência entre o código-fonte e o código intermediário é apresentado na Tabela 13. Já o mapeamento de cada quádrupla e suas sequências de instruções *assembly* e de máquina são ilustradas na Tabela 14.

Tabela 13 – Mapeamento entre o código-fonte `fact.cm` e o código intermediário gerado

| Código C-                            | Código Intermediário                                                                                                                                                                                  |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int fact(int n)</code>         | (FUNC, int, fact, -)<br>(ARG, int, n, fact)                                                                                                                                                           |
| <code>if (n &lt;= 0)</code>          | (LOADVAR, \$t1, n, -)<br>(LOADIMM, \$t2, 0, -)<br>(BGT, \$t1, \$t2, L1)                                                                                                                               |
| <code>return 1;</code>               | (LOADIMM, \$t3, 1, -)<br>(RETURN, \$t3, -, -)                                                                                                                                                         |
| <i>(fim do if)</i>                   | (LABEL, L1, -, -)                                                                                                                                                                                     |
| <code>return n * fact(n - 1);</code> | (LOADVAR, \$t4, n, -)<br>(LOADVAR, \$t5, n, -)<br>(LOADIMM, \$t6, 1, -)<br>(SUB, \$t7, \$t5, \$t6)<br>(PARAM, \$t7, -, -)<br>(CALL, \$t8, fact, 1)<br>(MUL, \$t9, \$t4, \$t8)<br>(RETURN, \$t9, -, -) |
| <code>}</code>                       | (ENDFUNC, fact, -, -)                                                                                                                                                                                 |
| <code>void main(void) {</code>       | (FUNC, void, main, -)                                                                                                                                                                                 |
| <code>int n;</code>                  | (ALLOCVAR, n, main, -)                                                                                                                                                                                |
| <code>n = input();</code>            | (INPUT, \$t10, -, -)<br>(STOREVAR, n, \$t10, -)                                                                                                                                                       |
| <code>output(fact(n));</code>        | (LOADVAR, \$t11, n, -)<br>(PARAM, \$t11, -, -)<br>(CALL, \$t12, fact, 1)<br>(OUTPUT, -, \$t12, -)                                                                                                     |
| <code>}</code>                       | (ENDFUNC, main, -, -)                                                                                                                                                                                 |

Fonte: O Autor.

Tabela 14 – Mapeamento de `fact.cm`: Quádrupla, Assembly e Binário

| Quádrupla | Assembly                             | Binário                                |
|-----------|--------------------------------------|----------------------------------------|
|           | <code>addi \$sp, \$zero, 1023</code> | 00000100000011101000000111111111       |
|           | <code>addi \$fp, \$zero, 1023</code> | 00000100000011110000000111111111       |
|           | <code>addi \$sp, \$sp, -1</code>     | 00000111101111011111111111111111       |
|           | <code>j main</code>                  | 01000000000000000000000000000000100010 |

Continua na próxima página...

Tabela 14 – continuação

|                         |                         |                                   |
|-------------------------|-------------------------|-----------------------------------|
| (FUNC, int, fact, -)    | fact:                   |                                   |
|                         | sw \$ra, -1(\$fp)       | 00011011110111111111111111111111  |
| (ARG, int, n, fact)     |                         |                                   |
| (LOADVAR, \$t1, n, -)   | lw \$t1, -2(\$fp)       | 00010111110000011111111111111110  |
| (LOADIMM, \$t2, 0, -)   | addi \$t2, \$zero, 0    | 00000100000000100000000000000000  |
| (BGT, \$t1, \$t2, L1)   | bgt \$t1, \$t2, L1      | 00101100001000100000000000000011  |
| (LOADIMM, \$t3, 1, -)   | addi \$t3, \$zero, 1    | 00000100000000110000000000000001  |
| (RETURN, \$t3, -, -)    | add \$ret, \$t3, \$zero | 00000000011000001110000000000000  |
|                         | j END_fact              | 010000000000000000000000000011110 |
| (LABEL, L1, -, -)       | L1:                     |                                   |
| (LOADVAR, \$t4, n, -)   | lw \$t4, -2(\$fp)       | 00010111110001001111111111111110  |
| (LOADVAR, \$t5, n, -)   | lw \$t5, -2(\$fp)       | 00010111110001011111111111111110  |
| (LOADIMM, \$t6, 1, -)   | addi \$t6, \$zero, 1    | 00000100000001100000000000000001  |
| (SUB, \$t7, \$t5, \$t6) | sub \$t7, \$t5, \$t6    | 00000000101001100011100000000001  |
| (PARAM, \$t7, -, -)     |                         |                                   |
| (CALL, \$t8, fact, 1)   | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |
|                         | sw \$t4, 0(\$sp)        | 00011011101001000000000000000000  |
|                         | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |
|                         | sw \$fp, 0(\$sp)        | 00011011101111100000000000000000  |
|                         | add \$fp, \$sp, \$zero  | 00000011101000001111000000000000  |
|                         | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |
|                         | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |
|                         | sw \$t7, 0(\$sp)        | 00011011101001110000000000000000  |
|                         | jal fact                | 01000100000000000000000000000100  |
|                         | add \$t8, \$ret, \$zero | 00000011100000000100000000000000  |
|                         | lw \$t4, 0(\$sp)        | 00010111101001000000000000000000  |
|                         | addi \$sp, \$sp, 1      | 00000111101111010000000000000001  |
| (MUL, \$t9, \$t4, \$t8) | mul \$t9, \$t4, \$t8    | 00000000100010000100100000000010  |
| (RETURN, \$t9, -, -)    | add \$ret, \$t9, \$zero | 00000001001000001110000000000000  |
|                         | j END_fact              | 010000000000000000000000000011110 |
| (ENDFUNC, fact, -, -)   | END_fact:               |                                   |
|                         | lw \$ra, -1(\$fp)       | 00010111110111111111111111111111  |

(ENDFUNC, fact, -, -)

Continua na proxima pagina...

Tabela 14 – continuação

|                         |                          |                                  |
|-------------------------|--------------------------|----------------------------------|
|                         | addi \$sp, \$fp, 1       | 00000111110111010000000000000001 |
|                         | lw \$fp, 0(\$fp)         | 00010111110111100000000000000000 |
|                         | jr \$ra                  | 00000011111000000000000000001000 |
| (FUNC, void, main, -)   | main:                    |                                  |
| (ALLOCVAR, n, main, -)  | addi \$sp, \$sp, -1      | 00000111110111101111111111111111 |
| (INPUT, \$t10, -, -)    | in \$t10                 | 00111000000010100000000000000000 |
| (STOREVAR, n, \$t10, -) | sw \$t10, -2(\$fp)       | 00011011110010101111111111111110 |
| (LOADVAR, \$t11, n, -)  | lw \$t11, -2(\$fp)       | 00010111110010111111111111111110 |
| (PARAM, \$t11, -, -)    |                          |                                  |
| (CALL, \$t12, fact, 1)  | addi \$sp, \$sp, -1      | 00000111110111101111111111111111 |
|                         | sw \$fp, 0(\$sp)         | 00011011110111110000000000000000 |
|                         | add \$fp, \$sp, \$zero   | 00000011101000001111000000000000 |
|                         | addi \$sp, \$sp, -1      | 00000111110111101111111111111111 |
|                         | addi \$sp, \$sp, -1      | 00000111110111101111111111111111 |
|                         | sw \$t11, 0(\$sp)        | 00011011110101011000000000000000 |
|                         | jal fact                 | 0100010000000000000000000000100  |
|                         | add \$t12, \$ret, \$zero | 00000011100000000110000000000000 |
| (OUTPUT, -, \$t12, -)   | out \$t12                | 00111101100000000000000000000000 |
| (ENDFUNC, main, -, -)   | halt                     | 01001100000000000000000000000000 |

Fonte: O autor.

## 5.2 GCD

O programa `gcd.cm`, apresentado no Listing 5.2, calcula o máximo divisor comum entre dois inteiros lidos da entrada, implementando o algoritmo de Euclides de forma recursiva: caso  $v = 0$ , retorna  $u$  como caso base; caso contrário, retorna  $\text{gcd}(v, u - u/v \times v)$ , onde a expressão de segundo parâmetro corresponde ao resto da divisão de  $u$  por  $v$ .

```

1 /* Um programa para calcular o mdc
2 segundo o algoritmo de Euclides. */
3
4 int gcd(int u, int v) {
5 if (v == 0) return u;
6 else return gcd(v, u - u / v * v);
7 /* u-u/v*v == u mod v */
8 }

```

```
9
10 void main(void) {
11 int x; int y;
12 x = input(); y = input();
13 output(gcd(x, y));
14 }
```

Listing 5.2 – Código-fonte de gcd.cm

Esse exemplo exercita recursão com múltiplos parâmetros, expressões aritméticas compostas e encadeamento direto entre chamada de função e operação de saída. A decomposição da expressão de cálculo de resto em operações elementares ilustra como o gerador de código intermediário preserva a ordem de avaliação de expressões complexas por meio de registradores temporários encadeados. A chamada recursiva com dois argumentos exercita o empilhamento de múltiplos parâmetros antes de cada CALL, e o encadeamento de gcd diretamente como argumento de output na função main demonstra o reaproveitamento imediato do registrador de retorno sem armazenamento intermediário.

O mapeamento do código-fonte para o código intermediário é apresentado na Tabela 15 e a tradução das quádruplas pode ser observada na Tabela 16.

Tabela 15 – Mapeamento entre o código-fonte gcd.cm e o código intermediário gerado

| Código C–                       | Código Intermediário                                                                                                                                                                                                                                                                                           |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| int gcd(int u, int v)           | (FUNC, int, gcd, -)<br>(ARG, int, u, gcd)<br>(ARG, int, v, gcd)                                                                                                                                                                                                                                                |
| if (v == 0)                     | (LOADVAR, \$t1, v, -)<br>(LOADIMM, \$t2, 0, -)<br>(BNE, \$t1, \$t2, L1)                                                                                                                                                                                                                                        |
| return u;                       | (LOADVAR, \$t3, u, -)<br>(RETURN, \$t3, -, -)                                                                                                                                                                                                                                                                  |
| <i>(salto do if para o fim)</i> | (JUMP, L2, -, -)                                                                                                                                                                                                                                                                                               |
| else                            | (LABEL, L1, -, -)                                                                                                                                                                                                                                                                                              |
| return gcd(v, u-u/v*v);         | (LOADVAR, \$t4, v, -)<br>(PARAM, \$t4, -, -)<br>(LOADVAR, \$t5, u, -)<br>(LOADVAR, \$t6, u, -)<br>(LOADVAR, \$t7, v, -)<br>(DIV, \$t8, \$t6, \$t7)<br>(LOADVAR, \$t9, v, -)<br>(MUL, \$t10, \$t8, \$t9)<br>(SUB, \$t11, \$t5, \$t10)<br>(PARAM, \$t11, -, -)<br>(CALL, \$t12, gcd, 2)<br>(RETURN, \$t12, -, -) |
| <i>(fim do if/else)</i>         | (LABEL, L2, -, -)                                                                                                                                                                                                                                                                                              |
| }                               | (ENDFUNC, gcd, -, -)                                                                                                                                                                                                                                                                                           |
| void main(void) {               | (FUNC, void, main, -)                                                                                                                                                                                                                                                                                          |
| int x; int y;                   | (ALLOCVAR, x, main, -)<br>(ALLOCVAR, y, main, -)                                                                                                                                                                                                                                                               |
| x = input();                    | (INPUT, \$t13, -, -)<br>(STOREVAR, x, \$t13, -)                                                                                                                                                                                                                                                                |
| y = input();                    | (INPUT, \$t14, -, -)<br>(STOREVAR, y, \$t14, -)                                                                                                                                                                                                                                                                |
| output(gcd(x, y));              | (LOADVAR, \$t15, x, -)<br>(PARAM, \$t15, -, -)<br>(LOADVAR, \$t16, y, -)<br>(PARAM, \$t16, -, -)<br>(CALL, \$t17, gcd, 2)<br>(OUTPUT, -, \$t17, -)                                                                                                                                                             |
| }                               | (ENDFUNC, main, -, -)                                                                                                                                                                                                                                                                                          |

Fonte: O Autor.

Tabela 16 – Mapeamento de gcd.cm: Quádrupla, Assembly e Binário

| Quádrupla                 | Assembly                | Binário                           |
|---------------------------|-------------------------|-----------------------------------|
|                           | addi \$sp, \$zero, 1023 | 00000100000111010000001111111111  |
|                           | addi \$fp, \$zero, 1023 | 00000100000111100000001111111111  |
|                           | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |
|                           | j main                  | 01000000000000000000000000000000  |
| (FUNC, int, gcd, -)       | gcd:                    |                                   |
|                           | sw \$ra, -1(\$fp)       | 00011011110111111111111111111111  |
| (ARG, int, u, gcd)        |                         |                                   |
| (ARG, int, v, gcd)        |                         |                                   |
| (LOADVAR, \$t1, v, -)     | lw \$t1, -3(\$fp)       | 00010111110000011111111111111101  |
| (LOADIMM, \$t2, 0, -)     | addi \$t2, \$zero, 0    | 00000100000000100000000000000000  |
| (BNE, \$t1, \$t2, L1)     | bne \$t1, \$t2, L1      | 001001000010001000000000000000100 |
| (LOADVAR, \$t3, u, -)     | lw \$t3, -2(\$fp)       | 00010111110000111111111111111110  |
| (RETURN, \$t3, -, -)      | add \$t28, \$t3, \$zero | 00000000011000001110000000000000  |
|                           | j END_gcd               | 01000000000000000000000000000000  |
| (JUMP, L2, -, -)          | j L2                    | 01000000000000000000000000000000  |
| (LABEL, L1, -, -)         | L1:                     |                                   |
| (LOADVAR, \$t4, v, -)     | lw \$t4, -3(\$fp)       | 00010111110001001111111111111101  |
| (PARAM, \$t4, -, -)       |                         |                                   |
| (LOADVAR, \$t5, u, -)     | lw \$t5, -2(\$fp)       | 00010111110001011111111111111110  |
| (LOADVAR, \$t6, u, -)     | lw \$t6, -2(\$fp)       | 00010111110001101111111111111110  |
| (LOADVAR, \$t7, v, -)     | lw \$t7, -3(\$fp)       | 00010111110001111111111111111101  |
| (DIV, \$t8, \$t6, \$t7)   | div \$t8, \$t6, \$t7    | 00000000110001110100000000000011  |
| (LOADVAR, \$t9, v, -)     | lw \$t9, -3(\$fp)       | 00010111110010011111111111111101  |
| (MUL, \$t10, \$t8, \$t9)  | mul \$t10, \$t8, \$t9   | 00000001000010010101000000000010  |
| (SUB, \$t11, \$t5, \$t10) | sub \$t11, \$t5, \$t10  | 00000000101010100101100000000001  |
| (PARAM, \$t11, -, -)      |                         |                                   |
|                           | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |
|                           | sw \$fp, 0(\$sp)        | 00011011101111100000000000000000  |
|                           | add \$fp, \$sp, \$zero  | 00000011101000001111000000000000  |
|                           | addi \$sp, \$sp, -1     | 00000111101111011111111111111111  |

(CALL, \$t12, gcd, 2)

Continua na proxima pagina...

Tabela 16 – continuação

|                         |                          |                                    |
|-------------------------|--------------------------|------------------------------------|
|                         | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
|                         | sw \$t4, 0(\$sp)         | 00011011101001000000000000000000   |
|                         | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
|                         | sw \$t11, 0(\$sp)        | 00011011101010110000000000000000   |
|                         | jal gcd                  | 01000100000000000000000000000100   |
| (RETURN, \$t12, -, -)   | add \$t12, \$t28, \$zero | 00000011100000000110000000000000   |
|                         | add \$t28, \$t12, \$zero | 00000001100000001110000000000000   |
|                         | j END_gcd                | 0100000000000000000000000000100000 |
| (LABEL, L2, -, -)       | L2:                      |                                    |
| (ENDFUNC, gcd, -, -)    | END_gcd:                 |                                    |
|                         | lw \$ra, -1(\$fp)        | 00010111110111111111111111111111   |
|                         | addi \$sp, \$fp, 1       | 00000111110111010000000000000001   |
|                         | lw \$fp, 0(\$fp)         | 00010111110111100000000000000000   |
|                         | jr \$ra                  | 00000011111000000000000000001000   |
| (FUNC, void, main, -)   | main:                    |                                    |
| (ALLOCVAR, x, main, -)  | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
| (ALLOCVAR, y, main, -)  | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
| (INPUT, \$t13, -, -)    | in \$t13                 | 00111000000011010000000000000000   |
| (STOREVAR, x, \$t13, -) | sw \$t13, -2(\$fp)       | 00011011110011011111111111111110   |
| (INPUT, \$t14, -, -)    | in \$t14                 | 00111000000011100000000000000000   |
| (STOREVAR, y, \$t14, -) | sw \$t14, -3(\$fp)       | 00011011110011101111111111111101   |
| (LOADVAR, \$t15, x, -)  | lw \$t15, -2(\$fp)       | 00010111110011111111111111111110   |
| (PARAM, \$t15, -, -)    |                          |                                    |
| (LOADVAR, \$t16, y, -)  | lw \$t16, -3(\$fp)       | 00010111110100001111111111111101   |
| (PARAM, \$t16, -, -)    |                          |                                    |
| (CALL, \$t17, gcd, 2)   | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
|                         | sw \$fp, 0(\$sp)         | 00011011101111100000000000000000   |
|                         | add \$fp, \$sp, \$zero   | 00000011101000001111000000000000   |
|                         | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
|                         | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |
|                         | sw \$t15, 0(\$sp)        | 00011011101011110000000000000000   |
|                         | addi \$sp, \$sp, -1      | 00000111101111011111111111111111   |

Continua na próxima página...

Tabela 16 – continuação

|                       |                          |                                  |
|-----------------------|--------------------------|----------------------------------|
|                       | sw \$t16, 0(\$sp)        | 00011011101100000000000000000000 |
|                       | jal gcd                  | 01000100000000000000000000000100 |
|                       | add \$t17, \$t28, \$zero | 00000011100000001000100000000000 |
| (OUTPUT, -, \$t17, -) | out \$t17                | 00111110001000000000000000000000 |
| (ENDFUNC, main, -, -) | halt                     | 01001100000000000000000000000000 |

Fonte: O autor.

## 5.3 Sort

O programa `sort.cm`, ilustrado no Listing 5.3, realiza a ordenação por seleção de um vetor global de dez elementos lidos da entrada, utilizando duas funções auxiliares: `minloc`, que localiza o índice do menor elemento em um intervalo do vetor, e `sort`, que reorganiza o vetor por meio de trocas sucessivas invocando `minloc` a cada iteração.

```

1 /* programa para ordenacao por selecao de
2 uma matriz com dez elementos. */
3
4 int vet[10];
5
6 int minloc(int a[], int low, int high) {
7 int i; int x; int k;
8 k = low;
9 x = a[low];
10 i = low + 1;
11 while (i < high) {
12 if (a[i] < x) {
13 x = a[i];
14 k = i;
15 }
16 i = i + 1;
17 }
18 return k;
19 }
20
21 void sort(int a[], int low, int high) {
22 int i; int k;
23 i = low;
24 while (i < high - 1) {
25 int t;

```

```

26 k = minloc(a, i, high);
27 t = a[k];
28 a[k] = a[i];
29 a[i] = t;
30 i = i + 1;
31 }
32 }
33
34 void main(void) {
35 int i;
36 i = 0;
37 while (i < 10) {
38 vet[i] = input();
39 i = i + 1;
40 }
41 sort(vet, 0, 10);
42 i = 0;
43 while (i < 10) {
44 output(vet[i]);
45 i = i + 1;
46 }
47 }

```

Listing 5.3 – Código-fonte de `sort.cm`

Esse é o exemplo mais abrangente do conjunto, exercitando simultaneamente vetores globais e locais, passagem de vetores como parâmetro, múltiplas funções com chamadas não recursivas, laços de repetição com estruturas condicionais aninhadas e declarações de variáveis em blocos internos. O vetor global `vet` demonstra a alocação estática, em que nenhuma instrução é emitida para reservar memória, enquanto a variável local `t`, declarada dentro do corpo do laço de `sort`, exercita a alocação em escopo de bloco. A passagem do vetor como argumento exercita o carregamento do endereço base por `LOADVAR`, em vez do valor, e seu acesso por índice por meio de `LOADARR` e `STOREARR`. A chamada a `minloc` dentro de `sort`, com três argumentos, aciona o mecanismo de salvamento de contexto, já que registradores em uso pela função chamadora precisam ser preservados durante a chamada.

Os resultados das quádruplas e da tradução para *assembly* e código de máquina são apresentados nas Tabelas 17 e 18.

Tabela 17 – Mapeamento entre o código-fonte `sort.cm` e o código intermediário gerado

| Código C–                 | Código Intermediário                     |
|---------------------------|------------------------------------------|
| <code>int vet[10];</code> | <code>(ALLOCARR, vet, 10, global)</code> |

*Continua na próxima página...*

Tabela 17 – continuação

|                                                     |                                                                                                                                                               |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int minloc(int a[], int low, int high)</code> | <code>(FUNC, int, minloc, -)</code><br><code>(ARGARR, int, a, minloc)</code><br><code>(ARG, int, low, minloc)</code><br><code>(ARG, int, high, minloc)</code> |
| <code>int i; int x; int k;</code>                   | <code>(ALLOCVAR, i, minloc, -)</code><br><code>(ALLOCVAR, x, minloc, -)</code><br><code>(ALLOCVAR, k, minloc, -)</code>                                       |
| <code>k = low;</code>                               | <code>(LOADVAR, \$t1, low, -)</code><br><code>(STOREVAR, k, \$t1, -)</code>                                                                                   |
| <code>x = a[low];</code>                            | <code>(LOADVAR, \$t2, low, -)</code><br><code>(LOADARR, \$t3, a, \$t2)</code><br><code>(STOREVAR, x, \$t3, -)</code>                                          |
| <code>i = low + 1;</code>                           | <code>(LOADVAR, \$t4, low, -)</code><br><code>(LOADIMM, \$t5, 1, -)</code><br><code>(ADD, \$t6, \$t4, \$t5)</code><br><code>(STOREVAR, i, \$t6, -)</code>     |
| <code>while (i &lt; high)</code>                    | <code>(LABEL, L1, -, -)</code><br><code>(LOADVAR, \$t7, i, -)</code><br><code>(LOADVAR, \$t8, high, -)</code><br><code>(BGE, \$t7, \$t8, L2)</code>           |
| <code>if (a[i] &lt; x)</code>                       | <code>(LOADVAR, \$t9, i, -)</code><br><code>(LOADARR, \$t10, a, \$t9)</code><br><code>(LOADVAR, \$t11, x, -)</code><br><code>(BGE, \$t10, \$t11, L3)</code>   |
| <code>x = a[i];</code>                              | <code>(LOADVAR, \$t12, i, -)</code><br><code>(LOADARR, \$t13, a, \$t12)</code><br><code>(STOREVAR, x, \$t13, -)</code>                                        |
| <code>k = i;</code>                                 | <code>(LOADVAR, \$t14, i, -)</code><br><code>(STOREVAR, k, \$t14, -)</code>                                                                                   |
| <i>(fim do if)</i>                                  | <code>(LABEL, L3, -, -)</code>                                                                                                                                |
| <code>i = i + 1;</code>                             | <code>(LOADVAR, \$t15, i, -)</code>                                                                                                                           |

*Continua na próxima página...*

Tabela 17 – continuação

|                                       |                               |
|---------------------------------------|-------------------------------|
|                                       | (LOADIMM, \$t16, 1, -)        |
|                                       | (ADD, \$t17, \$t15, \$t16)    |
|                                       | (STOREVAR, i, \$t17, -)       |
|                                       | (JUMP, L1, -, -)              |
| <i>(fim do while)</i>                 | (LABEL, L2, -, -)             |
| return k;                             | (LOADVAR, \$t18, k, -)        |
|                                       | (RETURN, \$t18, -, -)         |
| }                                     | (ENDFUNC, minloc, -, -)       |
| void sort(int a[], int low, int high) | (FUNC, void, sort, -)         |
|                                       | (ARGARR, int, a, sort)        |
|                                       | (ARG, int, low, sort)         |
|                                       | (ARG, int, high, sort)        |
| int i; int k;                         | (ALLOCVAR, i, sort, -)        |
|                                       | (ALLOCVAR, k, sort, -)        |
| i = low;                              | (LOADVAR, \$t19, low, -)      |
|                                       | (STOREVAR, i, \$t19, -)       |
| while (i < high-1)                    | (LABEL, L4, -, -)             |
|                                       | (LOADVAR, \$t20, i, -)        |
|                                       | (LOADVAR, \$t21, high, -)     |
|                                       | (LOADIMM, \$t22, 1, -)        |
|                                       | (SUB, \$t23, \$t21, \$t22)    |
|                                       | (BGE, \$t20, \$t23, L5)       |
| int t;                                | (ALLOCVAR, t, sort:block1, -) |
|                                       | (LOADVAR, \$t24, a, -)        |
|                                       | (PARAM, \$t24, -, -)          |
| k = minloc(a, i, high);               | (LOADVAR, \$t25, i, -)        |
|                                       | (PARAM, \$t25, -, -)          |
|                                       | (LOADVAR, \$t26, high, -)     |
|                                       | (PARAM, \$t26, -, -)          |
|                                       | (CALL, \$t27, minloc, 3)      |
|                                       | (STOREVAR, k, \$t27, -)       |

Continua na próxima página...

Tabela 17 – continuação

|                                |                                                                                                                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>t = a[k];</code>         | <code>(LOADVAR, \$t28, k, -)</code><br><code>(LOADARR, \$t29, a, \$t28)</code><br><code>(STOREVAR, t, \$t29, -)</code>                                                                         |
| <code>a[k] = a[i];</code>      | <code>(LOADVAR, \$t30, i, -)</code><br><code>(LOADARR, \$t31, a, \$t30)</code><br><code>(LOADVAR, \$t32, k, -)</code><br><code>(STOREARR, a, \$t31, \$t32)</code>                              |
| <code>a[i] = t;</code>         | <code>(LOADVAR, \$t33, t, -)</code><br><code>(LOADVAR, \$t34, i, -)</code><br><code>(STOREARR, a, \$t33, \$t34)</code>                                                                         |
| <code>i = i + 1;</code>        | <code>(LOADVAR, \$t35, i, -)</code><br><code>(LOADIMM, \$t36, 1, -)</code><br><code>(ADD, \$t37, \$t35, \$t36)</code><br><code>(STOREVAR, i, \$t37, -)</code><br><code>(JUMP, L4, -, -)</code> |
| <i>(fim do while)</i>          | <code>(LABEL, L5, -, -)</code>                                                                                                                                                                 |
| <code>}</code>                 | <code>(ENDFUNC, sort, -, -)</code>                                                                                                                                                             |
| <code>void main(void) {</code> | <code>(FUNC, void, main, -)</code>                                                                                                                                                             |
| <code>int i;</code>            | <code>(ALLOCVAR, i, main, -)</code>                                                                                                                                                            |
| <code>i = 0;</code>            | <code>(LOADIMM, \$t38, 0, -)</code><br><code>(STOREVAR, i, \$t38, -)</code>                                                                                                                    |
| <code>while (i &lt; 10)</code> | <code>(LABEL, L6, -, -)</code><br><code>(LOADVAR, \$t39, i, -)</code><br><code>(LOADIMM, \$t40, 10, -)</code><br><code>(BGE, \$t39, \$t40, L7)</code>                                          |
| <code>vet[i] = input();</code> | <code>(INPUT, \$t41, -, -)</code><br><code>(LOADVAR, \$t42, i, -)</code><br><code>(STOREARR, vet, \$t41, \$t42)</code>                                                                         |
| <code>i = i + 1;</code>        | <code>(LOADVAR, \$t43, i, -)</code><br><code>(LOADIMM, \$t44, 1, -)</code><br><code>(ADD, \$t45, \$t43, \$t44)</code><br><code>(STOREVAR, i, \$t45, -)</code>                                  |

*Continua na próxima página...*

Tabela 17 – continuação

|                       |                                                                                                                                                                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                       | (JUMP, L6, -, -)                                                                                                                                                                |
| <i>(fim do while)</i> | (LABEL, L7, -, -)                                                                                                                                                               |
| sort(vet, 0, 10);     | (LOADVAR, \$t46, vet, -)<br>(PARAM, \$t46, -, -)<br>(LOADIMM, \$t47, 0, -)<br>(PARAM, \$t47, -, -)<br>(LOADIMM, \$t48, 10, -)<br>(PARAM, \$t48, -, -)<br>(CALL, \$t49, sort, 3) |
| i = 0;                | (LOADIMM, \$t50, 0, -)<br>(STOREVAR, i, \$t50, -)                                                                                                                               |
| while (i < 10)        | (LABEL, L8, -, -)<br>(LOADVAR, \$t51, i, -)<br>(LOADIMM, \$t52, 10, -)<br>(BGE, \$t51, \$t52, L9)                                                                               |
| output(vet[i]);       | (LOADVAR, \$t53, i, -)<br>(LOADARR, \$t54, vet, \$t53)<br>(OUTPUT, -, \$t54, -)                                                                                                 |
| i = i + 1;            | (LOADVAR, \$t55, i, -)<br>(LOADIMM, \$t56, 1, -)<br>(ADD, \$t57, \$t55, \$t56)<br>(STOREVAR, i, \$t57, -)<br>(JUMP, L8, -, -)                                                   |
| <i>(fim do while)</i> | (LABEL, L9, -, -)                                                                                                                                                               |
| }                     | (ENDFUNC, main, -, -)                                                                                                                                                           |

Fonte: O Autor.

Tabela 18 – Mapeamento de sort.cm: Quádrupla, Assembly e Binário

| Quádrupla | Assembly                | Binário                          |
|-----------|-------------------------|----------------------------------|
|           | addi \$sp, \$zero, 1023 | 00000100000111010000001111111111 |

Continua na próxima página...

Tabela 18 – continuação

|                             |                         |                                   |
|-----------------------------|-------------------------|-----------------------------------|
|                             | addi \$fp, \$zero, 1023 | 000001000001111100000001111111111 |
|                             | addi \$sp, \$sp, -1     | 000001111011110111111111111111111 |
|                             | j main                  | 01000000000000000000000001100101  |
| (ALLOCARR, vet, 10, global) |                         |                                   |
| (FUNC, int, minloc, -)      | minloc:                 |                                   |
|                             | sw \$ra, -1(\$fp)       | 000110111101111111111111111111111 |
| (ARGARR, int, a, minloc)    |                         |                                   |
| (ARG, int, low, minloc)     |                         |                                   |
| (ARG, int, high, minloc)    |                         |                                   |
| (ALLOCVAR, i, minloc, -)    | addi \$sp, \$sp, -1     | 000001111011110111111111111111111 |
| (ALLOCVAR, x, minloc, -)    | addi \$sp, \$sp, -1     | 000001111011110111111111111111111 |
| (ALLOCVAR, k, minloc, -)    | addi \$sp, \$sp, -1     | 000001111011110111111111111111111 |
| (LOADVAR, \$t1, low, -)     | lw \$t1, -3(\$fp)       | 00010111110000011111111111111101  |
| (STOREVAR, k, \$t1, -)      | sw \$t1, -7(\$fp)       | 0001101111000001111111111111001   |
| (LOADVAR, \$t2, low, -)     | lw \$t2, -3(\$fp)       | 0001011111000010111111111111101   |
| (LOADARR, \$t3, a, \$t2)    | lw \$tmp, -2(\$fp)      | 0001011111011011111111111111110   |
|                             | add \$tmp, \$tmp, \$t2  | 00000011011000101101100000000000  |
|                             | lw \$t3, 0(\$tmp)       | 00010111011000110000000000000000  |
| (STOREVAR, x, \$t3, -)      | sw \$t3, -6(\$fp)       | 0001101111000011111111111111010   |
| (LOADVAR, \$t4, low, -)     | lw \$t4, -3(\$fp)       | 0001011111000100111111111111101   |
| (LOADIMM, \$t5, 1, -)       | addi \$t5, \$zero, 1    | 00000100000001010000000000000001  |
| (ADD, \$t6, \$t4, \$t5)     | add \$t6, \$t4, \$t5    | 00000000100001010011000000000000  |
| (STOREVAR, i, \$t6, -)      | sw \$t6, -5(\$fp)       | 0001101111000110111111111111011   |
| (LABEL, L1, -, -)           | L1:                     |                                   |
| (LOADVAR, \$t7, i, -)       | lw \$t7, -5(\$fp)       | 0001011111000111111111111111011   |
| (LOADVAR, \$t8, high, -)    | lw \$t8, -4(\$fp)       | 0001011111001000111111111111100   |
| (BGE, \$t7, \$t8, L2)       | bge \$t7, \$t8, L2      | 0011010011101000000000000010010   |
| (LOADVAR, \$t9, i, -)       | lw \$t9, -5(\$fp)       | 0001011111001001111111111111011   |
| (LOADARR, \$t10, a, \$t9)   | lw \$tmp, -2(\$fp)      | 0001011111011011111111111111110   |
|                             | add \$tmp, \$tmp, \$t9  | 00000011011010011101100000000000  |
|                             | lw \$t10, 0(\$tmp)      | 00010111011010100000000000000000  |

Continua na próxima página...

Tabela 18 – continuação

|                            |                          |                                   |
|----------------------------|--------------------------|-----------------------------------|
| (LOADVAR, \$t11, x, -)     | lw \$t11, -6(\$fp)       | 0001011111001011111111111111010   |
| (BGE, \$t10, \$t11, L3)    | bge \$t10, \$t11, L3     | 00110101010010110000000000000111  |
| (LOADVAR, \$t12, i, -)     | lw \$t12, -5(\$fp)       | 0001011111001100111111111111011   |
| (LOADARR, \$t13, a, \$t12) | lw \$tmp, -2(\$fp)       | 000101111101101111111111111110    |
|                            | add \$tmp, \$tmp, \$t12  | 00000011011011001101100000000000  |
|                            | lw \$t13, 0(\$tmp)       | 00010111011011010000000000000000  |
| (STOREVAR, x, \$t13, -)    | sw \$t13, -6(\$fp)       | 0001101111001101111111111111010   |
| (LOADVAR, \$t14, i, -)     | lw \$t14, -5(\$fp)       | 0001011111001110111111111111011   |
| (STOREVAR, k, \$t14, -)    | sw \$t14, -7(\$fp)       | 0001101111001110111111111111001   |
| (LABEL, L3, -, -)          | L3:                      |                                   |
| (LOADVAR, \$t15, i, -)     | lw \$t15, -5(\$fp)       | 0001011111001111111111111111011   |
| (LOADIMM, \$t16, 1, -)     | addi \$t16, \$zero, 1    | 00000100000100000000000000000001  |
| (ADD, \$t17, \$t15, \$t16) | add \$t17, \$t15, \$t16  | 00000001111100001000100000000000  |
| (STOREVAR, i, \$t17, -)    | sw \$t17, -5(\$fp)       | 0001101111010001111111111111011   |
| (JUMP, L1, -, -)           | j L1                     | 010000000000000000000000000010011 |
| (LABEL, L2, -, -)          | L2:                      |                                   |
| (LOADVAR, \$t18, k, -)     | lw \$t18, -7(\$fp)       | 0001011111010010111111111111001   |
| (RETURN, \$t18, -, -)      | add \$ret, \$t18, \$zero | 00000010010000001110000000000000  |
|                            | j END_minloc             | 010000000000000000000000000010111 |
| (ENDFUNC, minloc, -, -)    | END_minloc:              |                                   |
|                            | lw \$ra, -1(\$fp)        | 00010111110111111111111111111111  |
|                            | addi \$sp, \$fp, 1       | 00000111110111010000000000000001  |
|                            | lw \$fp, 0(\$fp)         | 00010111110111100000000000000000  |
|                            | jr \$ra                  | 00000011111000000000000000001000  |
| (FUNC, void, sort, -)      | sort:                    |                                   |
|                            | sw \$ra, -1(\$fp)        | 00011011110111111111111111111111  |
| (ARGARR, int, a, sort)     |                          |                                   |
| (ARG, int, low, sort)      |                          |                                   |
| (ARG, int, high, sort)     |                          |                                   |
| (ALLOCVAR, i, sort, -)     | addi \$sp, \$sp, -1      | 00000111101111011111111111111111  |
| (ALLOCVAR, k, sort, -)     | addi \$sp, \$sp, -1      | 00000111101111011111111111111111  |

Continua na proxima pagina...

Tabela 18 – continuação

|                               |                         |                                  |
|-------------------------------|-------------------------|----------------------------------|
| (LOADVAR, \$t19, low, -)      | lw \$t19, -3(\$fp)      | 000101111101001111111111111101   |
| (STOREVAR, i, \$t19, -)       | sw \$t19, -5(\$fp)      | 000110111101001111111111111011   |
| (LABEL, L4, -, -)             | L4:                     |                                  |
| (LOADVAR, \$t20, i, -)        | lw \$t20, -5(\$fp)      | 000101111101010011111111111011   |
| (LOADVAR, \$t21, high, -)     | lw \$t21, -4(\$fp)      | 000101111101010111111111111100   |
| (LOADIMM, \$t22, 1, -)        | addi \$t22, \$zero, 1   | 00000100000101100000000000000001 |
| (SUB, \$t23, \$t21, \$t22)    | sub \$t23, \$t21, \$t22 | 00000010101101101011100000000001 |
| (BGE, \$t20, \$t23, L5)       | bge \$t20, \$t23, L5    | 0011011010010111000000000101000  |
| (ALLOCVAR, t, sort:block1, -) | addi \$sp, \$sp, -1     | 00000111101111011111111111111111 |
| (LOADVAR, \$t24, a, -)        | lw \$t24, -2(\$fp)      | 0001011111011000111111111111110  |
| (PARAM, \$t24, -, -)          |                         |                                  |
| (LOADVAR, \$t25, i, -)        | lw \$t25, -5(\$fp)      | 0001011111011001111111111111011  |
| (PARAM, \$t25, -, -)          |                         |                                  |
| (LOADVAR, \$t26, high, -)     | lw \$t26, -4(\$fp)      | 0001011111011010111111111111100  |
| (PARAM, \$t26, -, -)          |                         |                                  |
| (CALL, \$t27, minloc, 3)      | addi \$sp, \$sp, -1     | 00000111101111011111111111111111 |
|                               | sw \$fp, 0(\$sp)        | 00011011101111110000000000000000 |
|                               | add \$fp, \$sp, \$zero  | 00000011101000001111000000000000 |
|                               | addi \$sp, \$sp, -1     | 00000111101111011111111111111111 |
|                               | addi \$sp, \$sp, -1     | 00000111101111011111111111111111 |
|                               | sw \$t24, 0(\$sp)       | 00011011101110000000000000000000 |
|                               | addi \$sp, \$sp, -1     | 00000111101111011111111111111111 |
|                               | sw \$t25, 0(\$sp)       | 00011011101110010000000000000000 |
|                               | addi \$sp, \$sp, -1     | 00000111101111011111111111111111 |
|                               | sw \$t26, 0(\$sp)       | 00011011101110100000000000000000 |
|                               | jal minloc              | 0100010000000000000000000000100  |
|                               | add \$t1, \$ret, \$zero | 00000011100000000000100000000000 |
| (STOREVAR, k, \$t27, -)       | sw \$t1, -6(\$fp)       | 0001101111000001111111111111010  |
| (LOADVAR, \$t28, k, -)        | lw \$t2, -6(\$fp)       | 0001011111000010111111111111010  |
| (LOADARR, \$t29, a, \$t28)    | lw \$tmp, -2(\$fp)      | 0001011111011011111111111111110  |
|                               | add \$tmp, \$tmp, \$t2  | 00000011011000101101100000000000 |

Continua na proxima pagina...

Tabela 18 – continuação

|                             |                        |                                    |
|-----------------------------|------------------------|------------------------------------|
|                             | lw \$t3, 0(\$tmp)      | 00010111011000110000000000000000   |
| (STOREVAR, t, \$t29, -)     | sw \$t3, -7(\$fp)      | 0001101111000011111111111111001    |
| (LOADVAR, \$t30, i, -)      | lw \$t4, -5(\$fp)      | 0001011111000100111111111111011    |
| (LOADARR, \$t31, a, \$t30)  | lw \$tmp, -2(\$fp)     | 000101111101101111111111111110     |
|                             | add \$tmp, \$tmp, \$t4 | 00000011011001001101100000000000   |
|                             | lw \$t5, 0(\$tmp)      | 00010111011001010000000000000000   |
| (LOADVAR, \$t32, k, -)      | lw \$t6, -6(\$fp)      | 0001011111000110111111111111010    |
| (STOREARR, a, \$t31, \$t32) | lw \$tmp, -2(\$fp)     | 000101111101101111111111111110     |
|                             | add \$tmp, \$tmp, \$t6 | 00000011011001101101100000000000   |
|                             | sw \$t5, 0(\$tmp)      | 00011011011001010000000000000000   |
| (LOADVAR, \$t33, t, -)      | lw \$t7, -7(\$fp)      | 0001011111000111111111111111001    |
| (LOADVAR, \$t34, i, -)      | lw \$t8, -5(\$fp)      | 0001011111001000111111111111011    |
| (STOREARR, a, \$t33, \$t34) | lw \$tmp, -2(\$fp)     | 000101111101101111111111111110     |
|                             | add \$tmp, \$tmp, \$t8 | 00000011011010001101100000000000   |
|                             | sw \$t7, 0(\$tmp)      | 00011011011001110000000000000000   |
| (LOADVAR, \$t35, i, -)      | lw \$t9, -5(\$fp)      | 0001011111001001111111111111011    |
| (LOADIMM, \$t36, 1, -)      | addi \$t10, \$zero, 1  | 00000100000010100000000000000001   |
| (ADD, \$t37, \$t35, \$t36)  | add \$t11, \$t9, \$t10 | 00000001001010100101100000000000   |
| (STOREVAR, i, \$t37, -)     | sw \$t11, -5(\$fp)     | 0001101111001011111111111111011    |
| (JUMP, L4, -, -)            | j L4                   | 0100000000000000000000000000110100 |
| (LABEL, L5, -, -)           | L5:                    |                                    |
| (ENDFUNC, sort, -, -)       | END_sort:              |                                    |
|                             | lw \$ra, -1(\$fp)      | 00010111110111111111111111111111   |
|                             | addi \$sp, \$fp, 1     | 00000111110111010000000000000001   |
|                             | lw \$fp, 0(\$fp)       | 00010111110111100000000000000000   |
|                             | jr \$ra                | 00000011111000000000000000001000   |
| (FUNC, void, main, -)       | main:                  |                                    |
| (ALLOCVAR, i, main, -)      | addi \$sp, \$sp, -1    | 00000111110111110111111111111111   |
| (LOADIMM, \$t38, 0, -)      | addi \$t12, \$zero, 0  | 00000100000011000000000000000000   |
| (STOREVAR, i, \$t38, -)     | sw \$t12, -2(\$fp)     | 00011011110011001111111111111110   |
| (LABEL, L6, -, -)           | L6:                    |                                    |

Continua na proxima pagina...

Tabela 18 – continuação

|                               |                          |                                     |
|-------------------------------|--------------------------|-------------------------------------|
| (LOADVAR, \$t39, i, -)        | lw \$t13, -2(\$fp)       | 00010111110011011111111111111110    |
| (LOADIMM, \$t40, 10, -)       | addi \$t14, \$zero, 10   | 00000100000011100000000000001010    |
| (BGE, \$t39, \$t40, L7)       | bge \$t13, \$t14, L7     | 00110101101011100000000000001001    |
| (INPUT, \$t41, -, -)          | in \$t15                 | 00111000000011110000000000000000    |
| (LOADVAR, \$t42, i, -)        | lw \$t16, -2(\$fp)       | 00010111110100001111111111111110    |
| (STOREARR, vet, \$t41, \$t42) | add \$tmp, \$zero, \$t16 | 00000000000100001101100000000000    |
|                               | sw \$t15, 0(\$tmp)       | 00011011011011110000000000000000    |
| (LOADVAR, \$t43, i, -)        | lw \$t17, -2(\$fp)       | 00010111110100001111111111111110    |
| (LOADIMM, \$t44, 1, -)        | addi \$t18, \$zero, 1    | 00000100000100100000000000000001    |
| (ADD, \$t45, \$t43, \$t44)    | add \$t19, \$t17, \$t18  | 00000010001100101001100000000000    |
| (STOREVAR, i, \$t45, -)       | sw \$t19, -2(\$fp)       | 00011011110100111111111111111110    |
| (JUMP, L6, -, -)              | j L6                     | 01000000000000000000000000001101000 |
| (LABEL, L7, -, -)             | L7:                      |                                     |
| (LOADVAR, \$t46, vet, -)      | addi \$t20, \$zero, 0    | 00000100000101000000000000000000    |
| (PARAM, \$t46, -, -)          |                          |                                     |
| (LOADIMM, \$t47, 0, -)        | addi \$t21, \$zero, 0    | 00000100000101010000000000000000    |
| (PARAM, \$t47, -, -)          |                          |                                     |
| (LOADIMM, \$t48, 10, -)       | addi \$t22, \$zero, 10   | 000001000001011000000000000001010   |
| (PARAM, \$t48, -, -)          |                          |                                     |
| (CALL, \$t49, sort, 3)        | addi \$sp, \$sp, -1      | 00000111101111011111111111111111    |
|                               | sw \$fp, 0(\$sp)         | 00011011101111100000000000000000    |
|                               | add \$fp, \$sp, \$zero   | 00000011101000001111000000000000    |
|                               | addi \$sp, \$sp, -1      | 00000111101111011111111111111111    |
|                               | addi \$sp, \$sp, -1      | 00000111101111011111111111111111    |
|                               | sw \$t20, 0(\$sp)        | 00011011101101000000000000000000    |
|                               | addi \$sp, \$sp, -1      | 00000111101111011111111111111111    |
|                               | sw \$t21, 0(\$sp)        | 00011011101101010000000000000000    |
|                               | addi \$sp, \$sp, -1      | 00000111101111011111111111111111    |
|                               | sw \$t22, 0(\$sp)        | 00011011101101100000000000000000    |
|                               | jal sort                 | 0100010000000000000000000000101111  |
| (LOADIMM, \$t50, 0, -)        | addi \$t24, \$zero, 0    | 00000100000110000000000000000000    |

Continua na proxima pagina...

Tabela 18 – continuação

|                              |                         |                                   |
|------------------------------|-------------------------|-----------------------------------|
| (STOREVAR, i, \$t50, -)      | sw \$t24, -2(\$fp)      | 00011011110110001111111111111110  |
| (LABEL, L8, -, -)            | L8:                     |                                   |
| (LOADVAR, \$t51, i, -)       | lw \$t25, -2(\$fp)      | 00010111110110011111111111111110  |
| (LOADIMM, \$t52, 10, -)      | addi \$t26, \$zero, 10  | 00000100000110100000000000001010  |
| (BGE, \$t51, \$t52, L9)      | bge \$t25, \$t26, L9    | 00110111001110100000000000001001  |
| (LOADVAR, \$t53, i, -)       | lw \$t1, -2(\$fp)       | 00010111110000011111111111111110  |
| (LOADARR, \$t54, vet, \$t53) | add \$tmp, \$zero, \$t1 | 00000000000000011101100000000000  |
|                              | lw \$t2, 0(\$tmp)       | 00010111011000100000000000000000  |
| (OUTPUT, -, \$t54, -)        | out \$t2                | 00111100010000000000000000000000  |
| (LOADVAR, \$t55, i, -)       | lw \$t3, -2(\$fp)       | 00010111110000111111111111111110  |
| (LOADIMM, \$t56, 1, -)       | addi \$t4, \$zero, 1    | 00000100000001000000000000000001  |
| (ADD, \$t57, \$t55, \$t56)   | add \$t5, \$t3, \$t4    | 00000000011001000010100000000000  |
| (STOREVAR, i, \$t57, -)      | sw \$t5, -2(\$fp)       | 00011011110001011111111111111110  |
| (JUMP, L8, -, -)             | j L8                    | 010000000000000000000000010000100 |
| (LABEL, L9, -, -)            | L9:                     |                                   |
| (ENDFUNC, main, -, -)        | halt                    | 01001100000000000000000000000000  |

Fonte: O autor.

## 6 Conclusão

O compilador projetado e desenvolvido neste trabalho demonstrou, por meio dos exemplos testados, capacidade de traduzir corretamente programas escritos em C— até o código binário executável pelo processador alvo. A geração de código se alinhou ao conjunto de instruções definido para o processador MIPS 32, produzindo sequências válidas para todas as construções da linguagem contempladas. Um destaque relevante da implementação é sua modularidade, em que, ao se basear na estrutura proposta por Louden e Silva (2004), cada fase do compilador foi organizada em módulos bem delimitados e independentes, o que facilitou o desenvolvimento incremental e a identificação de erros ao longo do projeto.

Entre as dificuldades encontradas, a primeira foi a decisão de reescrever o compilador desenvolvido na disciplina de Compiladores, com o objetivo de manter maior fidelidade à bibliografia de Louden e Silva (2004) e obter uma base mais sólida para a fase de síntese. As demais dificuldades concentraram-se na geração de código *assembly*. O projeto do gerenciamento de memória exigiu decisões planejadas sobre como organizar o quadro de ativação e definir os deslocamentos de cada símbolo em relação ao ponteiro de quadro. O mecanismo de salvamento de registradores em uso na troca de contexto também representou um desafio, pois era necessário preservar apenas os registradores efetivamente em uso no momento de cada chamada, evitando o salvamento desnecessário de todos os registradores. Por fim, o tratamento de vetores locais demandou a introdução de um campo adicional no quadro de ativação para armazenar o ponteiro de base do vetor, permitindo o acesso correto por índice durante a execução.

Como trabalhos futuros, destaca-se a possibilidade de aplicar otimizações ao compilador, reduzindo o número de instruções geradas em determinadas situações, como a eliminação de carregamentos redundantes de uma mesma variável em sequências próximas. Além disso, a próxima etapa prevista para a disciplina de Laboratório de Sistemas Computacionais é o projeto de um Sistema Operacional que fará uso direto do compilador C— desenvolvido neste trabalho, representando uma aplicação concreta e integrada de todo o sistema computacional construído ao longo dos laboratórios.

# Referências

LOUDEN, K. C.; SILVA, F. S. *Compiladores: princípios e práticas*. 1. ed. São Paulo: Thomson, 2004. Citado 6 vezes nas páginas [14](#), [15](#), [16](#), [17](#), [26](#) e [68](#).

PATTERSON, D. A.; HENNESY, J. L. *Organização e projeto de computadores*. 3. ed. Rio de Janeiro: Campus, 2005. Citado 3 vezes nas páginas [8](#), [10](#) e [12](#).

## Apêndices

# APÊNDICE A – globals.h

Código-fonte do *header* de definições globais (globals.h)

```

1 #ifndef _GLOBALS_H_
2 #define _GLOBALS_H_
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <ctype.h>
7 #include <string.h>
8
9 #define MAXCHILDREN 3
10
11 typedef int TokenType;
12 extern int lineNo;
13 extern FILE* source;
14
15 // LOUDEN (2004, p.505)
16 typedef enum {Void, Integer} ExpType;
17 typedef enum {StmtK, ExpK} NodeKind;
18 typedef enum {IfK, WhileK, CompoundK, FunctDeclK, FunctBodyK, ReturnK} StmtKind;
19 typedef enum {OpK, ConstK, VarK, ArrK, VarDeclK, ArrDeclK, ParamK, ParamArrK, AssignK,
 CallK} ExpKind;
20 typedef struct treeNode{
21 struct treeNode *child[MAXCHILDREN];
22 struct treeNode *sibling;
23 int lineNo;
24 NodeKind nodeKind;
25 union {StmtKind stmt; ExpKind exp;} kind;
26 union {TokenType op; int val; char *name;} attr;
27 ExpType type;
28 char *scope;
29 struct SymbolNode *symb;
30 } TreeNode;
31
32 #endif

```

# APÊNDICE B – scanner.l

Código-fonte do analisador léxico (scanner.l)

```

1 %option noyywrap
2 %option yylineno
3
4 %{
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <string.h>
8 #include "globals.h"
9 #include "util.h"
10 #include "parser.tab.h"
11
12 extern int lineNo;
13 %}
14
15 /* Definicoes*/
16 DIGITO [0-9]
17 LETRA [a-zA-Z]
18 ID {LETRA}+
19 NUM {DIGITO}+
20 ESPACO [\t\r]+
21
22 /* Estado para Comentarios Multilinha*/
23 %x COMENTARIO
24
25 %%
26
27 /* Palavras-Reservadas*/
28 "if" { return IF; }
29 "else" { return ELSE; }
30 "int" { return INT; }
31 "return" { return RETURN; }
32 "void" { return VOID; }
33 "while" { return WHILE; }
34
35 /*Operadores e Simbolos*/
36 "+" { return SOMA; }
37 "-" { return SUB; }
38 "*" { return MUL; }
39 "/" { return DIV; }
40 "<=" { return MENOR_IGUAL; }
41 "<" { return MENOR; }
42 ">=" { return MAIOR_IGUAL; }
43 ">" { return MAIOR; }
44 "==" { return IGUAL_IGUAL; }
45 "!=" { return DIFERENTE; }
46 "=" { return ATRIBUICAO; }
47 ";" { return PONTO_VIRGULA; }
48 "," { return VIRGULA; }
49 "(" { return ABRE_PARENTESE; }
50 ")" { return FECHA_PARENTESE; }
51 "[" { return ABRE_COLCHETE; }
52 "]" { return FECHA_COLCHETE; }

```

```
53 {" " { return ABRE_CHAVE; }
54 "}" { return FECHA_CHAVE; }
55
56 /*Identificadores e Numeros*/
57 {ID} {
58 yyval.name = copyString(yytext);
59 return ID;
60 }
61 {NUM} {
62 yyval.val = atoi(yytext);
63 return NUM;
64 }
65
66 /*Gerenciamento de Espacos e Linhas*/
67 \n { lineNo++; }
68 {ESPACO} { /* ignora */ }
69
70 /*Tratamento de Comentarios*/
71 "/*" { BEGIN(COMENTARIO); }
72 <COMENTARIO>"*/" { BEGIN(INITIAL); }
73 <COMENTARIO>\n { lineNo++; }
74 <COMENTARIO>. { /* ignora conteudo */ }
75
76 /*Erros Lexicos*/
77 . {
78 fprintf(stderr, "ERRO LEXICO: Caractere '%s' invalido - LINHA:%d\n", yytext, lineNo);
79 }
80
81 %%
```

# APÊNDICE C – parser.y

Código-fonte do analisador sintático (parser.y)

```

1 %{
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include "globals.h"
6 #include "util.h"
7
8 extern FILE *yyin;
9 extern int lineNo;
10 extern int yylex(void); // Chamada do scanner
11 extern char *yytext;
12
13 void yyerror(const char *s);
14
15 TreeNode *AST = NULL;
16 %}
17
18 %union {
19 TreeNode *tree;
20 int val;
21 char *name;
22 ExpType type;
23 }
24
25 //Tokens Valorados
26 %token<val> NUM
27 %token<name> ID
28
29 //Palavras reservadas
30 %token IF ELSE INT RETURN VOID WHILE
31
32 //Simbolos Especiais
33 %token SOMA SUB MUL DIV
34 %token MENOR MENOR_IGUAL MAIOR MAIOR_IGUAL IGUAL_IGUAL DIFERENTE
35 %token ATRIBUICAO PONTO_VIRGULA VIRGULA
36 %token ABRE_PARENTESE FECHA_PARENTESE
37 %token ABRE_COLCHETE FECHA_COLCHETE
38 %token ABRE_CHAVE FECHA_CHAVE
39
40 %token ERROR
41
42 // Declaracao de tipos para nao-terminais (tipagem de retorno $$)
43 %type <val> relacional soma mult
44 %type <type> tipo_especificador
45 %type <tree> programa declaracao_lista declaracao var_declaracao
46 %type <tree> fun_declaracao params param_lista param
47 %type <tree> composto_decl local_declaracoes statement_lista statement
48 %type <tree> expressao_decl selecao_decl iteracao_decl retorno_decl
49 %type <tree> expressao var simples_expressao soma_expressao termo fator
50 %type <tree> ativacao args arg_lista
51
52 %right ELSE // Solucao Dangling Else

```

```

53
54 %%
55
56 /* Gramatica (LOUDEN, 2004, p.494)
57 1. programa -> declaracao-lista
58 2. declaracao-lista -> declaracao-lista declaracao | declaracao
59 3. declaracao -> var-declaracao | fun-declaracao
60 4. var-declaracao -> tipo-especificador ID ; | tipo-especificador ID [NUM] ;
61 5. tipo-especificador -> int | void
62 6. fun-declaracao -> tipo-especificador ID(params)composto-decl
63 7. params -> param-lista | void
64 8. param-lista -> param-lista,param | param
65 9. param -> tipo-especificador ID | tipo-especificador ID []
66 10. composto-decl -> { local-declaracoes statement-lista }
67 11. local-declaracoes -> local-declaracoes var-declaracao | vazio
68 12. statement-lista -> statement-lista statement | vazio
69 13. statement -> expressao-decl | composto-decl | selecao-decl | iteracao-decl | retorno
 -decl
70 14. expressao-decl -> expressao ; | ;
71 15. selecao-decl -> if (expressao) statement | if (expressao) statement else
 statement
72 16. iteracao-decl -> while (expressao) statement
73 17. retorno-decl -> return ; | return expressao ;
74 18. expressao -> var = expressao | simples-expressao
75 19. var -> ID | ID [expressao]
76 20. simples-expressao -> soma-expressao relacional soma-expressao | soma-expressao
77 21. relacional -> <= | < | > | >= | == | !=
78 22. soma-expressao -> soma-expressao soma termo | termo
79 23. soma -> + | -
80 24. termo -> termo mult fator | fator
81 25. mult -> * | /
82 26. fator -> (expressao) | var | ativacao | NUM
83 27. ativacao -> ID (args)
84 28. args -> arg-lista | vazio
85 29. arg-lista -> arg-lista,expressao | expressao
86 */
87
88 //1. programa -> declaracao-lista
89 programa:
90 declaracao_lista {
91 AST = $1; //raiz da AST
92 }
93 ;
94
95 //2. declaracao-lista -> declaracao-lista declaracao | declaracao
96 declaracao_lista:
97 declaracao_lista declaracao {
98 TreeNode *t = $1;
99 // Lista encadeada de declaracoes no escopo
100 if (t != NULL){
101 while (t->sibling != NULL)
102 t = t->sibling;
103 t->sibling = $2;
104 $$ = $1;
105 }
106 else {
107 // Caso for uma primeira declaracao, sera um novo no com o item de '
108 declaracao'
109 $$ = $2;
110 }
111 }

```

```

110 }
111 | declaracao {
112 // Caso for uma primeira declaracao, sera um novo no com o item de '
113 // declaracao'
114 $$ = $1;
115 }
116 ;
117 //3. declaracao -> var-declaracao | fun-declaracao
118 declaracao:
119 var_declaracao {$$ = $1;}
120 | fun_declaracao {$$ = $1;}
121 ;
122 //4. var-declaracao -> tipo-especificador ID ; | tipo-especificador ID [NUM] ;
123 var_declaracao:
124 tipo_especificador ID PONTO_VIRGULA {
125 //Criacao do no da variavel
126 $$ = newExpNode(VarDeclK);
127 $$->type = $1;
128 $$->attr.name = copyString($2);
129 }
130 | tipo_especificador ID ABRE_COLCHETE NUM FECHA_COLCHETE PONTO_VIRGULA {
131 //Criacao do no do vetor
132 $$ = newExpNode(ArrDeclK);
133 $$->type = $1;
134 $$->attr.name = copyString($2);
135 //Tamanho do vetor salvo como filho da variavel
136 $$->child[0] = newExpNode(ConstK);
137 $$->child[0]->attr.val = $4;
138 }
139 ;
140 //5. tipo-especificador -> int | void
141 tipo_especificador:
142 INT {$$ = Integer;}
143 | VOID {$$ = Void;}
144 ;
145 //6. fun-declaracao -> tipo-especificador ID (params) composto-decl
146 fun_declaracao:
147 tipo_especificador ID {
148 $<val>$ = lineNo;
149 }
150 ABRE_PARENTESE params FECHA_PARENTESE composto_decl {
151 $$ = newStmtNode(FunctDeclK);
152 $$->lineNo = $<val>3; //resgata o numero da linha salvo na acao intermediaria
153 // (posicao 3 da regra)
154 $$->type = $1;
155 $$->attr.name = copyString($2);
156 $$->child[0] = $5; // params
157 $$->child[1] = $7; // composto_decl
158 //o compound da funcao e determinado como corpo da funcao (uso na analise
159 // semantica)
160 if ($$->child[1] != NULL) {
161 $$->child[1]->kind.stmt = FunctBodyK;
162 }
163 }
164 ;
165 //7. params -> param-lista | void
166 params:
167 param_lista {$$ = $1;}

```

```

167 | VOID {$$ = NULL;} //Funcao sem parametros (i.e: int main(void))
168 ;
169 //8. param-lista -> param-lista , param | param
170 param_lista:
171 param_lista VIRGULA param {
172 TreeNode *t = $1;
173 if (t != NULL){
174 while (t->sibling != NULL)
175 t = t->sibling;
176 t->sibling = $3;
177 $$ = $1;
178 } else {
179 $$ = $3;
180 }
181 }
182 | param {$$ = $1;}
183 ;
184 //9. param -> tipo-especificador ID | tipo-especificador ID []
185 param:
186 tipo_especificador ID {
187 $$ = newExpNode(ParamK);
188 $$->type = $1;
189 $$->attr.name = copyString($2);
190 }
191 | tipo_especificador ID ABRE_COLCHETE FECHA_COLCHETE {
192 $$ = newExpNode(ParamArrK);
193 $$->type = $1;
194 $$->attr.name = copyString($2);
195 }
196 ;
197
198 //10. composto-decl -> { local-declaracoes statement-lista }
199 // Bloco de codigo do escopo
200 composto_decl:
201 ABRE_CHAVE local_declaracoes statement_lista FECHA_CHAVE {
202 // No que define o escopo
203 $$ = newStmtNode(CompoundK);
204 $$->child[0] = $2; //Esq: lista de declaracoes
205 $$->child[1] = $3; //Dir: lista de instrucoes (statements)
206 }
207 ;
208
209 //11. local-declaracoes -> local-declaracoes var-declaracao | vazio
210 local_declaracoes:
211 local_declaracoes var_declaracao {
212 TreeNode *t = $1;
213 if (t!=NULL){
214 while (t->sibling != NULL)
215 t = t->sibling;
216 t->sibling = $2;
217 $$ = $1;
218 } else {
219 $$ = $2;
220 }
221 }
222 | /* Vazio */ {$$ = NULL;}
223 ;
224
225 //12. statement-lista -> statement-lista statement | vazio
226 statement_lista:

```

```

227 statement_lista statement {
228 TreeNode *t = $1;
229 if (t != NULL){
230 while(t->sibling != NULL)
231 t = t->sibling;
232 t->sibling = $2;
233 $$ = $1;
234 } else {
235 $$ = $2;
236 }
237 }
238 | /* vazio */ { $$ = NULL; }
239 ;
240
241 //13. statement -> expressao-decl | composto-decl | selecao-decl | iteracao-decl |
 retorno-decl
242 statement:
243 expressao_decl { $$ = $1; }
244 | composto_decl { $$ = $1; }
245 | selecao_decl { $$ = $1; }
246 | iteracao_decl { $$ = $1; }
247 | retorno_decl { $$ = $1; }
248 ;
249
250 //14. expressao-decl -> expressao ; | ;
251 expressao_decl:
252 expressao PONTO_VIRGULA { $$ = $1; } //A subarvore foi criada em 'expressao'
253 | PONTO_VIRGULA { $$ = NULL; }
254 ;
255
256 //15. selecao-decl -> if (expressao) statement | if (expressao) statement else
 statement
257 selecao_decl:
258 IF ABRE_PARENTESE expressao FECHA_PARENTESE statement {
259 $$ = newStmtNode(IfK); //Pai: IF
260 $$->child[0] = $3; //Filho 1: expressao
261 $$->child[1] = $5; //Filho 2: instrucoes
262 }
263 | IF ABRE_PARENTESE expressao FECHA_PARENTESE statement ELSE statement {
264 $$ = newStmtNode(IfK); //Pai: IF
265 $$->child[0] = $3; //Filho 1: expressao condicional
266 $$->child[1] = $5; //Filho 2: instrucoes then
267 $$->child[2] = $7; //Filho 3: instrucoes else
268 }
269 ;
270
271 //16. iteracao-decl -> while (expressao) statement
272 iteracao_decl:
273 WHILE ABRE_PARENTESE expressao FECHA_PARENTESE statement {
274 $$ = newStmtNode(WhileK);
275 $$->child[0] = $3; //Filho 1: expressao condicional
276 $$->child[1] = $5; //Filho 2: instrucoes
277 }
278 ;
279
280 //17. retorno-decl -> return ; | return expressao ;
281 retorno_decl:
282 RETURN PONTO_VIRGULA {
283 $$ = newStmtNode(ReturnK);
284 // Return; => Filhos nulos

```

```

285 }
286 | RETURN expressao PONTO_VIRGULA {
287 $$ = newStmtNode(ReturnK);
288 $$->child[0] = $2;
289 }
290 ;
291
292 //18. expressao -> var = expressao | simples-expressao
293 expressao:
294 var ATRIBUICAO expressao {
295 $$ = newExpNode(AssignK);
296 $$->child[0] = $1;
297 $$->child[1] = $3;
298 }
299 | simples_expressao {$$ = $1;}
300 ;
301
302 //19. var -> ID | ID [expressao]
303 var:
304 ID {
305 //Uso de variavel ja declarada
306 $$ = newExpNode(VarK);
307 $$->attr.name = copyString($1);
308 }
309 | ID ABRE_COLCHETE expressao FECHA_COLCHETE {
310 $$ = newExpNode(ArrK);
311 $$->attr.name = copyString($1);
312 $$->child[0] = $3; //indice dado pela expressao
313 }
314 }
315 ;
316
317 //20. simples-expressao -> soma-expressao relacional soma-expressao | soma-expressao
318 simples_expressao:
319 soma_expressao relacional soma_expressao {
320 //Operacao logica/relacional (i.e: a < b, x==5)
321 $$ = newExpNode(OpK);
322 $$->attr.op = $2;
323 $$->child[0] = $1;
324 $$->child[1] = $3;
325 }
326 | soma_expressao {$$ = $1;}
327 ;
328
329 //21. relacional -> <= | < | > | >= | == | !=
330 relacional:
331 MENOR_IGUAL {$$ = MENOR_IGUAL;}
332 | MENOR {$$ = MENOR;}
333 | MAIOR {$$ = MAIOR;}
334 | MAIOR_IGUAL {$$ = MAIOR_IGUAL;}
335 | IGUAL_IGUAL {$$ = IGUAL_IGUAL;}
336 | DIFERENTE {$$ = DIFERENTE;}
337 ;
338
339 //22. soma-expressao -> soma-expressao soma termo | termo
340 soma_expressao:
341 soma_expressao soma termo {
342 //Operacao de adicao/subtracao
343 $$ = newExpNode(OpK); //soma -> + | -
344 $$->attr.op = $2;

```

```

345 // TODO: verificar precedencia
346 $$->child[0] = $1;
347 $$->child[1] = $3;
348 }
349 | termo {$$ = $1;}
350 ;
351
352 //23. soma -> + | -
353 soma:
354 SOMA {$$ = SOMA;}
355 | SUB {$$ = SUB;}
356 ;
357
358 //24. termo -> termo mult fator | fator
359 termo:
360 termo mult fator {
361 $$ = newExpNode(OpK);
362 $$->attr.op = $2;
363 $$->child[0] = $1;
364 $$->child[1] = $3;
365 }
366 | fator {$$ = $1;}
367 ;
368
369 //25. mult -> * | /
370 mult:
371 MUL {$$ = MUL;}
372 | DIV {$$ = DIV;}
373 ;
374
375 //26. fator -> (expressao) | var | ativacao | NUM
376 fator:
377 ABRE_PARENTESE expressao FECHA_PARENTESE {
378 $$ = $2; //Parenteses forca o Bison a priorizar a conta
379 }
380 | var {$$ = $1;}
381 | ativacao {$$ = $1;}
382 | NUM {
383 $$ = newExpNode(ConstK);
384 $$->attr.val = $1;
385 }
386 ;
387
388 //27. ativacao -> ID (args)
389 ativacao:
390 ID ABRE_PARENTESE args FECHA_PARENTESE {
391 $$ = newExpNode(CallK);
392 $$->attr.name = copyString($1);
393 $$->child[0] = $3;
394 }
395 ;
396
397 //28. args -> arg-lista | vazio
398 args:
399 arg_lista {$$ = $1;}
400 | /* vazio */ {$$ = NULL;}
401 ;
402
403 //29. arg-lista -> arg-lista , expressao | expressao
404 arg_lista:

```

```
405 arg_lista VIRGULA expressao {
406 //Lista de argumentos para ativacao
407 TreeNode *t = $1;
408 if (t!=NULL){
409 while(t->sibling != NULL)
410 t = t->sibling;
411 t->sibling = $3;
412 $$ = $1;
413 } else {
414 $$ = $3;
415 }
416 }
417 | expressao {$$ = $1;}
418 ;
419
420 %%
421
422 void yyerror(const char *s) {
423 fprintf(stderr, "ERRO SINTATICO: token inesperado '%s' - LINHA: %d\n", yytext,
424 lineNo);
425 }
```

## APÊNDICE D – analyze.h

Código-fonte do *header* da análise semântica (`analyze.h`)

```
1 #ifndef _ANALYZE_H_
2 #define _ANALYZE_H_
3
4 #include "globals.h"
5
6 void buildSyntab(TreeNode *AST);
7 void typeCheck(TreeNode *AST);
8
9 #endif
```

# APÊNDICE E – analyze.c

Código-fonte da análise semântica (analyze.c)

```

1 #include "globals.h"
2 #include "syntab.h"
3 #include "util.h"
4
5 static int location = 0;
6 static int blockCounter = 0;
7 static int hasMain = 0;
8
9 static ExpType currentFuncType = Void;
10 static int hasReturn = 0;
11 static char* currentFuncName = NULL;
12 static Symbol currentFuncSymbol = NULL;
13
14 //Primeiro procedimento: Tratamento de declaracoes e escopos (PRE-ORDEM)
15 //Erros Semanticos: Valida main (falta e ordem), duplicatas (vars/funcs/params), void em
 variaveis e uso sem declaracao;
16 static void insertNode(TreeNode *t) {
17 char newScopeName[256]; //Buffer para nomes de escopo de blocos
18 if(currentScope != NULL) {
19 t->scope = copyString(currentScope->name);
20 }
21
22 if(hasMain && (currentScope == globalScope)) {
23 //apos a declaracao da main, nao e permitido haver novas declaracoes. (ausencia
 de prototipos na linguagem C-)
24 if((t->nodeKind == StmtK && t->kind.stmt == FunctDeclK) || (t->nodeKind == ExpK
 && (t->kind.exp == VarDeclK || t->kind.exp == ArrDeclK))) {
25 fprintf(stderr, "ERRO SEMANTICO: Declaracao de '%s' invalida. A funcao 'main'
 deve ser a ultima declaracao do arquivo - LINHA: %d\n", t->attr.name, t
 ->lineNo);
26 }
27 }
28
29 switch (t->nodeKind) {
30 case StmtK:
31 switch (t->kind.stmt) {
32 case FunctDeclK: {
33 if(st_lookup_scope(t->attr.name) != NULL){
34 fprintf(stderr, "ERRO SEMANTICO: Funcao '%s' ja declarada - LINHA
 : %d\n", t->attr.name, t->lineNo);
35 currentFuncSymbol = st_lookup_scope(t->attr.name); //fallback
36 t->symb = currentFuncSymbol;
37 } else {
38 currentFuncSymbol = st_insert(t->attr.name, t->type, SYMB_FUNC, t
 ->lineNo, -1);
39 t->symb = currentFuncSymbol;
40 if(strcmp(t->attr.name, "main") == 0)
41 hasMain = 1;
42 }
43 pushScope(t->attr.name);
44 blockCounter = 0;
45 location = 0;

```

```

46 break;
47 }
48
49 case CompoundK: {
50 blockCounter++;
51 sprintf(newScopeName, "%s:block%d", currentScope->name, blockCounter)
52 ;
53 pushScope(newScopeName);
54 break;
55 }
56 default: break;
57 }
58 break;
59
60 case ExpK:
61 switch (t->kind.exp) {
62 case VarDeclK: {
63 if(t->type == Void) {
64 printf("ERRO SEMANTICO: Variavel '%s' nao pode ser do tipo 'void'
65 '\n - LINHA: %d\n", t->attr.name, t->lineNo);
66 } else if(st_lookup_scope(t->attr.name) != NULL) {
67 printf("ERRO SEMANTICO: Variavel '%s' ja declarada neste escopo.
68 - LINHA: %d\n", t->attr.name, t->lineNo);
69 } else {
70 Symbol s_node = st_insert(t->attr.name, t->type, SYMB_VAR, t->
71 lineNo, location++);
72 t->symb = s_node;
73 }
74 }
75 case ArrDeclK: {
76 if(t->type == Void) {
77 printf("ERRO SEMANTICO: Variavel '%s' nao pode ser do tipo 'void'
78 '\n - LINHA: %d\n", t->attr.name, t->lineNo);
79 } else if(st_lookup_scope(t->attr.name) != NULL) {
80 printf("ERRO SEMANTICO: Variavel '%s' ja declarada neste escopo.
81 - LINHA: %d\n", t->attr.name, t->lineNo);
82 } else {
83 Symbol s_node = st_insert(t->attr.name, t->type, SYMB_ARR, t->
84 lineNo, location);
85 t->symb = s_node;
86 location += t->child[0]->attr.val; //memloc sera deslocado de
87 acordo com o tamanho do vetor
88 }
89 }
90 case ParamK: {
91 if(t->type == Void) {
92 // t->attr.name == NULL e o caso do void isolado, como em int
93 main(void)
94 if(t->attr.name != NULL) {
95 fprintf(stderr, "ERRO SEMANTICO: Parametro '%s' nao pode ser
96 do tipo 'void' - LINHA: %d\n", t->attr.name, t->lineNo);
97 }
98 //fallback
99 Symbol s_node = st_insert(t->attr.name, Integer, SYMB_VAR, t
100 ->lineNo, location++);
101 t->symb = s_node;
102 st_add_param(currentFuncSymbol, Integer);
103 }
104 }
105 }

```

```

95 }
96 else if(st_lookup_scope(t->attr.name) != NULL) {
97 fprintf(stderr, "ERRO SEMANTICO: Parametro '%s' ja declarado
98 nesta funcao - LINHA: %d\n", t->attr.name, t->lineNo);
99 } else {
100 Symbol s_node = st_insert(t->attr.name, t->type, SYMB_VAR, t->
101 lineNo, location++);
102 t->symb = s_node;
103 st_add_param(currentFuncSymbol, t->type);
104 }
105 }
106 break;
107 }
108 case ParamArrK: {
109 if(t->type == Void) {
110 fprintf(stderr, "ERRO SEMANTICO: Parametro de vetor '%s' nao pode
111 ser do tipo 'void' - LINHA: %d\n", t->attr.name, t->lineNo);
112 //fallback
113 Symbol s_node = st_insert(t->attr.name, Integer, SYMB_ARR, t->
114 lineNo, location++);
115 t->symb = s_node;
116 st_add_param(currentFuncSymbol, Integer);
117 } else if(st_lookup_scope(t->attr.name) != NULL) {
118 fprintf(stderr, "ERRO SEMANTICO: Parametro '%s' ja declarado
119 nesta funcao - LINHA: %d\n", t->attr.name, t->lineNo);
120 } else {
121 Symbol s_node = st_insert(t->attr.name, t->type, SYMB_ARR, t->
122 lineNo, location++);
123 t->symb = s_node;
124 st_add_param(currentFuncSymbol, t->type);
125 }
126 break;
127 }
128 case VarK: {
129 Symbol s_node = st_lookup(t->attr.name);
130 if(s_node == NULL) {
131 fprintf(stderr, "ERRO SEMANTICO: Variavel '%s' nao declarada -
132 LINHA: %d\n", t->attr.name, t->lineNo);
133 t->type = Integer; // fallback
134 } else {
135 t->type = s_node->type;
136 st_add_line(s_node, t->lineNo);
137 t->symb = s_node;
138 }
139 break;
140 }
141 case ArrK: {
142 Symbol s_node = st_lookup(t->attr.name);
143 if(s_node == NULL) {
144 fprintf(stderr, "ERRO SEMANTICO: Variavel '%s' nao declarada -
145 LINHA: %d\n", t->attr.name, t->lineNo);
146 } else {
147 if(s_node->kind != SYMB_ARR)
148 fprintf(stderr, "ERRO SEMANTICO: Variavel '%s' nao e um vetor
149 - LINHA: %d\n", t->attr.name, t->lineNo);
150 t->type = s_node->type;
151 st_add_line(s_node, t->lineNo);
152 t->symb = s_node;
153 }
154 }

```

```

146 break;
147 }
148 case CallK: {
149 Symbol s_node = st_lookup_global(t->attr.name);
150
151 if(s_node == NULL) {
152 fprintf(stderr, "ERRO SEMANTICO: Funcao '%s' nao declarada - LINHA
153 : %d\n", t->attr.name, t->lineNo);
154 t->type = Integer; //fallback
155 } else {
156 if(s_node->kind != SYMB_FUNC)
157 fprintf(stderr, "ERRO SEMANTICO: '%s' nao e uma funcao - LINHA
158 : %d\n", t->attr.name, t->lineNo);
159
160 t->type = s_node->type;
161 t->symb = s_node;
162 st_add_line(s_node, t->lineNo);
163 }
164 break;
165 }
166 default: break;
167 }
168
169 default: break;
170 }
171 }
172
173 //topo da pilha atualizado para esconder escopos que nao podem ser acessados
174 static void leaveScope(TreeNode *t) {
175 if(t->nodeKind == StmtK) {
176 if(t->kind.stmt == FunctDeclK || t->kind.stmt == CompoundK)
177 popScope();
178 }
179 }
180
181 //percurso pre-ordem (Pai -> Filhos -> Irmão)
182 static void buildSyntabRec(TreeNode *t) {
183 if(t != NULL) {
184 insertNode(t);
185
186 for(int i = 0; i < MAXCHILDREN; i++) {
187 buildSyntabRec(t->child[i]);
188 }
189 leaveScope(t);
190 buildSyntabRec(t->sibling); // Vai para o irmão
191 }
192 }
193
194 void buildSyntab(TreeNode *AST) {
195 globalScope = pushScope("global");
196
197 st_insert("input", Integer, SYMB_FUNC, 0, -1);
198 Symbol out_node = st_insert("output", Void, SYMB_FUNC, 0, -1);
199 st_add_param(out_node, Integer);
200
201 buildSyntabRec(AST);
202
203 if(hasMain == 0)

```



```

256 }
257 default: break;
258 }
259 break;
260
261 case StmtK:
262 switch (t->kind.stmt) {
263 case IfK: {
264 if(t->child[0]->type != Integer)
265 fprintf(stderr, "ERRO SEMANTICO: A condicao do 'if' deve ser do
266 tipo 'int' - LINHA: %d\n", t->lineNo);
267 break;
268 }
269 case WhileK: {
270 if(t->child[0]->type != Integer)
271 fprintf(stderr, "ERRO SEMANTICO: A condicao do 'while' deve ser do
272 tipo 'int' - LINHA: %d\n", t->lineNo);
273 break;
274 }
275 case ReturnK: {
276 if(currentFuncType == Void) {
277 if(t->child[0] != NULL) {
278 fprintf(stderr, "ERRO SEMANTICO: Funcao 'void' nao deve
279 retornar um valor - LINHA: %d\n", t->lineNo);
280 }
281 } else {
282 //Louden (p.495, 2004) funcoes int devem retornar valores
283 hasReturn = 1; // registra-se que a funcao tem pelo menos um
284 retorno
285
286 if(t->child[0] == NULL) {
287 fprintf(stderr, "ERRO SEMANTICO: Retorno vazio em funcao do
288 tipo 'int' - LINHA: %d\n", t->lineNo);
289 } else if(t->child[0]->type != Integer) {
290 fprintf(stderr, "ERRO SEMANTICO: O valor retornado deve ser
291 do tipo 'int' - LINHA: %d\n", t->lineNo);
292 }
293 }
294 break;
295 }
296 default: break;
297 }
298 }
299 }
300
301 //percurso Pos-ordem (Filhos -> Pai -> Irmão)
302 static void typeCheckRec(TreeNode *t) {
303 if(t != NULL) {
304 // ao encontrar uma declaracao de funcao, guarda o contexto
305 if(t->nodeKind == StmtK && t->kind.stmt == FunctDeclK) {
306 currentFuncType = t->type;
307 currentFuncName = t->attr.name;
308 hasReturn = 0; //reiniciar total de returns
309 }

```

```
310
311 for(int i = 0; i < MAXCHILDREN; i++) {
312 typeCheckRec(t->child[i]);
313 }
314
315 checkNode(t);
316
317 //apos processar todos os filhos, verifica se uma funcao int cumpriu o requisito
 de return
318 if(t->nodeKind == StmtK && t->kind.stmt == FunctDeclK) {
319 if(currentFuncType == Integer && hasReturn == 0) {
320 fprintf(stderr, "ERRO SEMANTICO: A funcao '%s' eh do tipo 'int' e deve
 possuir um retorno - LINHA: %d\n", t->attr.name, t->lineNo);
321 }
322 }
323
324 typeCheckRec(t->sibling);
325 }
326 }
327
328 void typeCheck(TreeNode *AST) {
329 typeCheckRec(AST);
330 }
```

# APÊNDICE F – symtab.h

Código-fonte do *header* da tabela de símbolos (symtab.h)

```

1 #ifndef _SYMTAB_H_
2 #define _SYMTAB_H_
3 #include "globals.h"
4
5 typedef enum {SYMB_VAR, SYMB_ARR, SYMB_FUNC} SymbolKind;
6 typedef struct LineListNode *LineList;
7
8 typedef struct ParamListNode {
9 ExpType type;
10 struct ParamListNode *next;
11 } *ParamList;
12
13 typedef struct SymbolNode {
14 char *name;
15 char *scope;
16 ExpType type;
17 SymbolKind kind;
18 int memloc;
19
20 //atributos de funcao
21 int numParams;
22 ParamList params;
23
24 LineList lines;
25 struct SymbolNode *next;
26 } *Symbol;
27
28 typedef struct ScopeNode {
29 char *name;
30 Symbol h_symbols;
31 struct ScopeNode *parent;
32 struct ScopeNode *next;
33 } *Scope;
34
35 extern Scope currentScope;
36 extern Scope globalScope;
37
38 Scope pushScope(char *name);
39 void popScope();
40 Symbol st_insert(char *name, ExpType type, SymbolKind kind, int lineNo, int loc);
41 void st_add_line(Symbol s_node, int lineNo);
42 void st_add_param(Symbol func_node, ExpType paramType);
43 Symbol st_lookup(char *name);
44 Symbol st_lookup_scope(char *name);
45 Symbol st_lookup_global(char *name);
46
47 void printSymTab(FILE *listing);
48
49 #endif

```

# APÊNDICE G – symtab.c

Código-fonte da tabela de símbolos (symtab.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "symtab.h"
5 #include "util.h"
6
7 Scope currentScope = NULL;
8 Scope globalScope = NULL;
9 Scope scopeHistory = NULL;
10
11 //Lista com numero da linha de todas as ocorrencias de uma variavel
12 typedef struct LineListNode{
13 int lineNo;
14 struct LineListNode *next; // listar todas as ocorrencias de uma mesma variavel
15 } *LineList;
16
17 //empilhar escopo e atualizar currentScope
18 Scope pushScope(char *name){
19 Scope newScope = (Scope)malloc(sizeof(struct ScopeNode));
20 newScope->name = copyString(name);
21 newScope->h_symbols = NULL;
22 newScope->parent = currentScope;
23 currentScope = newScope;
24
25 //push historico para impressao: insercao ao final da lista
26 newScope->next = NULL;
27 if(scopeHistory == NULL)
28 scopeHistory = newScope;
29 else {
30 //atualizar next do ultimo no
31 Scope temp = scopeHistory;
32 while (temp->next != NULL)
33 temp = temp->next;
34
35 temp->next = newScope;
36 }
37 return newScope;
38 }
39
40 //apontar currentScope para novo topo (pop)
41 void popScope(){
42 if(currentScope != NULL){
43 currentScope = currentScope->parent;
44 }
45 }
46
47 //insercao de novo simbolo no escopo atual
48 Symbol st_insert(char *name, ExpType type, SymbolKind kind, int lineNo, int loc){
49 Symbol newS = (Symbol)malloc(sizeof(struct SymbolNode));
50 newS->name = name;
51 newS->scope = copyString(currentScope->name);
52 newS->type = type;

```

```

53 newS->kind = kind;
54 newS->memloc = loc;
55 newS->lines = (LineList)malloc(sizeof(struct LineListNode));
56 newS->lines->lineNo = lineNo;
57 newS->lines->next = NULL;
58
59 newS->next = currentScope->h_symbols;
60 currentScope->h_symbols = newS;
61
62 return newS;
63 }
64
65 //adicionar linha de uso de um simbolo ja declarado
66 void st_add_line(Symbol s_node, int lineNo){
67 if(s_node != NULL) {
68 LineList line_node = s_node->lines;
69
70 while(line_node->next != NULL){
71 line_node = line_node->next;
72 }
73
74 if(line_node->lineNo != lineNo) {
75 line_node->next = (LineList)malloc(sizeof(struct LineListNode));
76 line_node->next->lineNo = lineNo;
77 line_node->next->next = NULL;
78 }
79 }
80 }
81
82 //adicionar parametros de declaracao de funcao na struct do simbolo para rastreabilidade
83 void st_add_param(Symbol func_node, ExpType paramType) {
84 if(func_node != NULL && func_node->kind == SYMB_FUNC){
85 ParamList newP = (ParamList)malloc(sizeof(struct ParamListNode));
86 newP->type = paramType;
87 newP->next = NULL;
88
89 if(func_node->params == NULL){
90 func_node->params = newP;
91 } else {
92 ParamList temp = func_node->params;
93 while (temp->next != NULL)
94 temp = temp->next;
95 temp->next = newP;
96 }
97 func_node->numParams++;
98 }
99 }
100
101 //procura simbolo do topo atual para baixo (segue hierarquia de escopo)
102 Symbol st_lookup(char *name){
103 Scope scp_node = currentScope;
104 while (scp_node != NULL){
105 Symbol s_node = scp_node->h_symbols;
106 while(s_node != NULL){
107 if(strcmp(name, s_node->name) == 0)
108 return s_node;
109 s_node = s_node->next;
110 }
111 scp_node = scp_node->parent;
112 }

```

```

113 return NULL;
114 }
115
116 //procura no escopo atual somente
117 Symbol st_lookup_scope(char *name){
118 if(currentScope == NULL)
119 return NULL;
120 Symbol s_node = currentScope->h_symbols;
121 while (s_node != NULL){
122 if(strcmp(name, s_node->name) == 0)
123 return s_node;
124 s_node = s_node->next;
125 }
126 return NULL;
127 }
128
129 Symbol st_lookup_global(char *name){
130 if(globalScope == NULL)
131 return NULL;
132 Symbol s_node = globalScope->h_symbols;
133 while(s_node != NULL){
134 if(strcmp(name, s_node->name) == 0)
135 return s_node;
136 s_node = s_node->next;
137 }
138 return NULL;
139 }
140
141 //impressao
142 static const char* typeToString(ExpType type){
143 switch(type) {
144 case Void: return "void";
145 case Integer: return "int";
146 default: return "unknown";
147 }
148 }
149
150 static const char* kindToString(SymbolKind kind){
151 switch(kind) {
152 case SYMB_VAR: return "Variavel";
153 case SYMB_ARR: return "Vetor";
154 case SYMB_FUNC: return "Funcao";
155 default: return "unknown";
156 }
157 }
158
159 void printSymTab(FILE * listing){
160 fprintf(listing, "Nome do Simbolo | Tipo | Classificacao | Escopo\n");
161 fprintf(listing, " | Loc | Linhas\n");
162 fprintf(listing, "-----\n");
163 Scope scp = scopeHistory;
164 while (scp != NULL) {
165 Symbol sym = scp->h_symbols;
166
167 //barre todas as variaveis/funcoes dentro do escopo
168 while (sym != NULL) {
169 fprintf(listing, "%-17s | ", sym->name);

```

```
170 fprintf(listing, "%-8s | ", typeToString(sym->type));
171 fprintf(listing, "%-13s | ", kindToString(sym->kind));
172 fprintf(listing, "%-14s | ", scp->name);
173 if(sym->memloc == -1){
174 fprintf(listing, "%-3s | ", "-");
175 } else {
176 fprintf(listing, "%-3d | ", sym->memloc);
177 }
178
179 //imprime todas as linhas onde o simbolo apareceu
180 LineList line = sym->lines;
181 while (line != NULL) {
182 fprintf(listing, "%d ", line->lineNo);
183 line = line->next;
184 }
185 fprintf(listing, "\n");
186
187 sym = sym->next;
188 }
189 scp = scp->next;
190 }
191 }
```

# APÊNDICE H – cgen.h

Código-fonte do *header* do gerador de código intermediário (cgen.h)

```

1 #ifndef _CGEN_H_
2 #define _CGEN_H_
3
4 #include "globals.h"
5 #include "parser.tab.h"
6 #include "syntab.h"
7
8 //Operacoes intermediarias
9 typedef enum {
10 OPND_EMPTY,
11 OPND_NUM,
12 OPND_TEMP,
13 OPND_SCOPE,
14 OPND_LABEL,
15 OPND_TYPE,
16 OPND_SYMB
17 } OpndKind;
18
19 typedef enum {
20 OP_ADD, OP_SUB, OP_MUL, OP_DIV,
21 OP_ASSIGN,
22 OP_JUMP,
23 OP_BEQ, OP_BNE, OP_BLT, OP_BGT, OP_BLE, OP_BGE,
24 OP_LABEL,
25 OP_FUNC, OP_ENDFUNC,
26 OP_ARG, OP_ARG_ARR, OP_PARAM,
27 OP_CALL,
28 OP_ALLOCVAR, OP_ALLOCARR,
29 OP_LOADVAR, OP_LOADARR, OP_LOADIMM,
30 OP_STOREVAR, OP_STOREARR,
31 OP_RETURN,
32 OP_IN, OP_OUT,
33 OP_HALT
34 } OpKind;
35
36 typedef struct {
37 OpndKind kind;
38 union {
39 int imm; //valor de registrador e imediato
40 int reg_id;
41 char *label_name;
42 char *type_name;
43 char *scope_name;
44 Symbol s_node;
45 } content;
46 } Operand;
47
48 typedef struct QuadList {
49 OpKind op;
50 Operand arg1;
51 Operand arg2;
52 Operand arg3;

```

```
53 struct QuadList* next;
54 } Quad;
55
56 extern Quad* headQuad;
57 extern Quad* currentQuad;
58
59 void printIntermediateCode(FILE *listing);
60 void generateIntermediateCode(TreeNode *AST);
61
62 #endif
```

# APÊNDICE I – cgen.c

Código-fonte do gerador de código intermediário (cgen.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "globals.h"
5 #include "syntab.h"
6 #include "util.h"
7 #include "cgen.h"
8
9 #define EMPTY_OPND emptyOperand()
10
11 Quad* headQuad = NULL;
12 Quad* currentQuad = NULL;
13
14 static int tempOffset = 0;
15 static int labelOffset = 0;
16
17 static void cGen(TreeNode *t);
18
19 static Operand newTemp(){
20 Operand tempOpnd;
21 tempOpnd.kind = OPND_TEMP;
22 tempOffset++;
23 tempOpnd.content.reg_id = tempOffset;
24 return tempOpnd;
25 }
26
27 static Operand newLabel(){
28 Operand labelOpnd;
29 labelOpnd.kind = OPND_LABEL;
30 char *labelName = (char*)malloc(10*sizeof(char));
31 labelOffset++;
32 sprintf(labelName, "L%d", labelOffset);
33 labelOpnd.content.label_name = labelName;
34 return labelOpnd;
35 }
36
37 static Operand emptyOperand(){
38 Operand emptyOpnd;
39 emptyOpnd.kind = OPND_EMPTY;
40 emptyOpnd.content.s_node = NULL;
41 return emptyOpnd;
42 }
43
44 static Operand symbOperand(Symbol symb){
45 Operand symbOpnd;
46 symbOpnd.kind = OPND_SYMB;
47 symbOpnd.content.s_node = symb;
48 return symbOpnd;
49 }
50
51 static Operand numOperand(int val){
52 Operand numOpnd;

```

```

53 numOpnd.kind = OPND_NUM;
54 numOpnd.content.imm = val;
55 return numOpnd;
56 }
57
58 static Operand scopeOperand(char *scope) {
59 Operand scopeOpnd;
60 scopeOpnd.kind = OPND_SCOPE;
61 scopeOpnd.content.scope_name = scope;
62 return scopeOpnd;
63 }
64
65 static Operand typeOperand(ExpType type){
66 Operand typeOpnd;
67 typeOpnd.kind = OPND_TYPE;
68 if(type == Integer)
69 typeOpnd.content.type_name = "int";
70 else if(type == Void)
71 typeOpnd.content.type_name = "void";
72 return typeOpnd;
73 }
74
75 OpKind opTokenToOpKind(TokenType op){
76 switch(op){
77 case SOMA: return OP_ADD;
78 case SUB: return OP_SUB;
79 case MUL: return OP_MUL;
80 case DIV: return OP_DIV;
81 default:
82 fprintf(stderr, "Erro GCI: Operador invalido encontrado");
83 exit(1);
84 }
85 }
86
87 static const char* opKindToString(OpKind op){
88 switch(op){
89 case OP_ADD: return "ADD";
90 case OP_SUB: return "SUB";
91 case OP_MUL: return "MUL";
92 case OP_DIV: return "DIV";
93 case OP_ASSIGN: return "ASSIGN";
94 case OP_JUMP: return "JUMP";
95 case OP_BEQ: return "BEQ";
96 case OP_BNE: return "BNE";
97 case OP_BGE: return "BGE";
98 case OP_BLE: return "BLE";
99 case OP_BGT: return "BGT";
100 case OP_BLT: return "BLT";
101 case OP_FUNC: return "FUNC";
102 case OP_ENDFUNC: return "ENDFUNC";
103 case OP_LABEL: return "LABEL";
104 case OP_PARAM: return "PARAM";
105 case OP_CALL: return "CALL";
106 case OP_ARG: return "ARG";
107 case OP_ARG_ARR: return "ARGARR";
108 case OP_ALLOCVAR: return "ALLOCVAR";
109 case OP_ALLOCARR: return "ALLOCARR";
110 case OP_LOADVAR: return "LOADVAR";
111 case OP_LOADARR: return "LOADARR";
112 case OP_LOADIMM: return "LOADIMM";

```

```

113 case OP_STOREVAR: return "STOREVAR";
114 case OP_STOREARR: return "STOREARR";
115 case OP_RETURN: return "RETURN";
116 case OP_IN: return "INPUT";
117 case OP_OUT: return "OUTPUT";
118 case OP_HALT: return "HALT";
119 default: return "UNKNOWN";
120 }
121 }
122
123 void emitQuad(OpKind op, Operand arg1, Operand arg2, Operand arg3){
124 Quad* newQ = (Quad*)malloc(sizeof(Quad));
125 newQ->op = op;
126 newQ->arg1 = arg1;
127 newQ->arg2 = arg2;
128 newQ->arg3 = arg3;
129 newQ->next = NULL;
130
131 if(headQuad == NULL){
132 headQuad = newQ;
133 currentQuad = newQ;
134 } else {
135 currentQuad->next = newQ;
136 currentQuad = newQ;
137 }
138 }
139
140 static Operand genExp(TreeNode *t){
141 if(t != NULL){
142 switch(t->kind.exp){
143 case ConstK:{
144 // (LOAD, $t_a, const, -)
145 Operand resultTemp = newTemp();
146 Operand argNum = numOperand(t->attr.val);
147 emitQuad(OP_LOADIMM, resultTemp, argNum, EMPTY_OPND);
148 return resultTemp;
149 }
150 case VarK:{
151 // (LOAD, $t_a, var, -)
152 Operand resultTemp = newTemp();
153 Operand argVar = symbOperand(t->symb);
154 emitQuad(OP_LOADVAR, resultTemp, argVar, EMPTY_OPND);
155 return resultTemp;
156 }
157 case ArrK:{
158 Operand argArr = symbOperand(t->symb);
159 Operand argIdx = genExp(t->child[0]); //tratamento de constantes,
160 //operacoes matematicas ou uso de variaveis
161 Operand resultTemp = newTemp();
162 // (LOAD, $t_a, var, index)
163 emitQuad(OP_LOADARR, resultTemp, argArr, argIdx);
164 return resultTemp;
165 }
166 case VarDeclK:{
167 // (ALLOCVAR, var, scope, -)
168 Operand resultVar = symbOperand(t->symb);
169 Operand argScope = scopeOperand(t->scope);
170 emitQuad(OP_ALLOCVAR, resultVar, argScope, EMPTY_OPND);
171 return emptyOperand();
172 }
173 }
174 }
175 }

```

```

172 case ArrDeclK:{
173 //(ALLOCARR, var, arr_size, scope)
174 Operand resultArr = symbOperand(t->symb);
175 Operand argNum = numOperand(t->child[0]->attr.val); //C- nao permite
 declaracao de array usando [variavel]
176 Operand argScope = scopeOperand(t->scope);
177 emitQuad(OP_ALLOCARR, resultArr, argNum, argScope);
178 return emptyOperand();
179 }
180 case OpK:{
181 Operand arg1 = genExp(t->child[0]);
182 Operand arg2 = genExp(t->child[1]);
183 Operand resultTemp = newTemp();
184 //(OP, $t_a, $t_b, $t_c)
185 emitQuad(opTokenToOpKind(t->attr.op), resultTemp, arg1, arg2);
186 return resultTemp;
187 }
188 case AssignK:{
189 Operand argVal = genExp(t->child[1]); //tratamento de constantes,
 operacoes matematicas, uso de variaveis ou calls
190 TreeNode *l_tree = t->child[0];
191 if(l_tree->kind.exp == VarK){
192 Operand argVar = symbOperand(l_tree->symb);
193 //(STORE, var, $t_a, -)
194 emitQuad(OP_STOREVAR, argVar, argVal, EMPTY_OPND);
195 }
196 else if(l_tree->kind.exp == ArrK){
197 Operand argArr = symbOperand(l_tree->symb);
198 Operand argIdx = genExp(l_tree->child[0]);
199 //(STORE, var, $t_a, index)
200 emitQuad(OP_STOREARR, argArr, argVal, argIdx);
201 }
202 return argVal;
203 }
204 case ParamK:{
205 Operand argType = typeOperand(t->type);
206 Operand argVar = symbOperand(t->symb);
207 Operand argScope = scopeOperand(t->scope);
208 //(ARG, type, param, scope)
209 emitQuad(OP_ARG, argType, argVar, argScope);
210 break;
211 }
212 case ParamArrK: {
213 Operand argType = typeOperand(t->type);
214 Operand argArr = symbOperand(t->symb);
215 Operand argScope = scopeOperand(t->scope);
216 emitQuad(OP_ARG_ARR, argType, argArr, argScope);
217 break;
218 }
219 case CallK:{
220 char *funcName = t->symb->name;
221 if (strcmp(funcName, "input") == 0) {
222 Operand resultTemp = newTemp();
223 //(IN, $t_a, -, -)
224 emitQuad(OP_IN, resultTemp, EMPTY_OPND, EMPTY_OPND);
225 return resultTemp;
226 }
227 else if (strcmp(funcName, "output") == 0) {
228 Operand argVal = genExp(t->child[0]);
229 //(OUT, -, arg, -)

```

```

230 emitQuad(OP_OUT, EMPTY_OPND, argVal, EMPTY_OPND);
231 return emptyOperand();
232 }
233 //emissao de quadruplas de params (percorrer t->child[0] e seus irmaos)
234 TreeNode *argNode = t->child[0];
235 int argCount = 0;
236 while(argNode != NULL){
237 Operand argVal = genExp(argNode);
238 emitQuad(OP_PARAM, argVal, EMPTY_OPND, EMPTY_OPND);
239 argCount++;
240 argNode = argNode->sibling;
241 }
242 Operand argFunc = symbOperand(t->symb);
243 Operand argNum = numOperand(argCount);
244 Operand resultTemp = newTemp();
245 //(CALL, $t_a, func, numParams)
246 emitQuad(OP_CALL, resultTemp, argFunc, argNum);
247 return resultTemp;
248 }
249 default:
250 return emptyOperand();
251 }
252 }
253 }
254
255 static void genStmt(TreeNode *t){
256 if(t != NULL){
257 switch(t->kind.stmt){
258 case IfK:{
259 //tratamento de expressao de condicao
260 TreeNode *condition = t->child[0];
261 Operand labelFalse = newLabel();
262 if(condition->kind.exp == OpK){
263 //tratar ambos os lados da operacao
264 Operand arg1 = genExp(condition->child[0]);
265 Operand arg2 = genExp(condition->child[1]);
266 OpKind branchOp;
267 //inverter operador para branch falso
268 switch(condition->attr.op){
269 case MAIOR:
270 branchOp = OP_BLE;
271 break;
272 case MENOR:
273 branchOp = OP_BGE;
274 break;
275 case IGUAL_IGUAL:
276 branchOp = OP_BNE;
277 break;
278 case DIFERENTE:
279 branchOp = OP_BEQ;
280 break;
281 case MAIOR_IGUAL:
282 branchOp = OP_BLT;
283 break;
284 case MENOR_IGUAL:
285 branchOp = OP_BGT;
286 break;
287 default: break;
288 }
289 //(BRANCH, $t_n, $t_m, L_k)

```

```

290 emitQuad(branchOp, arg1, arg2, labelFalse);
291 }
292 //comparacao implicita com zero
293 else{
294 Operand arg = genExp(condition);
295 Operand zero = numOperand(0);
296 //(BRANCH, $t_n, 0, L_k): branch falso
297 emitQuad(OP_BEQ, arg, zero, labelFalse);
298 }
299 //tratamento do compound do if
300 cGen(t->child[1]);
301 //tratamento do else
302 if(t->child[2] != NULL){
303 Operand labelEnd = newLabel();
304 //JUMP para bloco then nao invadir o bloco else
305 emitQuad(OP_JUMP, labelEnd, EMPTY_OPND, EMPTY_OPND);
306 //else: label para jump falso
307 emitQuad(OP_LABEL, labelFalse, EMPTY_OPND, EMPTY_OPND);
308 cGen(t->child[2]);
309 emitQuad(OP_LABEL, labelEnd, EMPTY_OPND, EMPTY_OPND);
310 } else {
311 //labelFalse para jump de condicao falsa (encerramento do bloco de
312 //comparacao)
313 emitQuad(OP_LABEL, labelFalse, EMPTY_OPND, EMPTY_OPND);
314 }
315 break;
316 }
317 case WhileK:{
318 Operand labelStart = newLabel();
319 Operand labelEnd = newLabel();
320 emitQuad(OP_LABEL, labelStart, EMPTY_OPND, EMPTY_OPND);
321 TreeNode *condition = t->child[0];
322 if(condition->kind.exp == OpK){
323 Operand arg1 = genExp(condition->child[0]);
324 Operand arg2 = genExp(condition->child[1]);
325 OpKind branchOp;
326 switch(condition->attr.op){
327 case MAIOR:
328 branchOp = OP_BLE;
329 break;
330 case MENOR:
331 branchOp = OP_BGE;
332 break;
333 case IGUAL_IGUAL:
334 branchOp = OP_BNE;
335 break;
336 case DIFERENTE:
337 branchOp = OP_BEQ;
338 break;
339 case MAIOR_IGUAL:
340 branchOp = OP_BLT;
341 break;
342 case MENOR_IGUAL:
343 branchOp = OP_BGT;
344 break;
345 default: break;
346 }
347 //(BRANCH, $t_n, $t_m, L_k)
348 emitQuad(branchOp, arg1, arg2, labelEnd);

```

```

349 }
350 //comparacao implicita com zero
351 else{
352 Operand arg = genExp(condition);
353 Operand zero = numOperand(0);
354 //(BRANCH, $t_n, 0, L_k): branch falso
355 emitQuad(OP_BEQ, arg, zero, labelEnd);
356 }
357 //tratamento do compound do while
358 cGen(t->child[1]);
359 //loop
360 emitQuad(OP_JUMP, labelStart, EMPTY_OPND, EMPTY_OPND);
361 //label de fim
362 emitQuad(OP_LABEL, labelEnd, EMPTY_OPND, EMPTY_OPND);
363 break;
364 }
365 case CompoundK:{
366 //tratamento das declaracoes de variaveis
367 cGen(t->child[0]);
368 //tratamento da sequencia de instrucoes
369 cGen(t->child[1]);
370 break;
371 }
372 case FunctBodyK:{
373 //tratamento das variaveis
374 cGen(t->child[0]);
375 //tratamento da sequencia de instrucoes
376 cGen(t->child[1]);
377 break;
378 }
379 case FunctDeclK:{
380 Operand argType = typeOperand(t->type);
381 Operand argFunc = symbOperand(t->symb);
382 emitQuad(OP_FUNC, argType, argFunc, EMPTY_OPND);
383 cGen(t->child[0]);
384 cGen(t->child[1]);
385 //(ENDFUNC, gcd, -, -)
386 emitQuad(OP_ENDFUNC, argFunc, EMPTY_OPND, EMPTY_OPND);
387 break;
388 }
389 case ReturnK:{
390 if(t->child[0] != NULL){
391 Operand argVal = genExp(t->child[0]); //tratamento de constantes,
392 operacoes matematicas ou variaveis
393 emitQuad(OP_RETURN, argVal, EMPTY_OPND, EMPTY_OPND);
394 }
395 else
396 emitQuad(OP_RETURN, EMPTY_OPND, EMPTY_OPND, EMPTY_OPND);
397 break;
398 }
399 default:
400 break;
401 }
402 }
403 }
404
405 static void cGen(TreeNode *t){
406 if(t != NULL){
407 switch(t->nodeKind){

```

```

408 case StmtK:
409 genStmt(t);
410 break;
411 case ExpK:
412 genExp(t);
413 break;
414 default:
415 break;
416 }
417 cGen(t->sibling);
418 }
419 }
420
421 void generateIntermediateCode(TreeNode *AST){
422 headQuad = NULL;
423 currentQuad = NULL;
424 tempOffset = 0;
425 labelOffset = 0;
426
427 cGen(AST);
428 }
429
430 static void printOperand(FILE *listing, Operand op){
431 switch(op.kind){
432 case OPND_EMPTY:
433 fprintf(listing, "-");
434 break;
435
436 case OPND_NUM:
437 fprintf(listing, "%d", op.content.imm);
438 break;
439
440 case OPND_TEMP:
441 fprintf(listing, "$t%d", op.content.reg_id);
442 break;
443
444 case OPND_LABEL:
445 fprintf(listing, "%s", op.content.label_name);
446 break;
447
448 case OPND_TYPE:
449 fprintf(listing, "%s", op.content.type_name);
450 break;
451
452 case OPND_SCOPE:
453 fprintf(listing, "%s", op.content.scope_name);
454 break;
455
456 case OPND_SYMB:
457 if (op.content.s_node != NULL) {
458 fprintf(listing, "%s", op.content.s_node->name);
459 } else {
460 fprintf(listing, "?NULL_SYMB?");
461 }
462 break;
463
464 default:
465 fprintf(listing, "?");
466 break;
467 }

```

```
468 }
469 void printIntermediateCode(FILE *listing){
470 Quad *curr = headQuad;
471
472 while(curr != NULL){
473 fprintf(listing, "(");
474 fprintf(listing, "%s, ", opKindToString(curr->op));
475
476 printOperand(listing, curr->arg1);
477 fprintf(listing, ", ");
478
479 printOperand(listing, curr->arg2);
480 fprintf(listing, ", ");
481
482 printOperand(listing, curr->arg3);
483 fprintf(listing, ")\n");
484
485 curr = curr->next;
486 }
487 }
```

## APÊNDICE J – assembly.h

Código-fonte do *header* do gerador de código *assembly* (assembly.h)

```

1 #ifndef ASSEMBLY_H
2 #define ASSEMBLY_H
3
4 #include "cgen.h"
5
6 typedef enum {
7 ASM_ADD, ASM_SUB, ASM_MUL, ASM_DIV,
8 ASM_ADDI, ASM_LW, ASM_SW,
9 ASM_BEQ, ASM_BNE, ASM_BLT, ASM_BGT, ASM_BLE, ASM_BGE,
10 ASM_LABEL, ASM_J, ASM_JAL, ASM_JR, ASM_HALT,
11 ASM_IN, ASM_OUT
12 } AsmOp;
13
14 typedef enum {FORMAT_R, FORMAT_I, FORMAT_J, FORMAT_LABEL} AsmFormat;
15
16 typedef struct AsmInstruction {
17 AsmOp op;
18 AsmFormat format;
19
20 union {
21 struct {
22 int rs;
23 int rt;
24 int rd;
25 int shamt;
26 } r;
27 struct {
28 int rs;
29 int rt;
30 int imm;
31 char *label_name;
32 } i;
33 struct {
34 char *target_name;
35 } j;
36 struct {
37 char *label_name;
38 } label;
39 } type;
40
41 struct AsmInstruction *next;
42 } AsmInstr;
43
44 extern AsmInstr *headAsm;
45 extern AsmInstr *currentAsm;
46
47 void generateAssembly(Quad *headGCI);
48 void printAssembly(FILE *listing);
49
50 #endif

```

# APÊNDICE K – assembly.c

Código-fonte do gerador de código *assembly* (assembly.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "globals.h"
5 #include "syntab.h"
6 #include "cgen.h"
7 #include "assembly.h"
8 #include "util.h"
9
10 #define RAM_SIZE 1024
11 #define MAX_PENDING_PARAMS 8
12 AsmInstr *headAsm = NULL;
13 AsmInstr *currentAsm = NULL;
14
15 static int pendingParams[MAX_PENDING_PARAMS];
16 static int pendingParamCount = 0;
17 static int regInUse[27] = {0};
18
19 static AsmInstr* createNode(AsmOp op, AsmFormat format) {
20 AsmInstr *newA = (AsmInstr*)malloc(sizeof(AsmInstr));
21 newA->op = op;
22 newA->format = format;
23 newA->next = NULL;
24
25 if (headAsm == NULL) {
26 headAsm = newA;
27 currentAsm = newA;
28 } else {
29 currentAsm->next = newA;
30 currentAsm = newA;
31 }
32 return newA;
33 }
34
35 void emitAsmR(AsmOp op, int rs, int rt, int rd, int shamt) {
36 AsmInstr *instr = createNode(op, FORMAT_R);
37 instr->type.r.rs = rs;
38 instr->type.r.rt = rt;
39 instr->type.r.rd = rd;
40 instr->type.r.shamt = shamt;
41 }
42
43 void emitAsmI(AsmOp op, int rs, int rt, int imm) {
44 AsmInstr* instr = createNode(op, FORMAT_I);
45 instr->type.i.rs = rs;
46 instr->type.i.rt = rt;
47 instr->type.i.imm = imm;
48 instr->type.i.label_name = NULL;
49 }
50
51 void emitAsmIBranch(AsmOp op, int rs, int rt, char *target) {
52 AsmInstr* instr = createNode(op, FORMAT_I);

```

```

53 instr->type.i.rs = rs;
54 instr->type.i.rt = rt;
55 instr->type.i.imm = 0;
56 instr->type.i.label_name = copyString(target);
57 }
58
59 void emitAsmJ(AsmOp op, char *target) {
60 AsmInstr *instr = createNode(op, FORMAT_J);
61 instr->type.j.target_name = copyString(target);
62 }
63
64 void emitAsmLabel(char *label) {
65 AsmInstr *instr = createNode(ASM_LABEL, FORMAT_LABEL);
66 instr->type.label.label_name = copyString(label);
67 }
68
69 int getPhysicalReg(int virtual_reg) {
70 if (virtual_reg <= 0) return 0; // Fallback
71 return ((virtual_reg - 1) % 26) + 1;
72 }
73
74 static void freeReg(Operand op){
75 if(op.kind == OPND_TEMP){
76 int regPhys = getPhysicalReg(op.content.reg_id);
77 if (regPhys >= 1 && regPhys <= 26)
78 regInUse[regPhys] = 0;
79 }
80 }
81
82 static void markReg(Operand op){
83 if(op.kind == OPND_TEMP){
84 int regPhys = getPhysicalReg(op.content.reg_id);
85 if (regPhys >= 1 && regPhys <= 26)
86 regInUse[regPhys] = 1;
87 }
88 }
89
90 static void updateRegsInUse(Quad *curr){
91 //desmarcar registradores de uso unico
92 switch(curr->op){
93 case OP_ADD: case OP_SUB: case OP_MUL: case OP_DIV:
94 case OP_LOADARR:
95 freeReg(curr->arg2);
96 freeReg(curr->arg3);
97 break;
98 case OP_BEQ: case OP_BNE: case OP_BLT: case OP_BGT: case OP_BLE: case OP_BGE:
99 freeReg(curr->arg1); //rs de comparacao
100 freeReg(curr->arg2); //rt de comparacao
101 break;
102 case OP_STOREVAR: case OP_OUT:
103 freeReg(curr->arg2);
104 break;
105 case OP_STOREARR:
106 freeReg(curr->arg2); //valor
107 freeReg(curr->arg3); //indice
108 break;
109 case OP_PARAM: case OP_RETURN:
110 freeReg(curr->arg1);
111 break;
112 default: break;

```

```

113 }
114
115 //marcar registradores com resultados
116 switch(curr->op) {
117 case OP_ADD: case OP_SUB: case OP_MUL: case OP_DIV:
118 case OP_LOADVAR: case OP_LOADARR: case OP_LOADIMM:
119 case OP_CALL: case OP_IN:
120 markReg(curr->arg1);
121 break;
122 default: break;
123 }
124 }
125
126 static void insertParamList(int reg) {
127 if (pendingParamCount >= MAX_PENDING_PARAMS) {
128 fprintf(stderr, "Erro: quantidade maxima de parametros excedida (%d).\n",
129 MAX_PENDING_PARAMS);
130 exit(1);
131 }
132 pendingParams[pendingParamCount++] = reg;
133 }
134
135 void generateAssembly(Quad *headGCI){
136 Quad *curr = headGCI;
137 headAsm = NULL;
138 currentAsm = NULL;
139 pendingParamCount = 0;
140 char current_func_name[256] = "";
141
142 // registradores especiais MIPS
143 int zero = 0;
144 int temp = 27;
145 int ret = 28;
146 int sp = 29;
147 int fp = 30;
148 int ra = 31;
149
150 emitAsmI(ASM_ADDI, zero, sp, RAM_SIZE-1);
151 emitAsmI(ASM_ADDI, zero, fp, RAM_SIZE-1);
152 emitAsmI(ASM_ADDI, sp, sp, -1);
153 emitAsmJ(ASM_J, "main");
154
155 while(curr != NULL){
156 switch(curr->op){
157 case OP_PARAM: {
158 int rt = getPhysicalReg(curr->arg1.content.reg_id);
159 insertParamList(rt);
160 break;
161 }
162 case OP_FUNC: {
163 Symbol s = curr->arg2.content.s_node;
164 strcpy(current_func_name, s->name);
165 emitAsmLabel(s->name);
166 // $ra salvo no slot 1 do frame
167 if(strcmp(s->name, "main") == 0) break;
168 emitAsmI(ASM_SW, fp, ra, -1);
169 break;
170 }
171 case OP_ENDFUNC: {
172 if (strcmp(current_func_name, "main") == 0) {

```

```

172 emitAsmI(ASM_HALT, zero, zero, 0);
173 } else {
174 char end_label[300];
175 sprintf(end_label, "END_%s", current_func_name);
176 emitAsmLabel(end_label);
177
178 //restaura $ra
179 emitAsmI(ASM_LW, fp, ra, -1);
180 //ajusta sp para a posicao: $sp original estava exata 1 posicao acima
 do ofp
181 emitAsmI(ASM_ADDI, fp, sp, 1);
182 //restaura $ofp
183 emitAsmI(ASM_LW, fp, fp, 0);
184 //jr para retorno
185 emitAsmR(ASM_JR, ra, zero, zero, 0);
186 }
187 break;
188 }
189 case OP_RETURN: {
190 if (curr->arg1.kind != OPND_EMPTY) {
191 int rt = getPhysicalReg(curr->arg1.content.reg_id);
192 emitAsmR(ASM_ADD, rt, zero, ret, 0);
193 }
194
195 //jump para fim de funcao
196 char end_label[300];
197 sprintf(end_label, "END_%s", current_func_name);
198 emitAsmJ(ASM_J, end_label);
199 break;
200 }
201 case OP_CALL: {
202 Symbol s = curr->arg2.content.s_node;
203 int numParams = curr->arg3.content.imm;
204
205 int savedRegs[27] = {0};
206 int countSavedRegs = 0;
207
208 for(int i=1; i<=26; i++){
209 if(regInUse[i] == 1){
210 savedRegs[i] = 1;
211 countSavedRegs++;
212 }
213 }
214
215 if(countSavedRegs > 0){
216 emitAsmI(ASM_ADDI, sp, sp, -countSavedRegs);
217 int offset = 0;
218 for(int i=1; i<=26; i++){
219 if(savedRegs[i] == 1){
220 emitAsmI(ASM_SW, sp, i, offset);
221 offset++;
222 }
223 }
224 }
225
226 //ofp armazenado
227 emitAsmI(ASM_ADDI, sp, sp, -1);
228 emitAsmI(ASM_SW, sp, fp, 0);
229
230 //fip deslocado para a base

```

```

231 emitAsmR(ASM_ADD, sp, zero, fp, 0);
232 //reservar espaco do $ra
233 emitAsmI(ASM_ADDI, sp, sp, -1);
234
235 //armazenar os parametros sequencialmente
236 int startIdx = pendingParamCount - numParams;
237 for (int i = startIdx; i < pendingParamCount; i++) {
238 emitAsmI(ASM_ADDI, sp, sp, -1);
239 emitAsmI(ASM_SW, sp, pendingParams[i], 0);
240 }
241 pendingParamCount -= numParams;
242
243 // jal funct
244 emitAsmJ(ASM_JAL, s->name);
245
246 //salvar valor retornado em um registrador
247 if(s->type != Void){
248 int rt_return = getPhysicalReg(curr->arg1.content.reg_id);
249 emitAsmR(ASM_ADD, ret, zero, rt_return, 0);
250 }
251
252 //carregar registradores armazenados
253 if(countSavedRegs > 0){
254 int offset = 0;
255 for(int i=1; i<=26; i++){
256 if(savedRegs[i]==1){
257 emitAsmI(ASM_LW, sp, i, offset);
258 offset++;
259 }
260 }
261 emitAsmI(ASM_ADDI, sp, sp, countSavedRegs);
262 }
263 break;
264 }
265 case OP_ALLOCVAR: {
266 Symbol s = curr->arg1.content.s_node;
267 if(strcmp(s->scope, "global") == 0){
268 //global: o endereco e estatico (s->memloc)
269 } else {
270 // addi $sp, $sp, -1
271 emitAsmI(ASM_ADDI, sp, sp, -1);
272 }
273 break;
274 }
275 case OP_ALLOCARR: {
276 Symbol s = curr->arg1.content.s_node;
277 int size = curr->arg2.content.imm;
278 if(strcmp(s->scope, "global") == 0){
279 //global: o endereco e estatico (s->memloc)
280 } else {
281 //reserva espaco do ponteiro e do array
282 emitAsmI(ASM_ADDI, sp, sp, -(size+2));
283 //primeiro slot da array (addr=fp+frameloc) e o endereco base (vet
284 [0])
285 int frameloc = -(s->memloc + 2);
286 emitAsmI(ASM_SW, fp, sp, frameloc);
287 }
288 break;
289 }
290 case OP_STOREVAR: {

```

```

290 Symbol s = curr->arg1.content.s_node;
291 int rt = getPhysicalReg(curr->arg2.content.reg_id);
292
293 if (strcmp(s->scope, "global") == 0) {
294 emitAsmI(ASM_SW, zero, rt, s->memloc); //insercao na base da memoria
295 } else {
296 int frameloc = -(s->memloc + 2);
297 emitAsmI(ASM_SW, fp, rt, frameloc);
298 }
299 break;
300 }
301 case OP_STOREARR: {
302 Symbol s = curr->arg1.content.s_node;
303 int rs_value = getPhysicalReg(curr->arg2.content.reg_id);
304 int rs_index = getPhysicalReg(curr->arg3.content.reg_id);
305
306 if (strcmp(s->scope, "global") == 0) {
307 emitAsmR(ASM_ADD, zero, rs_index, temp, 0); // temp = index
308 emitAsmI(ASM_SW, temp, rs_value, s->memloc); // mem[index + memloc] =
 value
309 } else {
310 int frameloc = -(s->memloc+2);
311 //carrega o ponteiro da array para o $temp
312 emitAsmI(ASM_LW, fp, temp, frameloc);
313 //temp = temp + index
314 emitAsmR(ASM_ADD, temp, rs_index, temp, 0);
315 //guarda o valor no endereco exato
316 emitAsmI(ASM_SW, temp, rs_value, 0); // sw value, 0(temp)
317 }
318 break;
319 }
320 case OP_LOADVAR: {
321 Symbol s = curr->arg2.content.s_node;
322 int rt_dest = getPhysicalReg(curr->arg1.content.reg_id);
323
324 if (s->kind == SYMB_ARR) {
325 if (strcmp(s->scope, "global") == 0) {
326 emitAsmI(ASM_ADDI, zero, rt_dest, s->memloc); //endereco = 0 +
 memloc
327 } else {
328 int frameloc = -(s->memloc + 2);
329 emitAsmI(ASM_LW, fp, rt_dest, frameloc);
330 }
331 } else {
332 if (strcmp(s->scope, "global") == 0) {
333 emitAsmI(ASM_LW, zero, rt_dest, s->memloc);
334 } else {
335 int frameloc = -(s->memloc+2);
336 emitAsmI(ASM_LW, fp, rt_dest, frameloc);
337 }
338 }
339 break;
340 }
341 case OP_LOADARR: {
342 Symbol s = curr->arg2.content.s_node;
343 int rt_dest = getPhysicalReg(curr->arg1.content.reg_id);
344 int rs_index = getPhysicalReg(curr->arg3.content.reg_id);
345
346 if (strcmp(s->scope, "global") == 0) {
347 emitAsmR(ASM_ADD, zero, rs_index, temp, 0); //temp = index

```

```

348 emitAsmI(ASM_LW, temp, rt_dest, s->memloc); //dest = mem[index +
 memloc]
349 } else {
350 int frameloc = -(s->memloc+2);
351 //carrega o ponteiro do frame para o $temp
352 emitAsmI(ASM_LW, fp, temp, frameloc);
353 //temp = temp + index
354 emitAsmR(ASM_ADD, temp, rs_index, temp, 0);
355 //le o valor no endereco exato
356 emitAsmI(ASM_LW, temp, rt_dest, 0); // lw dest, 0(temp)
357 }
358 break;
359 }
360 case OP_LOADIMM: {
361 int rt_dest = getPhysicalReg(curr->arg1.content.reg_id);
362 int value = curr->arg2.content.imm;
363 // addi rt, $0, value
364 emitAsmI(ASM_ADDI, zero, rt_dest, value);
365 break;
366 }
367 case OP_ADD: {
368 int rd = getPhysicalReg(curr->arg1.content.reg_id);
369 int rs = getPhysicalReg(curr->arg2.content.reg_id);
370 int rt = getPhysicalReg(curr->arg3.content.reg_id);
371 emitAsmR(ASM_ADD, rs, rt, rd, 0);
372 break;
373 }
374 case OP_SUB: {
375 int rd = getPhysicalReg(curr->arg1.content.reg_id);
376 int rs = getPhysicalReg(curr->arg2.content.reg_id);
377 int rt = getPhysicalReg(curr->arg3.content.reg_id);
378 emitAsmR(ASM_SUB, rs, rt, rd, 0);
379 break;
380 }
381 case OP_MUL: {
382 int rd = getPhysicalReg(curr->arg1.content.reg_id);
383 int rs = getPhysicalReg(curr->arg2.content.reg_id);
384 int rt = getPhysicalReg(curr->arg3.content.reg_id);
385 emitAsmR(ASM_MUL, rs, rt, rd, 0);
386 break;
387 }
388 case OP_DIV: {
389 int rd = getPhysicalReg(curr->arg1.content.reg_id);
390 int rs = getPhysicalReg(curr->arg2.content.reg_id);
391 int rt = getPhysicalReg(curr->arg3.content.reg_id);
392 emitAsmR(ASM_DIV, rs, rt, rd, 0);
393 break;
394 }
395 case OP_JUMP: {
396 // j target
397 emitAsmJ(ASM_J, curr->arg1.content.label_name);
398 break;
399 }
400 case OP_LABEL: {
401 emitAsmLabel(curr->arg1.content.label_name);
402 break;
403 }
404 case OP_BEQ: {
405 int rs = getPhysicalReg(curr->arg1.content.reg_id);
406 int rt = getPhysicalReg(curr->arg2.content.reg_id);

```

```

407 emitAsmIBranch(ASM_BEQ, rs, rt, curr->arg3.content.label_name);
408 break;
409 }
410 case OP_BNE: {
411 int rs = getPhysicalReg(curr->arg1.content.reg_id);
412 int rt = getPhysicalReg(curr->arg2.content.reg_id);
413 emitAsmIBranch(ASM_BNE, rs, rt, curr->arg3.content.label_name);
414 break;
415 }
416 case OP_BGT: {
417 int rs = getPhysicalReg(curr->arg1.content.reg_id);
418 int rt = getPhysicalReg(curr->arg2.content.reg_id);
419 emitAsmIBranch(ASM_BGT, rs, rt, curr->arg3.content.label_name);
420 break;
421 }
422 case OP_BLT: {
423 int rs = getPhysicalReg(curr->arg1.content.reg_id);
424 int rt = getPhysicalReg(curr->arg2.content.reg_id);
425 emitAsmIBranch(ASM_BLT, rs, rt, curr->arg3.content.label_name);
426 break;
427 }
428 case OP_BGE: {
429 int rs = getPhysicalReg(curr->arg1.content.reg_id);
430 int rt = getPhysicalReg(curr->arg2.content.reg_id);
431 emitAsmIBranch(ASM_BGE, rs, rt, curr->arg3.content.label_name);
432 break;
433 }
434 case OP_BLE: {
435 int rs = getPhysicalReg(curr->arg1.content.reg_id);
436 int rt = getPhysicalReg(curr->arg2.content.reg_id);
437 emitAsmIBranch(ASM_BLE, rs, rt, curr->arg3.content.label_name);
438 break;
439 }
440 case OP_IN: {
441 int rt_dest = getPhysicalReg(curr->arg1.content.reg_id);
442 emitAsmI(ASM_IN, zero, rt_dest, 0);
443 break;
444 }
445 case OP_OUT: {
446 int rs_src = getPhysicalReg(curr->arg2.content.reg_id);
447 emitAsmI(ASM_OUT, rs_src, zero, 0);
448 break;
449 }
450 case OP_HALT: {
451 emitAsmI(ASM_HALT, zero, zero, 0);
452 break;
453 }
454 default: break;
455 }
456 updateRegsInUse(curr);
457 curr = curr->next;
458 }
459 }
460
461 static const char* asmOpToString(AsmOp op){
462 switch(op){
463 case ASM_ADD: return "add";
464 case ASM_SUB: return "sub";
465 case ASM_MUL: return "mul";
466 case ASM_DIV: return "div";

```

```

467 case ASM_ADDI: return "addi";
468 case ASM_LW: return "lw";
469 case ASM_SW: return "sw";
470 case ASM_BEQ: return "beq";
471 case ASM_BNE: return "bne";
472 case ASM_BLT: return "blt";
473 case ASM_BGT: return "bgt";
474 case ASM_BLE: return "ble";
475 case ASM_BGE: return "bge";
476 case ASM_J: return "j";
477 case ASM_JAL: return "jal";
478 case ASM_JR: return "jr";
479 case ASM_HALT: return "halt";
480 case ASM_IN: return "in";
481 case ASM_OUT: return "out";
482 default: return "unknown";
483 }
484 }
485
486 static const char* printReg(int reg_id){
487 static const char* reg_names[32] = {
488 "$zero",
489 "$t1", "$t2", "$t3", "$t4", "$t5", "$t6", "$t7", "$t8", "$t9", "$t10", "$t11", "
 $t12", "$t13", "$t14", "$t15", "$t16", "$t17", "$t18", "$t19", "$t20", "$t21"
 , "$t22", "$t23", "$t24", "$t25", "$t26", "$tmp", "$ret",
490 "$sp", // $sp = $29
491 "$fp", // $fp = #30
492 "$ra" // $ra = $31
493 };
494
495 if(reg_id >= 0 && reg_id <= 31){
496 return reg_names[reg_id];
497 }
498
499 return "$?"; //fallback
500 }
501
502 void printAssembly(FILE *listing){
503 AsmInstr *curr = headAsm;
504
505 while(curr != NULL){
506 if(curr->format == FORMAT_LABEL) {
507 fprintf(listing, "%s:\n", curr->type.label.label_name);
508 }
509 else {
510 fprintf(listing, " ");
511 const char* opName = asmOpToString(curr->op);
512
513 switch (curr->format) {
514 case FORMAT_R:{
515 if (curr->op == ASM_JR) {
516 fprintf(listing, "%s %s\n", opName, printReg(curr->type.r.rs));
517 } else {
518 // op rd, rs, rt
519 fprintf(listing, "%s %s, %s, %s\n", opName, printReg(curr->type.r
 .rd), printReg(curr->type.r.rs), printReg(curr->type.r.rt));
520 }
521 break;
522 }
523 case FORMAT_I:{

```

```

524 if (curr->op == ASM_IN) {
525 fprintf(listing, "%s %s\n", opName, printReg(curr->type.i.rt));
526 }
527 else if (curr->op == ASM_OUT) {
528 fprintf(listing, "%s %s\n", opName, printReg(curr->type.i.rs));
529 }
530 else if (curr->op == ASM_LW || curr->op == ASM_SW) {
531 // memoria: op rt, offset(rs) (Ex: lw $1, -2($30))
532 fprintf(listing, "%s %s, %d(%s)\n", opName, printReg(curr->type.i
 .rt), curr->type.i.imm, printReg(curr->type.i.rs));
533 }
534 else if (curr->op == ASM_BEQ || curr->op == ASM_BNE || curr->op ==
 ASM_BLT || curr->op == ASM_BGT || curr->op == ASM_BLE || curr->op
 == ASM_BGE) {
535 // branch: op rs, rt, offset (Ex: beq $1, $2, 15)
536 fprintf(listing, "%s %s, %s, %s\n", opName, printReg(curr->type.i
 .rs), printReg(curr->type.i.rt), curr->type.i.label_name);
537 }
538 else if (curr->op == ASM_HALT) {
539 fprintf(listing, "halt\n");
540 } else {
541 // imediato: op rt, rs, imm (Ex: addi $1, $0, 5)
542 fprintf(listing, "%s %s, %s, %d\n", opName, printReg(curr->type.i
 .rt), printReg(curr->type.i.rs), curr->type.i.imm);
543 }
544 break;
545 }
546 case FORMAT_J:{
547 // tipo J: op target (Ex: j L1 / jal main)
548 fprintf(listing, "%s %s\n", opName, curr->type.j.target_name);
549 break;
550 }
551 default: break;
552 }
553 }
554 curr = curr->next;
555 }
556 }

```

## APÊNDICE L – binary.h

Código-fonte do *header* do gerador de código executável (`binary.h`)

```
1 #ifndef BINARY_H
2 #define BINARY_H
3
4 #include <stdio.h>
5 #include "assembly.h"
6
7 void generateBinary(AsmInstr *head, FILE *out);
8
9 #endif
```

# APÊNDICE M – binary.c

Código-fonte do gerador de código executável (binary.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdint.h>
5 #include "assembly.h"
6
7 //tipo R
8 #define OPCODE_R_TYPE 0b0000000
9 #define FUNCT_ADD 0b0000000
10 #define FUNCT_SUB 0b0000001
11 #define FUNCT_MUL 0b0000010
12 #define FUNCT_DIV 0b0000011
13 #define FUNCT_JR 0b001000
14
15 //tipo I
16 #define OPCODE_ADDI 0b0000001
17 #define OPCODE_LW 0b0000101
18 #define OPCODE_SW 0b0000110
19 #define OPCODE_BEQ 0b0001000
20 #define OPCODE_BNE 0b0001001
21 #define OPCODE_BLT 0b0001010
22 #define OPCODE_BGT 0b0001011
23 #define OPCODE_BLE 0b0001100
24 #define OPCODE_BGE 0b0001101
25 #define OPCODE_IN 0b0001110
26 #define OPCODE_OUT 0b0001111
27 #define OPCODE_HALT 0b010011
28
29 //tipo J
30 #define OPCODE_J 0b010000
31 #define OPCODE_JAL 0b010001
32
33 typedef struct LabelNode {
34 char *name;
35 int address; //endereço ROM
36 struct LabelNode *next;
37 } LabelNode;
38
39 static LabelNode *labelList = NULL;
40
41 static void insertLabel(char *name, int address) {
42 LabelNode *newNode = (LabelNode*)malloc(sizeof(LabelNode));
43 newNode->name = strdup(name);
44 newNode->address = address;
45 newNode->next = labelList;
46 labelList = newNode;
47 }
48
49 static int getLabelAddress(char *name) {
50 LabelNode *curr = labelList;
51 while (curr != NULL) {
52 if (strcmp(curr->name, name) == 0)

```

```

53 return curr->address;
54 curr = curr->next;
55 }
56 return -1;
57 }
58
59 static void printBinaryString(FILE *out, uint32_t instruction) {
60 for (int i = 31; i >= 0; i--) {
61 int bit = (instruction >> i) & 1;
62 fprintf(out, "%d", bit);
63 }
64 fprintf(out, "\n");
65 }
66
67 static void mapLabels(AsmInstr *head) {
68 AsmInstr *curr = head;
69 int rom_address = 0;
70
71 while (curr != NULL) {
72 if (curr->format == FORMAT_LABEL) {
73 insertLabel(curr->type.label.label_name, rom_address);
74 } else {
75 rom_address++;
76 }
77 curr = curr->next;
78 }
79 }
80
81 static void generateMachineCode(AsmInstr *head, FILE *out) {
82 AsmInstr *curr = head;
83 int rom_address = 0;
84
85 while (curr != NULL) {
86 if (curr->format == FORMAT_LABEL) {
87 curr = curr->next;
88 continue; //labels nao geram binario
89 }
90
91 uint32_t machine_code = 0;
92 uint32_t opcode = 0, rs = 0, rt = 0, rd = 0, shamt = 0, funct = 0, imm = 0,
93 address = 0;
94
95 switch (curr->format) {
96 case FORMAT_R: {
97 // MIPS Tipo R: [opcode:6] [rs:5] [rt:5] [rd:5] [shamt:5] [funct:6]
98 opcode = OPCODE_R_TYPE;
99 rs = curr->type.r.rs;
100 rt = curr->type.r.rt;
101 rd = curr->type.r.rd;
102 shamt = curr->type.r.shamt;
103
104 switch (curr->op) {
105 case ASM_ADD:{
106 funct = FUNCT_ADD;
107 break;
108 }
109 case ASM_SUB:{
110 funct = FUNCT_SUB;
111 break;
112 }
113 }
114 }
115 }
116 }
117 }

```

```

112 case ASM_MUL:{
113 funct = FUNCT_MUL;
114 break;
115 }
116 case ASM_DIV:{
117 funct = FUNCT_DIV;
118 break;
119 }
120 case ASM_JR:{
121 funct = FUNCT_JR;
122 break;
123 }
124 default: break;
125 }
126 machine_code = (opcode << 26) | (rs << 21) | (rt << 16) | (rd << 11) | (
127 shamt << 6) | funct;
128 break;
129 }
130
131 case FORMAT_I: {
132 // MIPS Tipo I: [opcode:6] [rs:5] [rt:5] [immediate:16]
133 rs = curr->type.i.rs;
134 rt = curr->type.i.rt;
135
136 switch (curr->op) {
137 case ASM_ADDI:{
138 opcode = OPCODE_ADDI;
139 imm = curr->type.i.imm & 0xFFFF;
140 break;
141 }
142 case ASM_LW:{
143 opcode = OPCODE_LW;
144 imm = curr->type.i.imm & 0xFFFF;
145 break;
146 }
147 case ASM_SW:{
148 opcode = OPCODE_SW;
149 imm = curr->type.i.imm & 0xFFFF;
150 break;
151 }
152 case ASM_IN:{
153 opcode = OPCODE_IN;
154 imm = 0;
155 break;
156 }
157 case ASM_OUT:{
158 opcode = OPCODE_OUT;
159 imm = 0;
160 break;
161 }
162 case ASM_HALT:{
163 opcode = OPCODE_HALT;
164 imm = 0;
165 break;
166 }
167 case ASM_BEQ:
168 case ASM_BNE:
169 case ASM_BLT:
170 case ASM_BGT:
171 case ASM_BLE:

```

```

171 case ASM_BGE:
172 if (curr->op == ASM_BEQ) opcode = OPCODE_BEQ;
173 else if (curr->op == ASM_BNE) opcode = OPCODE_BNE;
174 else if (curr->op == ASM_BLT) opcode = OPCODE_BLT;
175 else if (curr->op == ASM_BGT) opcode = OPCODE_BGT;
176 else if (curr->op == ASM_BLE) opcode = OPCODE_BLE;
177 else if (curr->op == ASM_BGE) opcode = OPCODE_BGE;
178
179 //calculo offset
180 int target_address = getLabelAddress(curr->type.i.label_name);
181 int offset = target_address - rom_address - 1;
182 imm = offset & 0xFFFF;
183 break;
184
185 default: break;
186 }
187
188 machine_code = (opcode << 26) | (rs << 21) | (rt << 16) | imm;
189 break;
190 }
191
192 case FORMAT_J: {
193 //MIPS Tipo J: [opcode:6] [address:26]
194 switch (curr->op) {
195 case ASM_J:{
196 opcode = OPCODE_J;
197 break;
198 }
199 case ASM_JAL:{
200 opcode = OPCODE_JAL;
201 break;
202 }
203 default: break;
204 }
205
206 address = getLabelAddress(curr->type.j.target_name);
207 machine_code = (opcode << 26) | (address & 0x3FFFFFFF);
208 break;
209 }
210
211 default: break;
212 }
213
214 printBinaryString(out, machine_code);
215 rom_address++;
216 curr = curr->next;
217 }
218 }
219
220 void generateBinary(AsmInstr *head, FILE *out) {
221 mapLabels(head);
222 generateMachineCode(head, out);
223 }

```

## APÊNDICE N – util.h

Código-fonte do *header* de utilitários (util.h)

```
1 #ifndef _UTIL_H_
2 #define _UTIL_H_
3
4 #include "globals.h"
5
6 TreeNode * newStmtNode(StmtKind kind);
7 TreeNode * newExpNode(ExpKind kind);
8
9 char * copyString(const char * s);
10 const char* opStr(TokenType op);
11
12
13 void printTreeGraphviz(TreeNode *tree);
14 #endif
```

# APÊNDICE O – util.c

Código-fonte de utilitários (util.c)

```

1 #include "globals.h"
2 #include "util.h"
3 #include "parser.tab.h"
4
5
6 //Louden(2004, p.507-9)
7 TreeNode* newStmtNode(StmtKind kind) {
8 TreeNode* t = (TreeNode*) malloc(sizeof(TreeNode));
9 int i;
10
11 if (t == NULL) {
12 fprintf(stderr, "Erro de memoria na lineNo %d\n", lineNo);
13 exit(1);
14 } else {
15 for (i = 0; i < MAXCHILDREN; i++) t->child[i] = NULL;
16 t->sibling = NULL;
17 t->nodeKind = StmtK;
18 t->kind.stmt = kind;
19 t->lineNo = lineNo;
20 t->scope = NULL;
21 }
22 return t;
23 }
24
25 TreeNode* newExpNode(ExpKind kind) {
26 TreeNode *t = (TreeNode*) malloc(sizeof(TreeNode));
27 int i;
28
29 if (t == NULL) {
30 fprintf(stderr, "Erro de memoria na lineNo %d\n", lineNo);
31 exit(1);
32 } else {
33 for (i = 0; i < MAXCHILDREN; i++) t->child[i] = NULL;
34 t->sibling = NULL;
35 t->nodeKind = ExpK;
36 t->kind.exp = kind;
37 t->lineNo = lineNo;
38 t->scope = NULL;
39 t->type = Void;
40 }
41 return t;
42 }
43
44 char* copyString(const char *s) {
45 int n;
46 char * t;
47
48 if(s == NULL) return NULL;
49
50 n = strlen(s) + 1;
51 t = malloc(n);
52

```

```

53 if(t == NULL) {
54 fprintf(stderr, "Erro de memoria na lineNo %d\n", lineNo);
55 exit(1);
56 } else {
57 strcpy(t, s);
58 }
59 return t;
60 }
61
62 const char* opStr(TokenType op) {
63 switch (op) {
64 case SOMA: return "+";
65 case SUB: return "-";
66 case MUL: return "*";
67 case DIV: return "/";
68 case MENOR: return "<";
69 case MENOR_IGUAL: return "<=";
70 case MAIOR: return ">";
71 case MAIOR_IGUAL: return ">=";
72 case IGUAL_IGUAL: return "==";
73 case DIFERENTE: return "!=";
74 default: return "?";
75 }
76 }
77
78 static const char* typeStr(ExpType t) {
79 switch (t) {
80 case Integer: return "int";
81 case Void: return "void";
82 default: return "";
83 }
84 }
85
86 static void writeNodeLabel(FILE *out, TreeNode *node) {
87 if (node->nodeKind == StmtK) {
88 switch (node->kind.stmt) {
89 case IfK:
90 fprintf(out, "if");
91 break;
92 case WhileK:
93 fprintf(out, "while");
94 break;
95 case CompoundK:
96 fprintf(out, "{ }");
97 break;
98 case FunctBodyK:
99 fprintf(out, "{ }");
100 break;
101 case FunctDeclK:
102 fprintf(out, "Fun: %s()", (node->attr.name != NULL) ? node->attr.name : "
?");
103 break;
104 case ReturnK:
105 fprintf(out, "return");
106 break;
107 default:
108 fprintf(out, "STMT");
109 break;
110 }
111 } else {

```

```

112 switch (node->kind.exp) {
113 case OpK:
114 fprintf(out, "%s", opStr(node->attr.op));
115 break;
116 case ConstK:
117 fprintf(out, "%d", node->attr.val);
118 break;
119 case VarK:
120 fprintf(out, "%s", (node->attr.name != NULL) ? node->attr.name : "?");
121 break;
122 case ArrK:
123 fprintf(out, "%s[]", (node->attr.name != NULL) ? node->attr.name : "?");
124 break;
125 case VarDeclK:
126 fprintf(out, "Var: %s", (node->attr.name != NULL) ? node->attr.name : "?");
127 break;
128 case ArrDeclK:
129 fprintf(out, "Arr: %s[]", (node->attr.name != NULL) ? node->attr.name : "?");
130 break;
131 case ParamK:
132 fprintf(out, "Param: %s", (node->attr.name != NULL) ? node->attr.name : "?");
133 break;
134 case ParamArrK:
135 fprintf(out, "Param: %s[]", (node->attr.name != NULL) ? node->attr.name : "?");
136 break;
137 case AssignK:
138 fprintf(out, "=");
139 break;
140 case CallK:
141 fprintf(out, "Call: %s()", (node->attr.name != NULL) ? node->attr.name : "?");
142 break;
143 default:
144 fprintf(out, "EXP");
145 break;
146 }
147 }
148
149 int isDecl = (node->nodeKind == StmtK && node->kind.stmt == FunctDeclK) ||
150 (node->nodeKind == ExpK && (node->kind.exp == VarDeclK || node->kind.exp == ArrDeclK || node->kind.exp == ParamK || node->kind.exp == ParamArrK));
151
152 if (isDecl) {
153 if (node->type == Integer || node->type == Void) {
154 fprintf(out, "\\n(%s)", typeStr(node->type));
155 }
156 if (node->scope != NULL) {
157 fprintf(out, "\\n[%s]", node->scope);
158 }
159 }
160 }
161
162 static void printTreeGraphvizRec(FILE *out, TreeNode *node) {
163 int i;
164 if (node == NULL) return;

```

```

165
166 const char* fillColor = "lightgray";
167 const char* shape = "box";
168
169 if (node->nodeKind == StmtK) {
170 if (node->kind.stmt == FunctDeclK) fillColor = "lightcoral";
171 else fillColor = "lightblue";
172 } else if (node->nodeKind == ExpK) {
173
174 if (node->kind.exp == OpK) {
175 fillColor = "plum";
176 shape = "circle";
177 }
178 else if (node->kind.exp == ConstK) {
179 fillColor = "lightgrey";
180 shape = "ellipse";
181 }
182 else if (node->kind.exp == AssignK) {
183 fillColor = "gold";
184 shape = "circle";
185 }
186 else if (node->kind.exp == CallK) {
187 fillColor = "aquamarine";
188 }
189 else if (node->kind.exp == VarDeclK || node->kind.exp == ArrDeclK || node->kind.
190 exp == ParamK || node->kind.exp == ParamArrK) {
191 fillColor = "palegreen";
192 }
193 else {
194 fillColor = "lightyellow";
195 shape = "ellipse";
196 }
197 }
198
199 fprintf(out, " node%p [shape=\"%s\", fillcolor=\"%s\", label=\"%s\", (void*)node, shape
200 , fillColor);
201 writeNodeLabel(out, node);
202 fprintf(out, "\"];\n");
203
204 //impressao filhos
205 for (i = 0; i < MAXCHILDREN; i++) {
206 if (node->child[i] != NULL) {
207 fprintf(out, " node%p -> node%p;\n", (void*)node, (void*)node->child[i]);
208 printTreeGraphvizRec(out, node->child[i]);
209 }
210 }
211
212 //impressao irmaos
213 if (node->sibling != NULL) {
214 fprintf(out, " node%p -> node%p [style=dashed, color=gray40];\n", (void*)node, (
215 void*)node->sibling);
216 fprintf(out, " { rank=same; node%p; node%p; }\n", (void*)node, (void*)node->
217 sibling);
218 printTreeGraphvizRec(out, node->sibling);
219 }
220 }
221
222 void printTreeGraphviz(TreeNode *AST){
223 FILE *out;

```

```
221 if (AST == NULL) {
222 fprintf(stderr, "AST vazia: nenhum arquivo Graphviz foi gerado.\n");
223 return;
224 }
225
226 out = fopen("output_files/arvore.dot", "w");
227 if (out == NULL) {
228 fprintf(stderr, "Erro ao criar arquivo arvore.dot\n");
229 return;
230 }
231
232 fprintf(out, "digraph AST {\n");
233 fprintf(out, " rankdir=TB;\n");
234 fprintf(out, " node [shape=box, style=\"rounded, filled\", fillcolor=\"lightgoldenrod1\", fontname=\"Consolas\"]; \n");
235 fprintf(out, " edge [fontname=\"Consolas\"]; \n");
236
237 printTreeGraphvizRec(out, AST);
238
239 fprintf(out, "}\n");
240 fclose(out);
241
242 printf("Arquivo Graphviz gerado: output_files/arvore.dot\n");
243 }
```

# APÊNDICE P – main.c

Código-fonte do módulo principal (main.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "globals.h"
4 #include "util.h"
5 #include "analyze.h"
6 #include "symtab.h"
7 #include "cgen.h"
8 #include "assembly.h"
9 #include "binary.h"
10
11 extern FILE *yyin;
12 extern int yyparse(void);
13 extern TreeNode *AST;
14 extern Quad *headQuad;
15
16 int lineNo = 1;
17 FILE *source = NULL;
18
19 int main(int argc, char **argv) {
20 int parseStatus;
21
22 if (argc < 2) {
23 fprintf(stderr, "Uso: %s <arquivo_entrada>\n", argv[0]);
24 return 1;
25 }
26
27 source = fopen(argv[1], "r");
28 if (source == NULL) {
29 fprintf(stderr, "Erro ao abrir arquivo de entrada: '%s'\n", argv[1]);
30 return 1;
31 }
32
33 yyin = source;
34 lineNo = 1;
35
36 //analise lexica e sintatica
37 parseStatus = yyparse();
38
39 fclose(source);
40 source = NULL;
41
42 if (parseStatus != 0)
43 return 1;
44
45 if (AST != NULL) {
46 //analise semantica
47 buildSymtab(AST);
48 typeCheck(AST);
49 generateIntermediateCode(AST);
50 printTreeGraphviz(AST);
51 generateAssembly(headQuad);
52 }
```

```
53 FILE *symtabFile = fopen("output_files/tabela_simbolos.txt", "w");
54 FILE *gciFile = fopen("output_files/codigo_intermediario.txt", "w");
55 FILE *assemblyFile = fopen("output_files/codigo_assembly.txt", "w");
56 FILE *binFile = fopen("output_files/codigo_binario.txt", "w");
57 if (symtabFile == NULL || gciFile == NULL) {
58 fprintf(stderr, "Erro ao criar arquivo. Verifique se a pasta 'output_files'
59 existe\n");
60 } else {
61 printSymTab(symtabFile);
62 fclose(symtabFile);
63 printf("Tabela de Simbolos gerada: output_files/tabela_simbolos.txt\n");
64 printIntermediateCode(gciFile);
65 fclose(gciFile);
66 printf("Codigo intermediario gerado: output_files/codigo_intermediario.txt\n"
67);
68 printAssembly(assemblyFile);
69 fclose(assemblyFile);
70 printf("Codigo assembly gerado: output_files/codigo_assembly.txt\n");
71 generateBinary(headAsm, binFile);
72 printf("Codigo binario gerado: output_files/codigo_binario.txt\n");
73 fclose(binFile);
74 }
75 return 0;
76 }
```